

- 两个任务都需要变量声明。在网络管理中，它由MIB处理。
- 两个任务都需要用语句来执行动作。在网络管理中，它由SNMP处理。

概要

在详细地讨论每个组件之前，我们用一个简单的情况说明每个组件涉及到的内容，这仅是概要的内容，在本章末将展开详述。管理器站点（SNMP客户）想要发送一个报文给代理站点（SNMP服务器）查找代理接收到UDP用户数据报的个数。图28.4表示每步的概要。

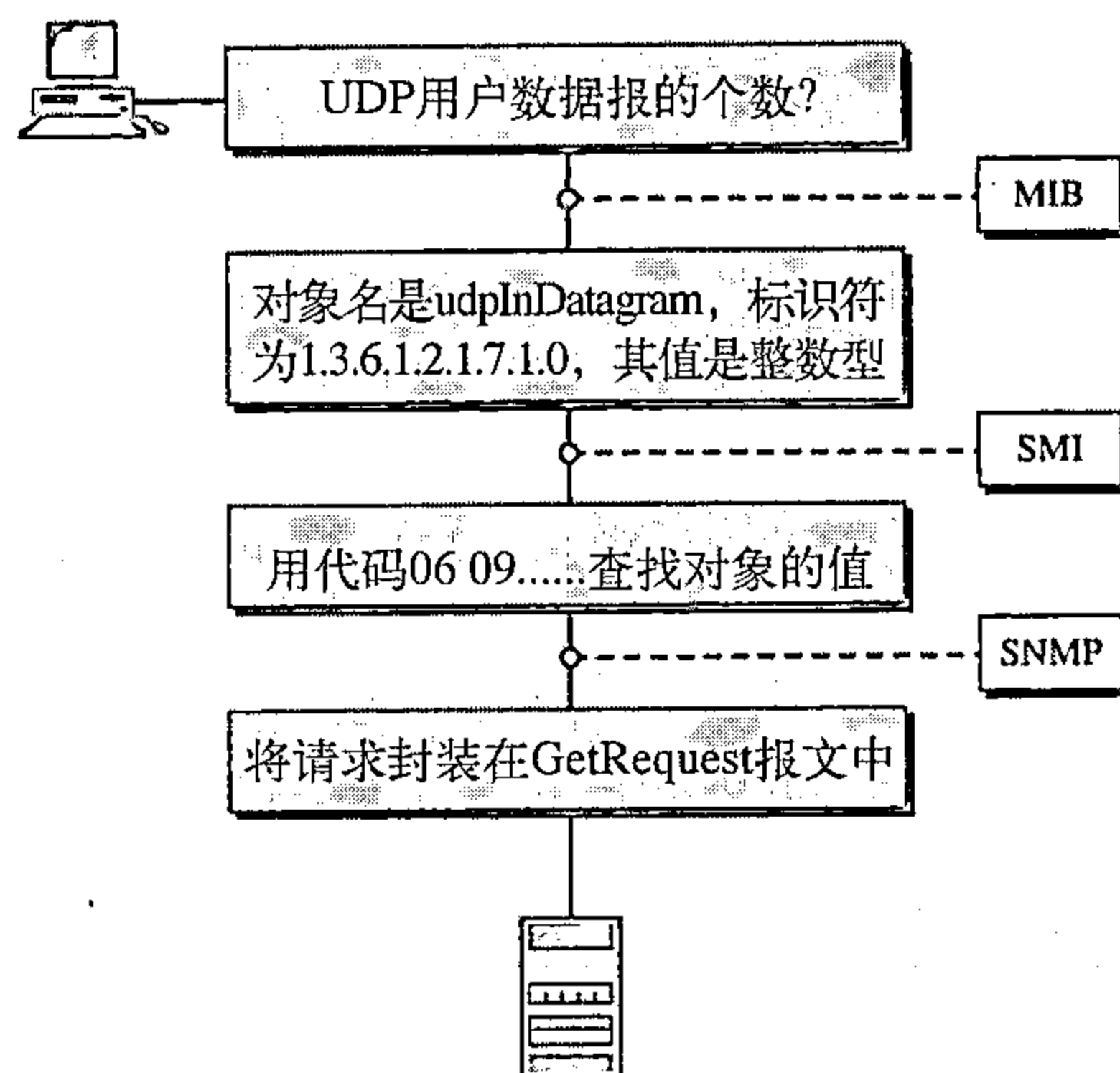


图28.4 管理概要

MIB负责查找保存接收到的UDP用户数据报个数的对象。在另一个嵌入协议的帮助下，SMI负责给对象名编码。SNMP负责创建一个称为GetRequest的报文，并封装已编码的报文。实际上，事情比概述复杂得多，但是，我们必须对每个协议有更详细的叙述。

28.2.3 管理信息结构

管理信息结构第二版（SMIv2）是网络管理中的一个组件。它的功能是：

1. 给对象命名；
2. 定义可在对象中存储的数据类型；
3. 给出如何对在网络上传输的数据

进行编码的方法。

SMI是SNMP的一个指南。它强调了处理对象的三个属性：名字、数据类型和编码方法（如图28.5所示）。

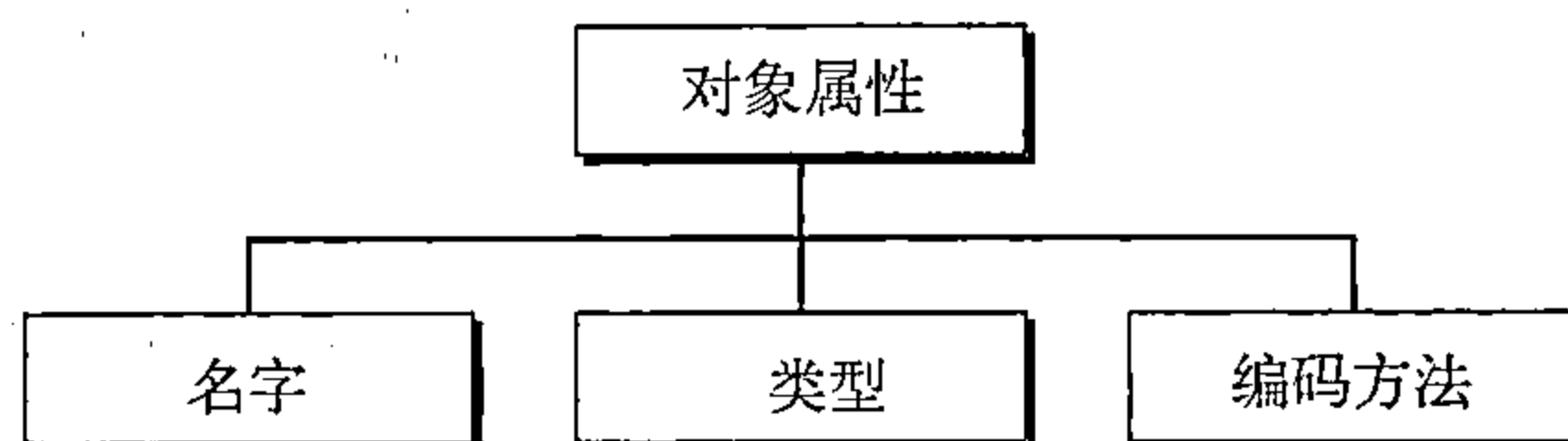


图28.5 对象属性

名字

SMI要求每一个被管理对象（如一个路由器、一个路由器中的变量、一个值）具有一个唯一的名字。为了在全局给对象命名，SMI使用对象标识符（object identifier），它是基于树结构的分层次标识符（如图28.6所示）。

树结构从一个未命名的根开始。每一个对象可用一个点分隔开的整数序列来定义，树结构也可用一个点分隔开名字的文本序列来定义。整数-点的表示法是SNMP使用的一种方法，人们使用名字-点表示法。例如，下面是同一对象的两不同的表示法：

iso.org.dod.internet.mgmt.mib-2 $\Rightarrow$ 1.3.6.1.2.1

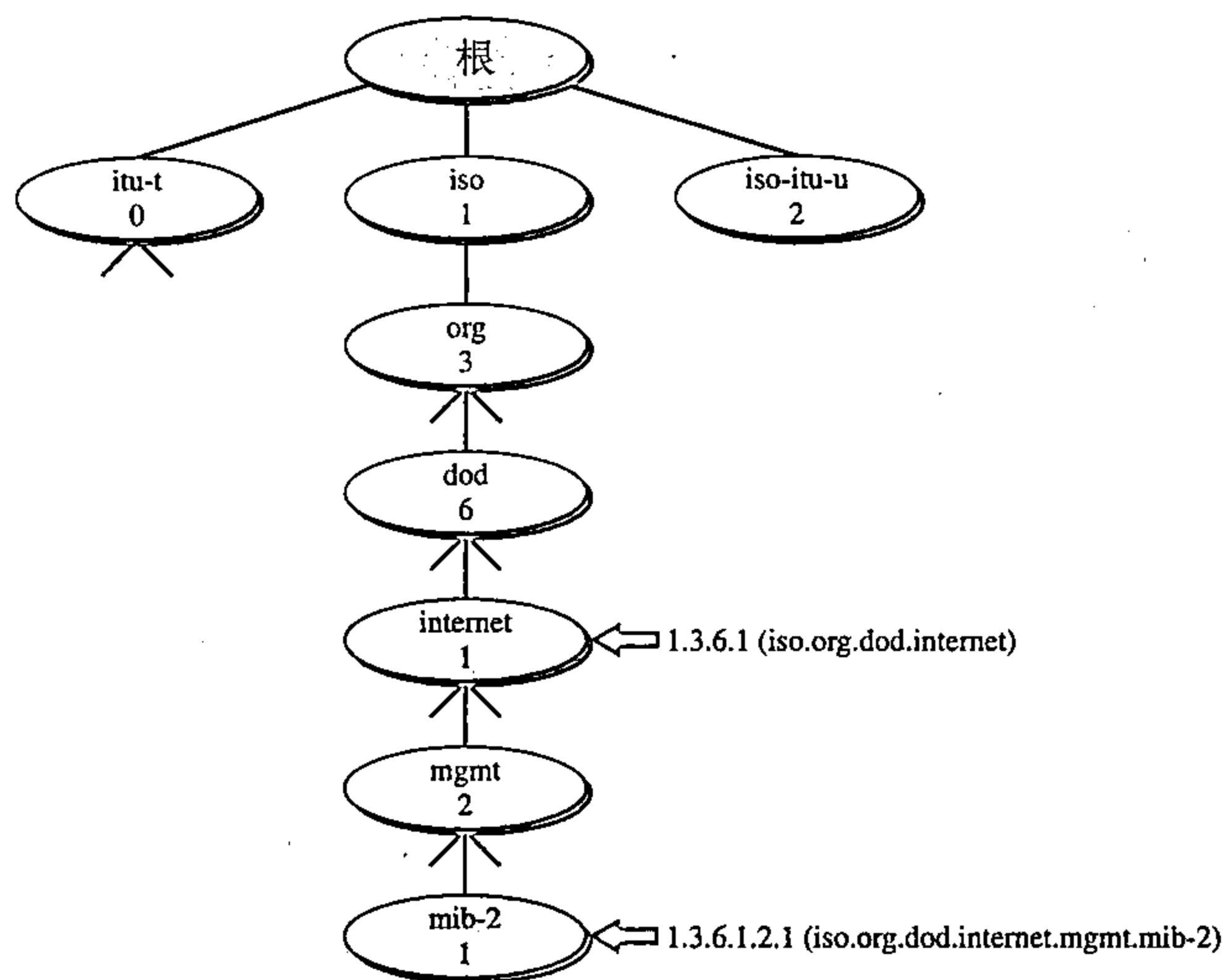


图28.6 对象标识符

在SNMP中使用的对象位于mib-2对象的下面，所以它们标识符都从1.3.6.1.2.1开始。

所有被SNMP管理的对象都有一个对象标识符，对象标识符都是从1.3.6.1.2.1开始。

类型

对象的第二个属性是在这个对象中存储的数据类型。为了定义数据类型，SMI使用抽象语法表示法1 (Abstract Syntax Notation1, ASN.1) 的一些基本定义，但还增加了几个新的定义。换言之，SMI既是ASN.1的一个子集，又是ASN.1的一个超集。

SMI使用两大类数据类型：简单的和结构化的。我们先定义简单类型，然后讨论如何从简单类型构造出结构化类型（如图28.7所示）。

简单数据类型 (simple data type) 是原子数据类型。这些类型中一些是直接取自ASN.1，另一些是SMI增加的。表28.1给出了最重要的一些类型。前5个取自ASN.1，后7个是SMI定义的。

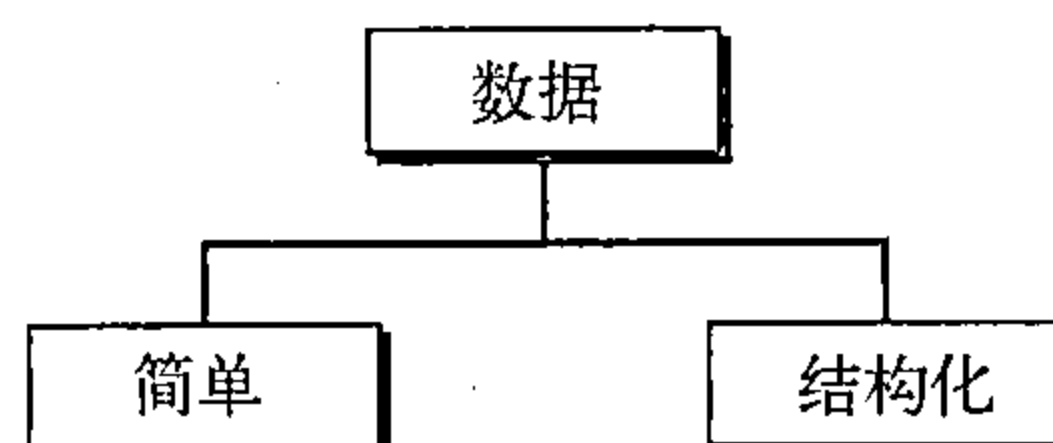


图28.7 数据类型

表28.1 数据类型

类 型	大 小	说 明
INTEGER	4字节	在 -2^{31} 到 $2^{31}-1$ 之间的一个整数
Integer32	4字节	与INTEGER类型相同
Unsigned32	4字节	在0到 $2^{32}-1$ 之间的一个无符号数
OCTET STRING	可变	长度最多达65 535的字节串
OBJECT IDENTIFIER	可变	对象标识符
IPAddress	4字节	由4个整数组成的IP地址
Counter32	4字节	一个整数，其值由0增加到 2^{32} ；当它达到最大值时就返回0
Counter64	8字节	64位计数器

(续)

类 型	大 小	说 明
Gauge32	4字节	与Counter32同，但当它达到最大值时不返回，而保持最大值直到复位为止
TimeTicks	4字节	记录时间的计数值，以1/100s为单位
BITS		位串
Opaque	可变	未解释的串

结构化类型 将简单的和结构化的数据类型结合起来，就可构成新的结构化数据类型。SMI定义了两种结构化数据类型（structured data type）：sequence和sequence of。

- Sequence。一个sequence数据类型是一些简单数据类型的组合，不必都是相同类型。它与C编程语言中使用的struct或record的概念相似。
- Sequence of。一个sequence of数据类型是所有相同类型的简单数据类型的组合，或所有相同类型的sequence数据类型的组合。它与C编程语言中使用的array的概念相似。

图28.8说明了数据类型。

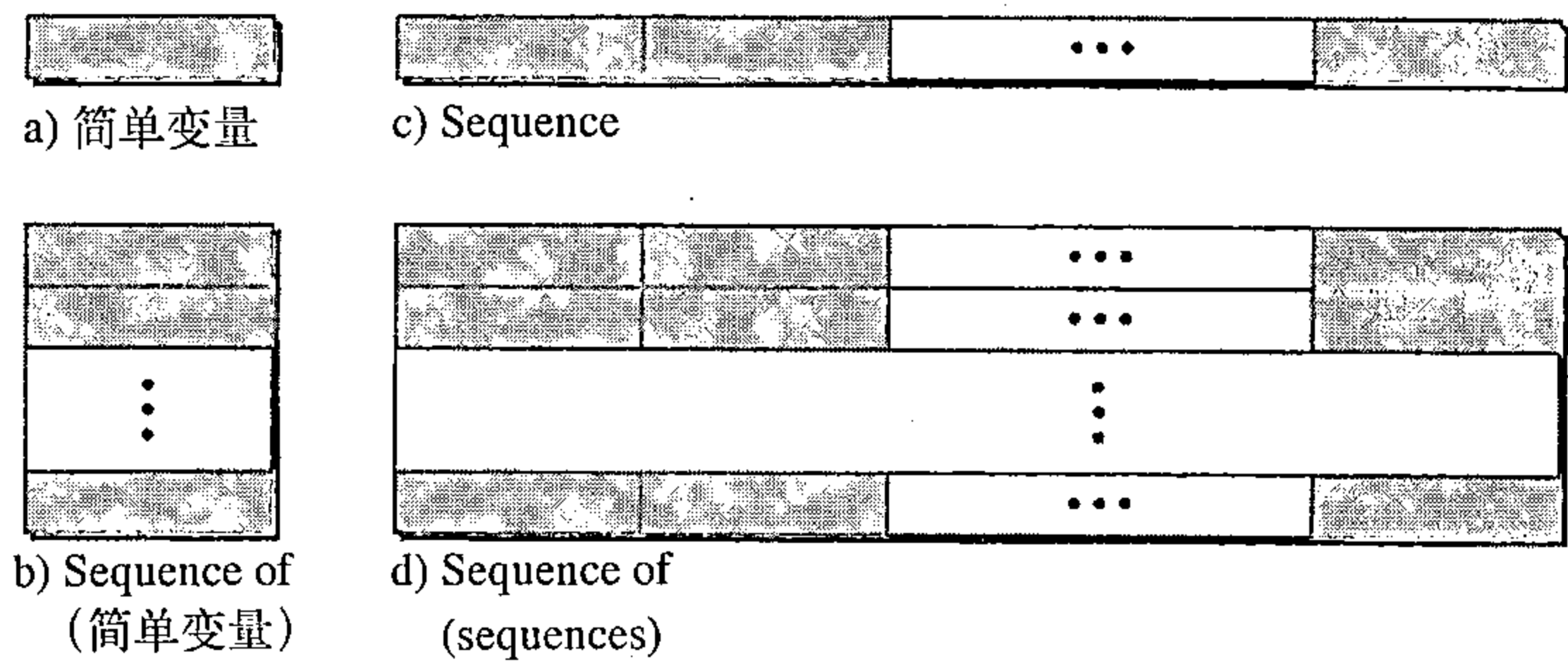


图28.8 概念性的数据类型

编码方法

SMI使用另一个标准，即基本编码规则（Basic Encoding Rule, BER），对数据进行编码并将编码后的数据在网络上传输。BER规定每一块数据都要被编码成三元组的格式：标签、长度和值，如图28.9所示。

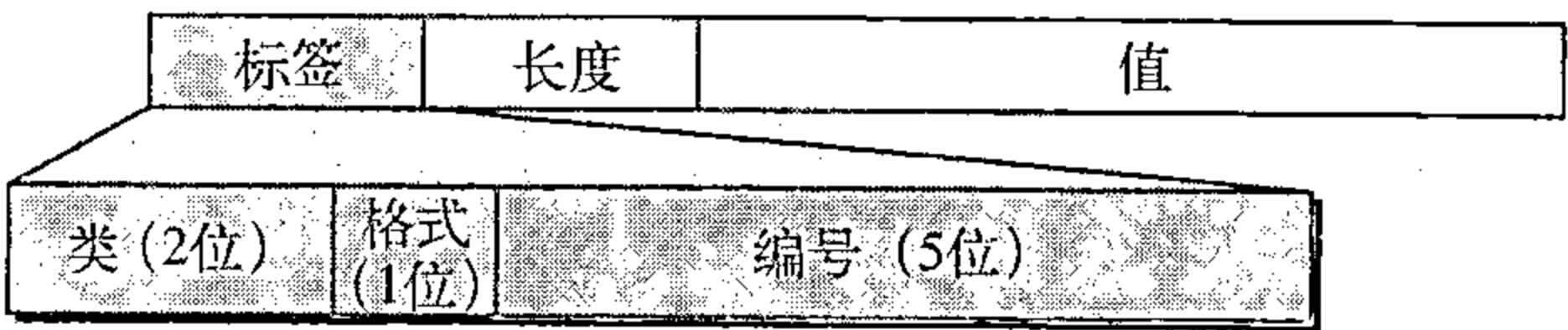


图28.9 编码格式

- 标签是一个字节的字段，它定义了数据类型。它由3个子字段组成：类（2位）、格式（1位）和编号（5位）。类子字段定义数据的作用域。共定义4类：通用类（00）、应用类（01）、特定上下文类（10）和专用类（11）。通用数据类型来自ASN.1（INTEGER、OCTET STRING和ObjectIdentifier）。应用数据类型是由SMI增加的（IPAddress、Counter、Gauge和TimeTicks）。有5种特定上下文数据类型，其意义随着协议的不同而不同，专用数据类型是特定厂商使用的。

格式子字段指出数据是简单的（0）还是结构化的（1）。编号子字段将简单的或结构化的数据进一步划分成一些子组。例如，在通用类中，对于简单格式，INTEGER的值是2，OCTET STRING的值是4等。表28.2给出本章所使用的数据类型，以及其标签的二进制和十六进制数。

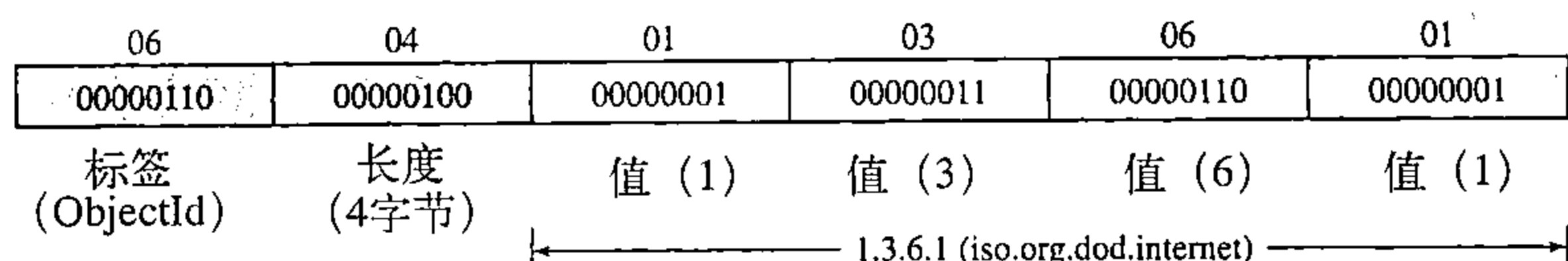


图28.13 例28.3 对象标识符1.3.6.1

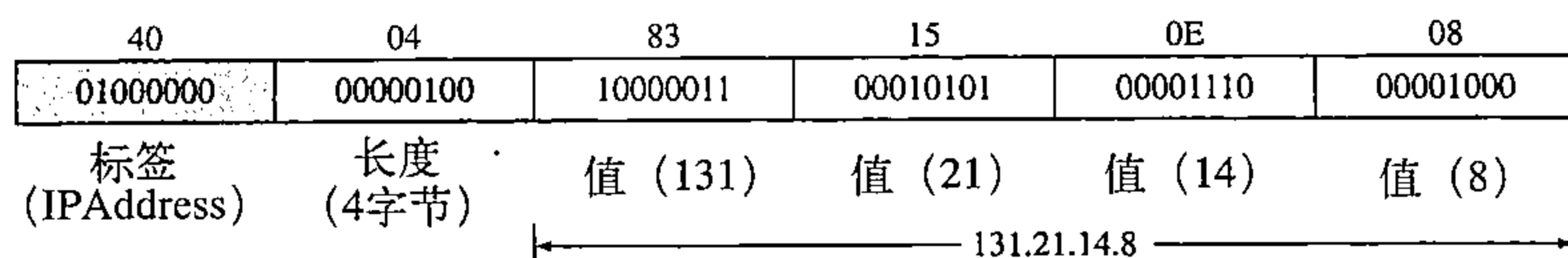


图28.14 例28.4 IP地址131.21.14.8

28.2.4 管理信息数据库 (MIB)

管理信息数据库版本2 (MIB2) 是网络管理中的第二个组件。每个代理都有它自己的MIB2, 这是管理器能够管理的所有对象的集合。在MIB中的对象分成10个不同的组: system、interface、address translation、ip、icmp、tcp、udp、egp、transmission和snmp。这些组都在对象标识符树中的mib-2对象的下面 (如图28.15所示)。每一个组定义一些变量和 (或) 表。

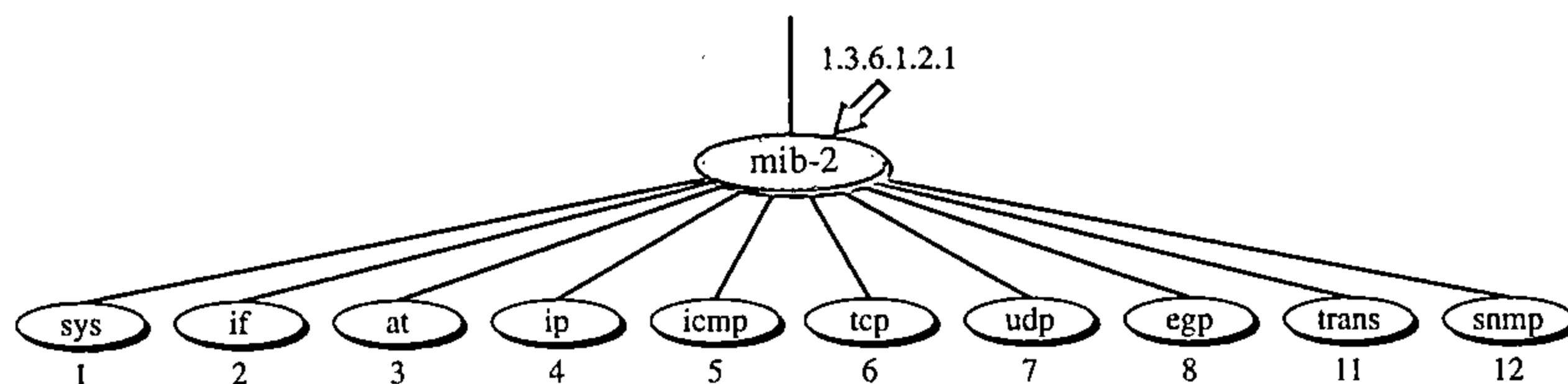


图28.15 mib-2

下列是一些对象的简要说明:

- sys 这个对象 (system) 定义了关于整个节点 (系统) 的通用信息, 如名字、位置和生命周期。
- if 这个对象 (interface) 定义了关于整个节点所有接口的信息, 如接口数、物理地址和IP地址。
- at 这个对象 (address translation) 定义了关于ARP表的信息。
- ip 这个对象定义了有关IP的信息, 如路由表和IP地址。
- icmp 这个对象定义了有关ICMP的信息, 如已发送和已接收到的分组个数和产生总差错数。
- tcp 这个对象定义了有关TCP的信息, 如连接表、超时值、端口数及已发送和已接收到的分组个数。
- udp 这个对象定义了有关UDP的信息, 如端口数及已发送和已接收到分组的个数。
- snmp 这个对象定义了有关SNMP本身的信息。

访问MIB变量

我们用udp组作为例子, 说明如何访问不同的变量。在udp组中有4个简单变量和一个记录 (表) 序列, 图28.16给出这些变量和表。

我们将说明如何访问每一个实体。

简单变量。要访问任何简单变量, 我们用组的id (1.3.6.2.1.7) 后面跟着该变量的id, 下

面表示如何访问每一个变量。

udpInDatagrams $\Rightarrow$ 1.3.6.1.2.1.7.1

udpNoPorts $\Rightarrow$ 1.3.6.1.2.1.7.2

udpInErrors $\Rightarrow$ 1.3.6.1.2.1.7.3

udpOutDatagrams $\Rightarrow$ 1.3.6.1.2.1.7.4

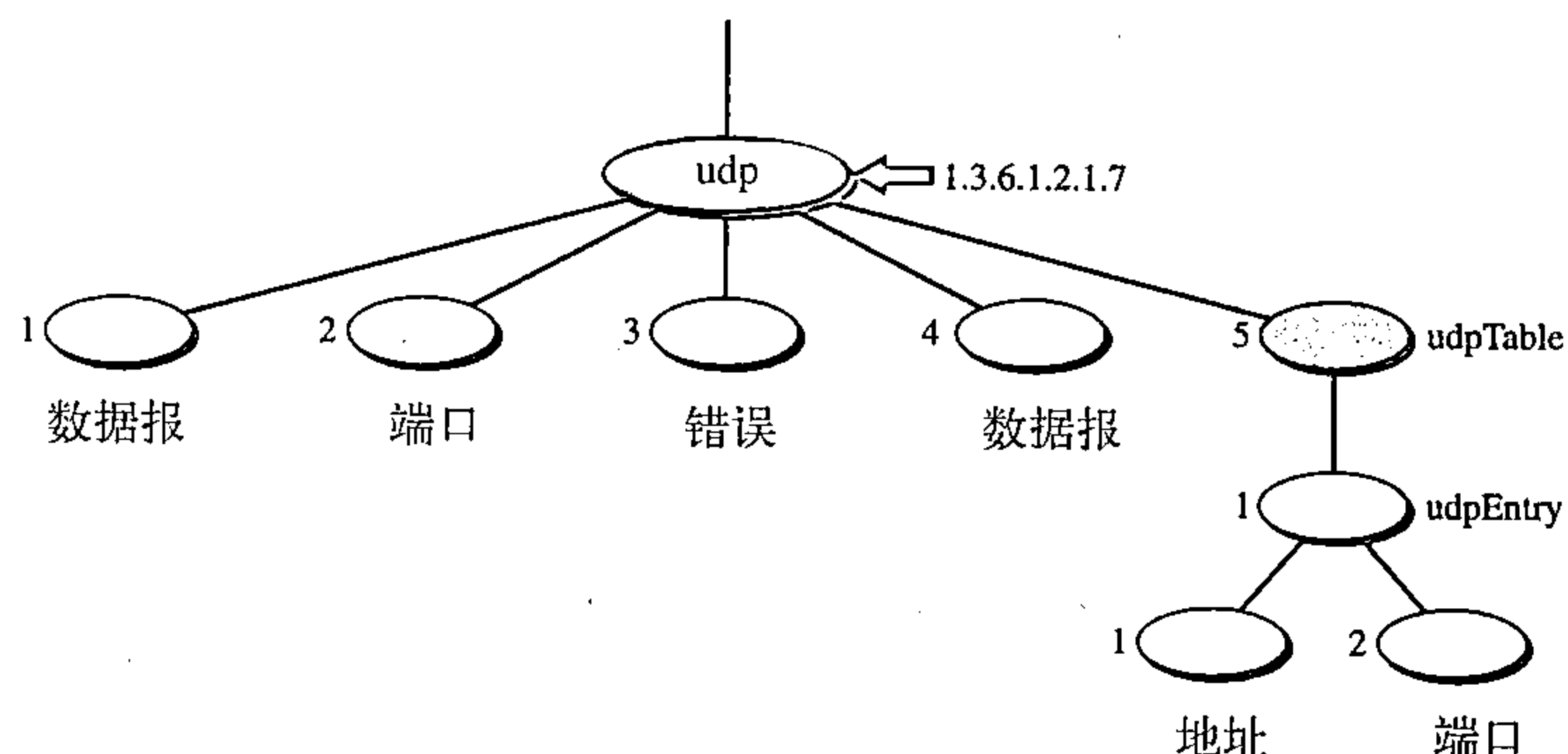


图28.16 udp组

但是，这些对象标识符定义的是变量而不是实例（内容）。要给出每一个变量的实例或内容，必须增加一个实例后缀。一个简单变量的实例后缀是一个0。换言之，要给出以上变量的实例，使用下列方法：

udpInDatagrams.“0” $\Rightarrow$ 1.3.6.1.2.1.7.1.0

udpNoPorts.“0” $\Rightarrow$ 1.3.6.1.2.1.7.2.0

udpInErrors.“0” $\Rightarrow$ 1.3.6.1.2.1.7.3.0

udpOutDatagrams.“0” $\Rightarrow$ 1.3.6.1.2.1.7.4.0

表 要标识一个表，首先使用表id。如图28.17所示，udp组只有一个表（id为5）。

因此，要访问这个表，使用下面形式：

udpTable $\Rightarrow$ 1.3.6.1.2.1.7.5

但是，这个表不是树结构的叶子级，我们不能访问这个表；因此如下定义在这个表（id为1）中的项目（序列）：

udpEntry $\Rightarrow$ 1.3.6.1.2.1.7.5.1

这个项目也不是一个叶子级，我们不能访问它。因此需要定义这个项目中的每一个实体（字段）。

udpLocalAddress $\Rightarrow$ 1.3.6.1.2.1.7.5.1.1

udpLocalPort $\Rightarrow$ 1.3.6.1.2.1.7.5.1.2

这两个变量在树的叶子上。虽然能访问它们的实例，但需要定义是哪一个实例。在任何时候，这个表对每一个本地地址/本地端口对可以有多个值。要访问表中的一个特定实例（行），应当给以上id加上索引。在MIB中，数组的索引不是整数（像大多数的编程语言那样）。这些索引是基于在这些项目中的一个或多个字段的值。在我们的例子中，udpTable的索引是基于本地地址和本地端口号。例如，图28.18显示一个具有4行的表和每一个字段的值。每一行索引是这两个值的组合。

要访问第一行本地地址的实例，使用标识符加上实例的索引：

udpLocalAddress.181.23.45.14.23 $\Rightarrow$ 1.3.6.1.2.7.5.1.1.181.23.45.14.23

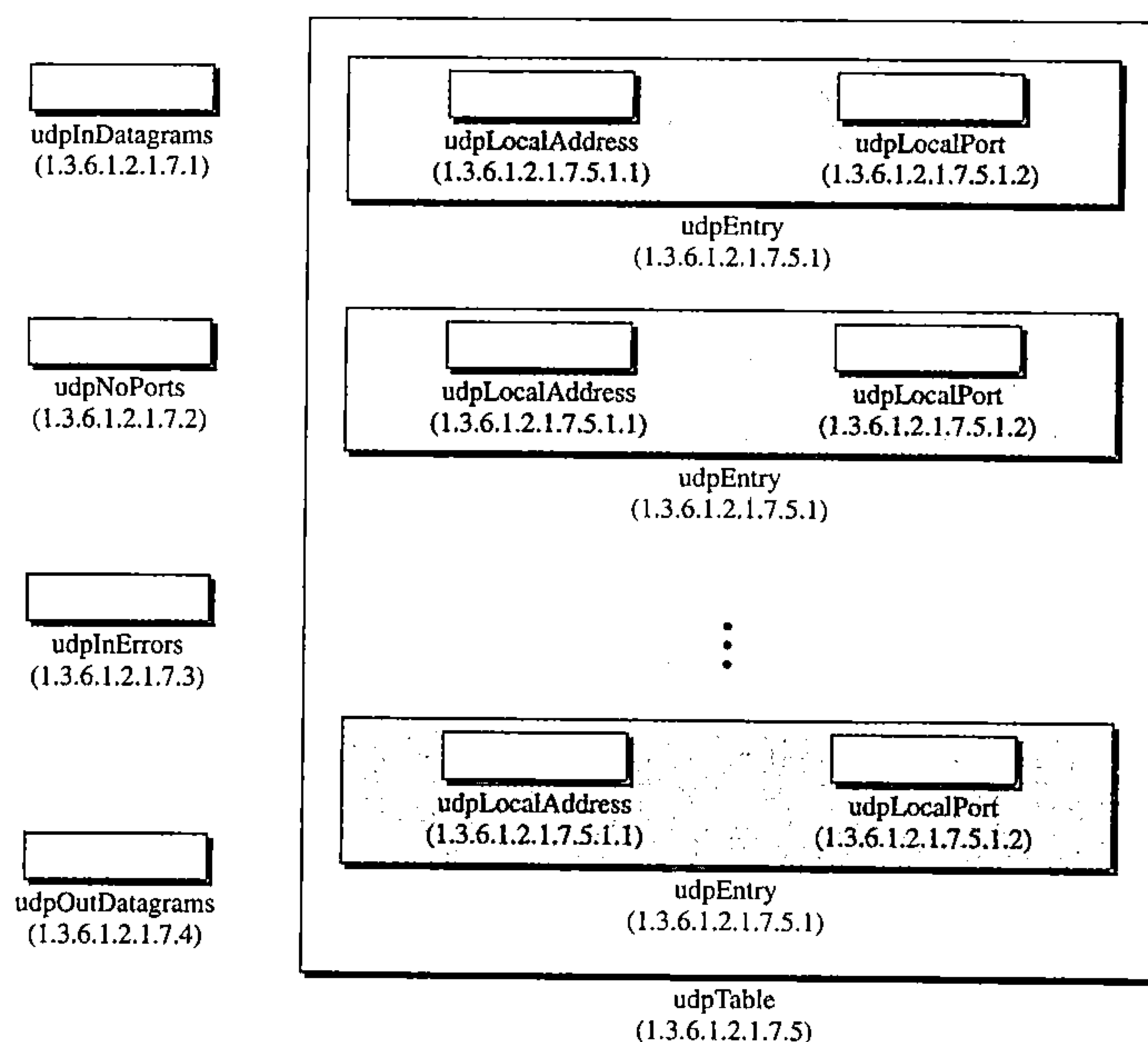


图28.17 udp的变量和表

注意：并非所有表的索引都是这样，有些索引使用一个字段的值，有些使用两个字段的值，等等。

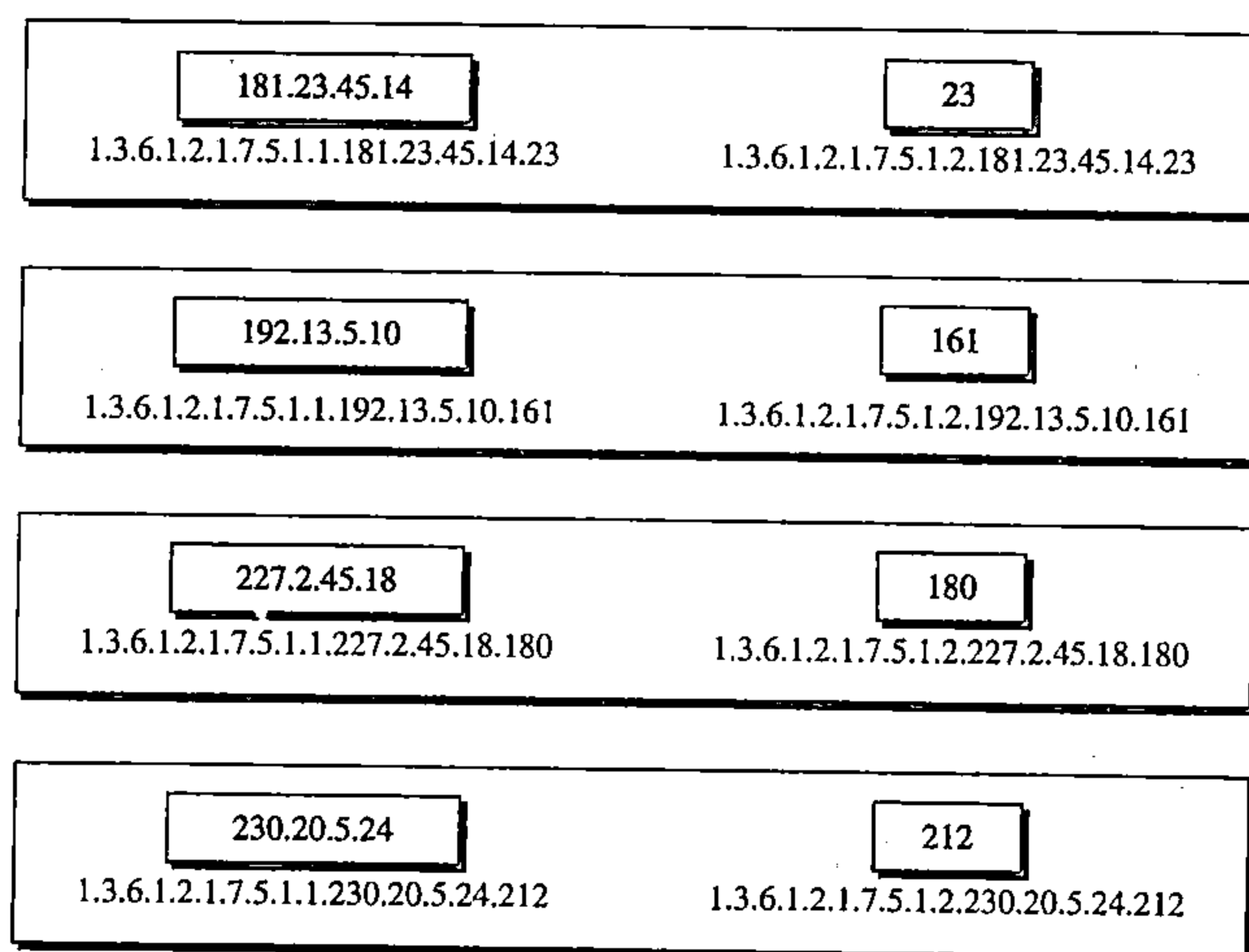


图28.18 udpTable的索引

28.2.5 字典排序

关于MIB变量的一个有趣问题就是对象标识符（包括实例标识符）是按字典的顺序排列的。表顺序是按照列-行规则，这就是说，要按逐列的顺序。而在每一列中，必须从顶到底，如图28.19所示。

字典排序（lexicographic ordering）能使一个管理器在定义了第一个变量后，可一个一个地访问一组变量，在下一节我们将会看到这一点。

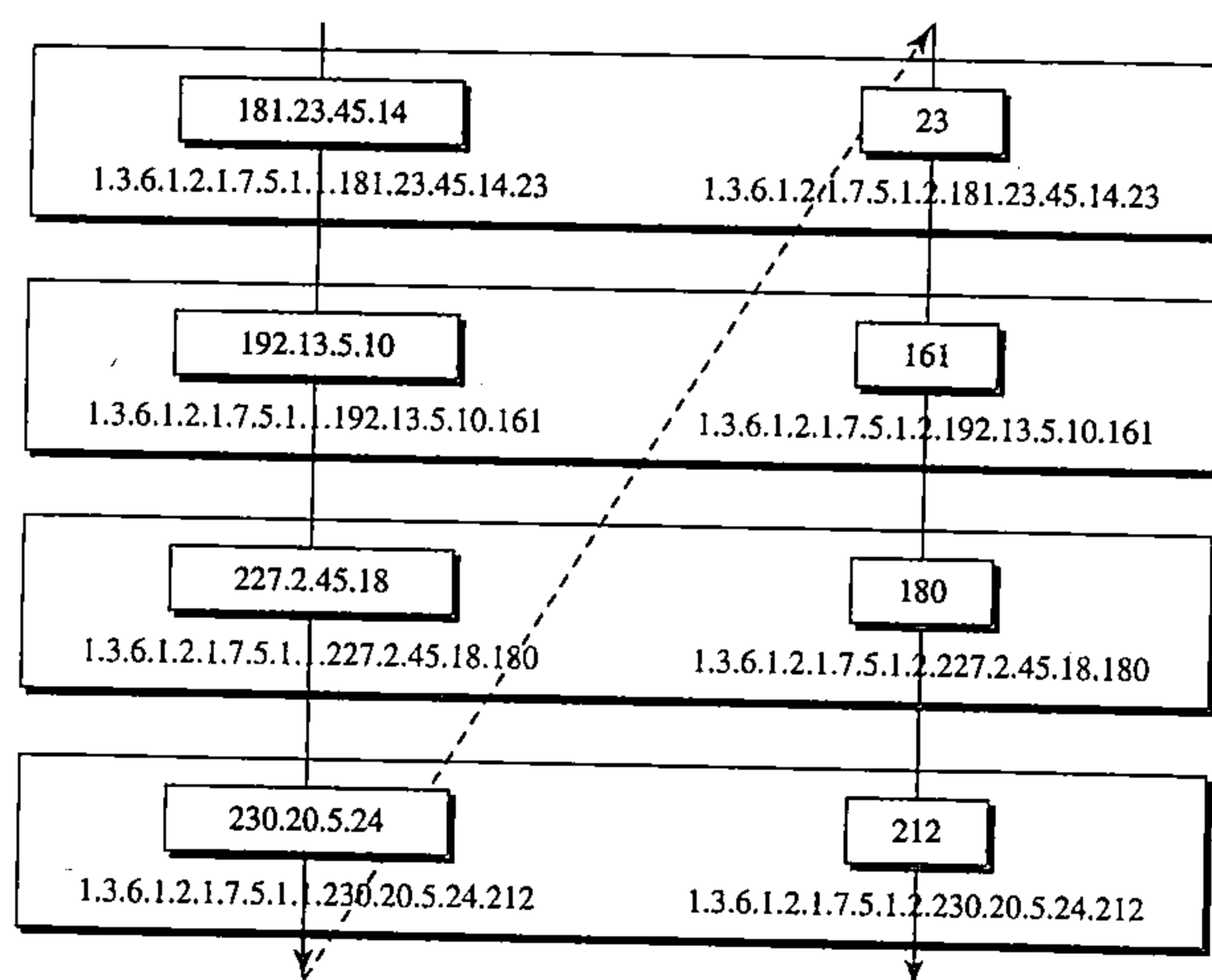


图28.19 字典排序

28.2.6 SNMP

SNMP在因特网的网络管理中使用SMI和MIB。SNMP是一个应用程序，它允许：

1. 管理器读取代理定义的一个对象的值；
2. 管理器将一个值存储在代理定义的一个对象中；
3. 代理将关于异常情况的告警报文发送给管理器。

PDU

SNMP版本3定义了8种报文（或PDU）：GetRequest、GetNextRequest、GetBulkRequest、SetRequest、Response、Trap、InformRequest和Report（如图28.20所示）。

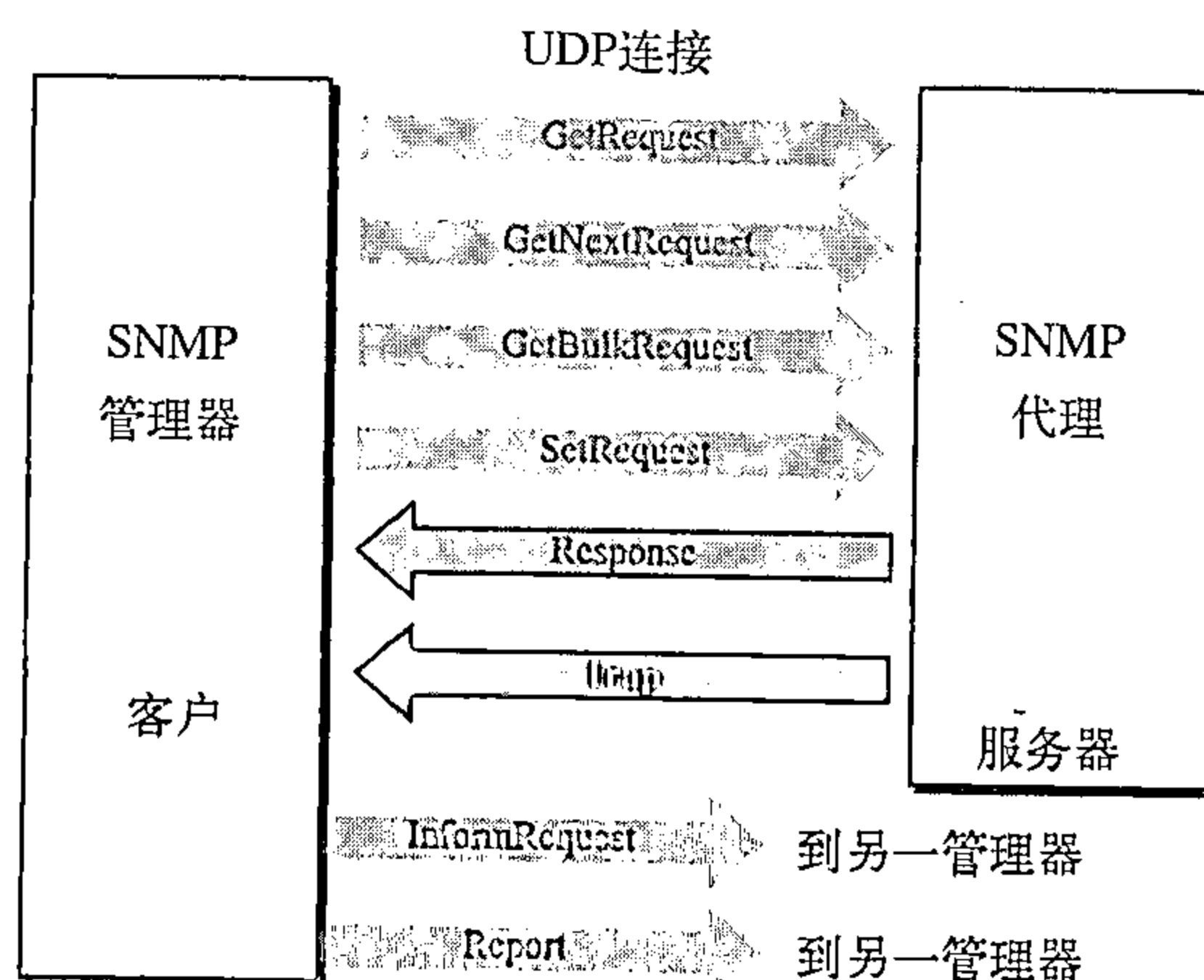


图28.20 SNMP PDU

GetRequest（请求读取）GetRequest

PDU报文是由管理器（客户机）发送给代理（服务器），用来读取一个变量或一组变量。

GetNextRequest（读取下一个请求）PDU报文是由管理器发送给代理，用来读取一个变量的值。所要读取的值是PDU中定义的ObjectID后面对象的值，它主要是用来读取一个表中的项目的值。如果管理器不知道该项目的索引，它就不能读这个值。但是，它可以使用GetNextRequest并定义表的ObjectID。因为第一个项目的ObjectID是紧跟在表的ObjectID后面，因此第一个项目的值立即返回了。管理器可使用这个ObjectID来得到下一个项目的值，等等。

GetBulkRequest（读取大量请求）PDU报文是由管理器发送给代理，用来读取大量数据。它可用来取代多个GetRequest和GetNextRequest PDU。

SetRequest（设置请求）PDU报文是由管理器发送给代理，用来设置（存储）一个变量的值。

Response（响应）PDU报文是由代理发送给管理器以响应GetResponse和GetNextRequest，它包含管理器所请求的变量的值。

Trap（陷阱）（也称为SNMP版本2的Trap，以区别SNMP版本1的Trap）PDU报文是由代理发送给管理器，用来报告一个事件。例如，如果代理重新启动，它通知管理器并报告重新启动的时间。

InformRequest（读取信息）PDU报文是由一个管理器发送给另一个远程管理器，用来获得远程管理器所控制的代理的某些变量的值。

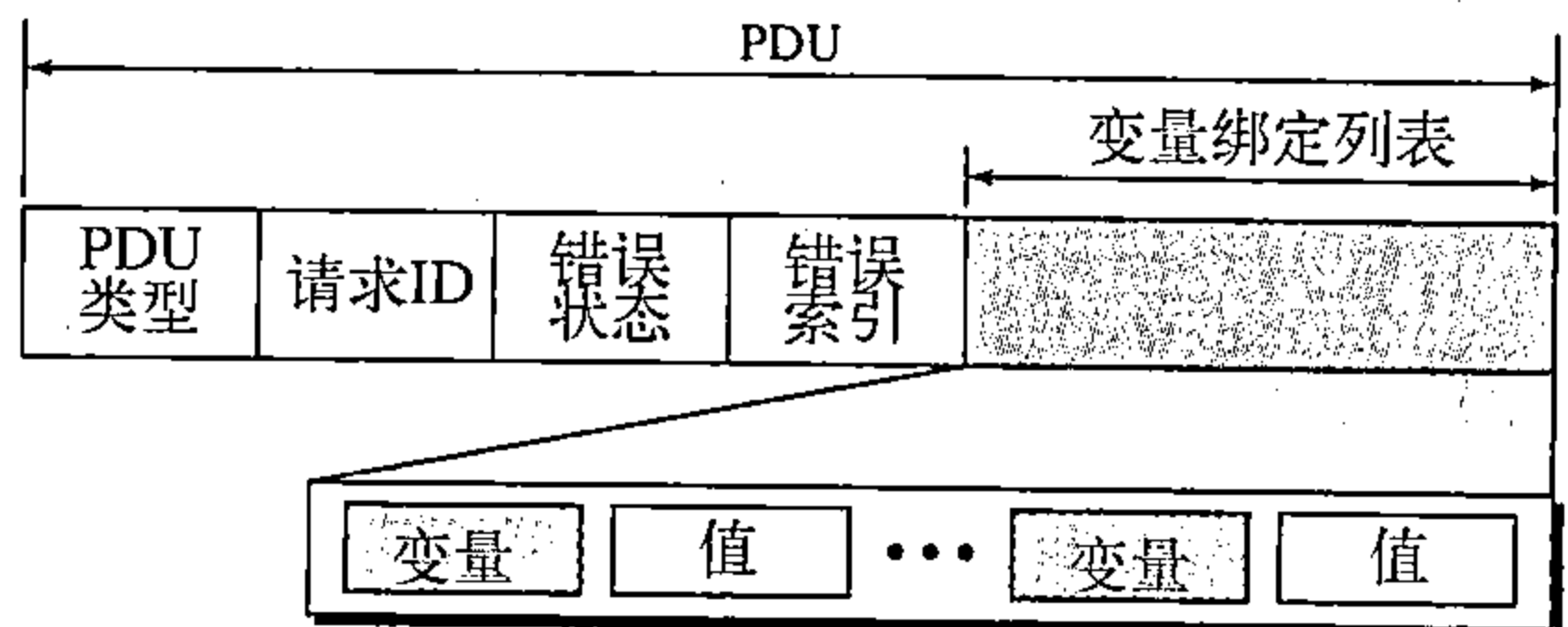
Report（报告）PDU用来报告管理器之间错误类型。

格式

8种SNMP PDU的格式如图28.21所示。在图28.21中，除GetBulkRequest有两处不同外，其他PDU都相同。

各字段列出如下：

- PDU类型（PDU type）。这个字段定义PDU类型（如表28.4所示）。
- 请求标识（Request ID）。这个字段是序列号，由管理器在Request PDU中使用，而代理在响应中重复它，它用来使响应和请求相匹配。
- 错误状态（Error status）。这是只用于Response PDU中的一个整数，它给出代理报告的错误类型。在Request PDU中它的值是0，表28.3列出了可能出现的错误类型。



不同处：

1. 除GetBulkRequest外，对于所有的请求报文错误状态和错误索引值都是0；
2. 在GetBulkRequest中用一次最多可以取回的MIB对象数（nonrepeater）字段代替错误状态字段，用其他MIB对象能被取回的最大次数字段代替错误索引字段。

图28.21 SNMP PDU格式

表28.3 错误类型

状 态	名 字	说 明
0	noError	无错误
1	tooBig	响应太大无法放入一个报文中
2	noSuchName	变量不存在
3	badValue	要存储的值无效
4	readOnly	这个值不能修改
5	genErr	其他错误

- 一次最多可以取回的MIB对象数（Nonrepeater）。这个字段只存在于GetBulkRequest PDU中，代替错误状态字段，在请求PDU中它的值是空。
- 错误索引（Error index）是一个偏移量，它告诉管理器哪一个变量引起这个错误。
- 其他MIB对象能被取回的最大次数（Max-repetition）。这个字段只存在于GetBulkRequest PDU中，代替错误索引字段，在请求PDU中它是空的。
- 变量绑定列表（VarBindList）。这是管理器希望读取或设置的一组具有相应值的变量，在GetRequest和GetNextRequest中它的值是空。在TrapPDU中，它给出了与一个特定PDU有关的变量和值。

28.2.7 报文

SNMP不发送单个PDU，而是将它嵌入到一个报文中发送。SNMPv3 中的报文是由四个元素组成：版本、头部、安全参数和数据（包含已编码的PDU），如图28.22所示。

因为不同报文中的这些元素的长度是不同的，所以SNMP使用BER标准对每个元素进行编

码。记住BER使用标签和长度来定义一个值。版本定义当前版本（3）。头部包含报文标识符、报文最大长度（回答的最大长度）、报文标记（OCTET STRING数据类型的一个8位组，其中每一位定义安全类型，如保密、鉴别或其他信息）以及报文安全性模型（定义安全协议）。报文安全参数用于创建报文摘要（见第31章）。数据包括PDU。如果数据是加密的，则存在有关加密引擎的信息（执行加密的管理器程序）和加密前后关系（加密类型）后跟被加密的PDU。如果数据没有加密，则数据就是PDU。

为了定义PDU的类型，SNMP使用一个标签。该类是上下文敏感的（10），格式是结构化的（1），不同类型的报文编号是0，1，2，3，5，6.7和8（如表28.4所示）。

注意：SNMPv1定义标签A4作为陷阱，目前已不用了。

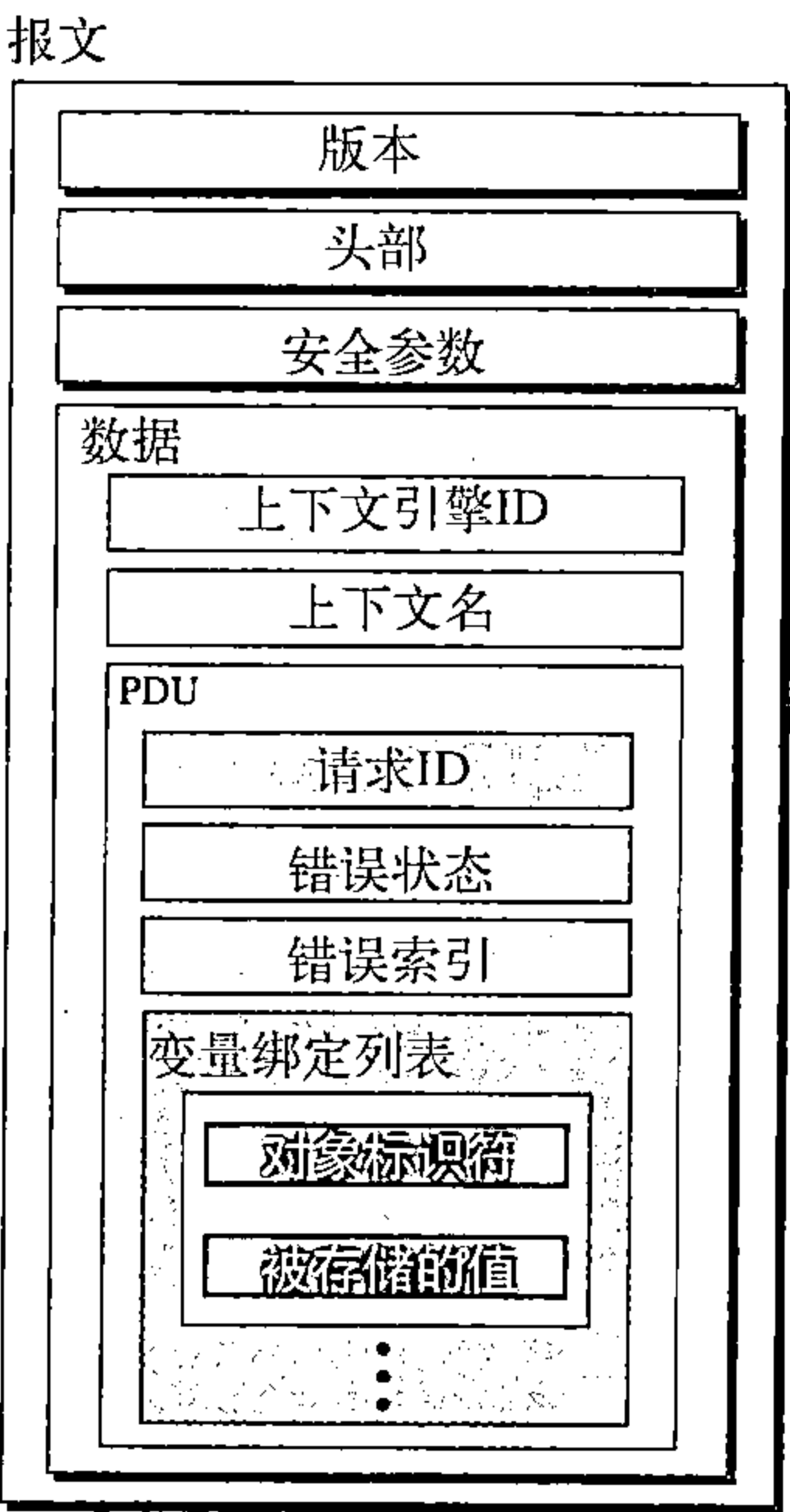


图28.22 SNMP 报文

表28.4 SNMP报文的代码

代码	类	格式	编号	整个标签（二进制）	整个标签（十六进制）
GetRequest	10	1	00000	10100000	A0
GetNextRequest	10	1	00001	10100001	A1
Response	10	1	00010	10100010	A2
SetRequest	10	1	00011	10100011	A3
GetBulkRequest	10	1	00101	10100101	A5
InformRequest	10	1	00110	10100110	A6
Trap (SNMPv2)	10	1	00111	10100111	A7
Report	10	1	01000	10101000	A8

例28.5

在本例中，管理器站点（SNMP客户机）使用GetRequest报文读取路由器接收到的一些UDP数据报。

仅有一个变量绑定表项，与这个信息相应MIB变量是udpInDatagram，其对象标识符是1.3.6.1.2.1.7.1.0。管理器打算读取一个值（而不是存储一个值），因此值定义为0的空实体。

图28.23说明了分组与sequence的分级性质的概念。我们用白色与最里面的框表示sequence，而用其他灰色框表示UDP。

变量绑定表只有一个绑定变量。变量类型06，长度为09。变量类型05，长度为00。整个变量绑定是sequence，长度为0D₁₆ (13)。变量绑定表也是sequence，长度为0F₁₆ (15)。GetRequest PDU是长度为1D₁₆ (29)。

此处，有3个OCTET STRING是关于安全参数、安全模型和标记的，还有2个integer定义最大长度 (1024) 和报文ID (64)。头部是sequence，长度为12，为简化用空白省略。整个报文是一个sequence，长度为52个字节。

图28.24 表示了管理器站点 (客户机) 向代理 (服务器) 发送实际的报文。

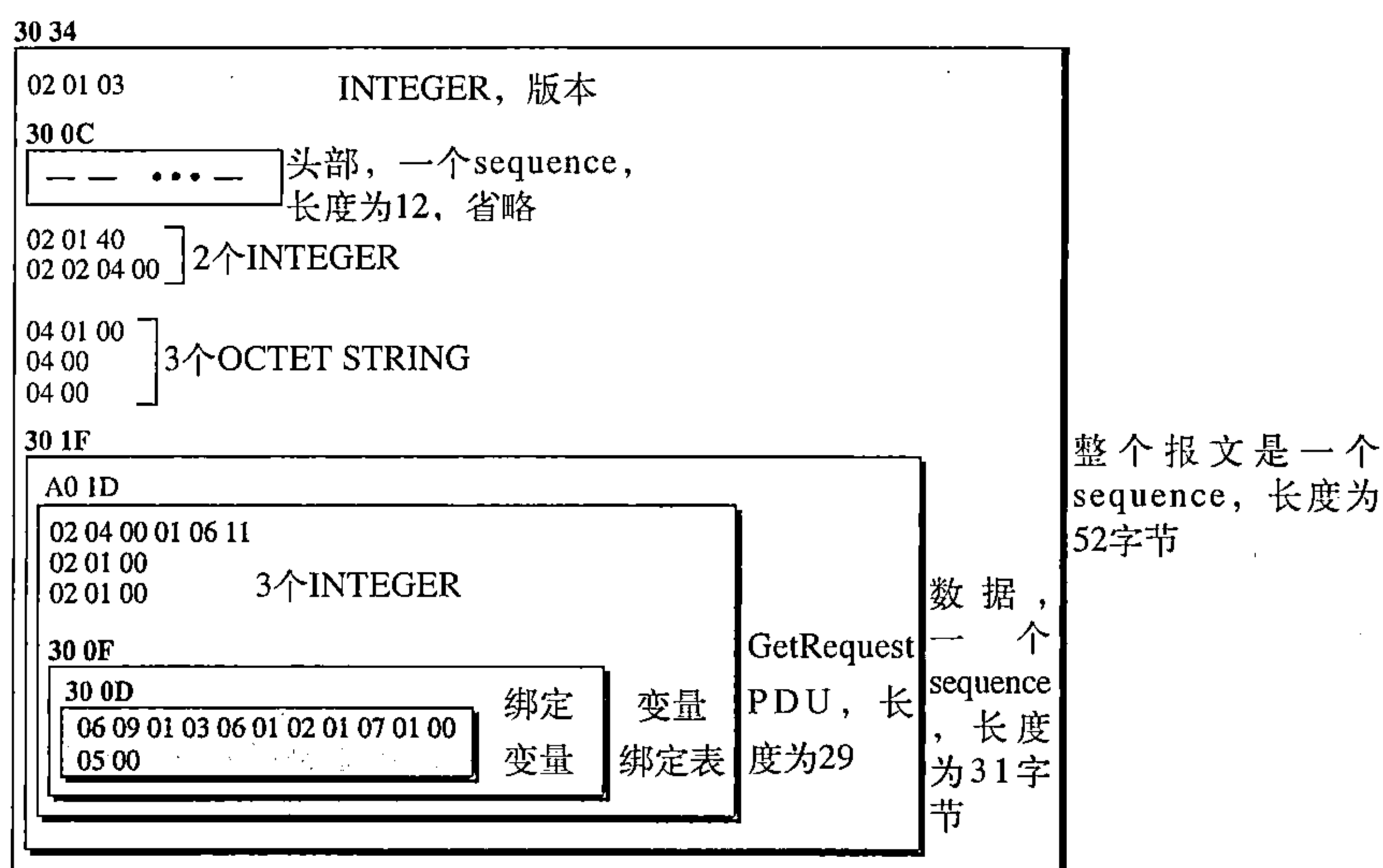


图28.23 例28.5

28.2.8 UDP端口

SNMP在两个熟知端口161和162上使用UDP的服务，熟知端口161由服务器 (代理) 使用，而熟知162端口由客户 (管理器) 使用。

代理 (服务器) 在端口161发出一个被动打开。然后它就等待从管理器 (客户) 来的连接。管理器 (客户) 使用临时端口发出一个主动打开。客户向服务器发送请求报文，使用临时端口作为源端口而熟知端口161作为目的端口。服务器向客户发送响应报文，使熟知端口161作为源端口而临时端口作为目的端口。

管理器 (客户) 在端口162发送一个被动打开，然后它就等待从代理 (服务器) 来连接。代理 (服务器) 只要有一个Trap报文要发送，就使用临时端口发出一个主动打开。这个连接是单向的，从服务器到客户 (如图28.25所示)。

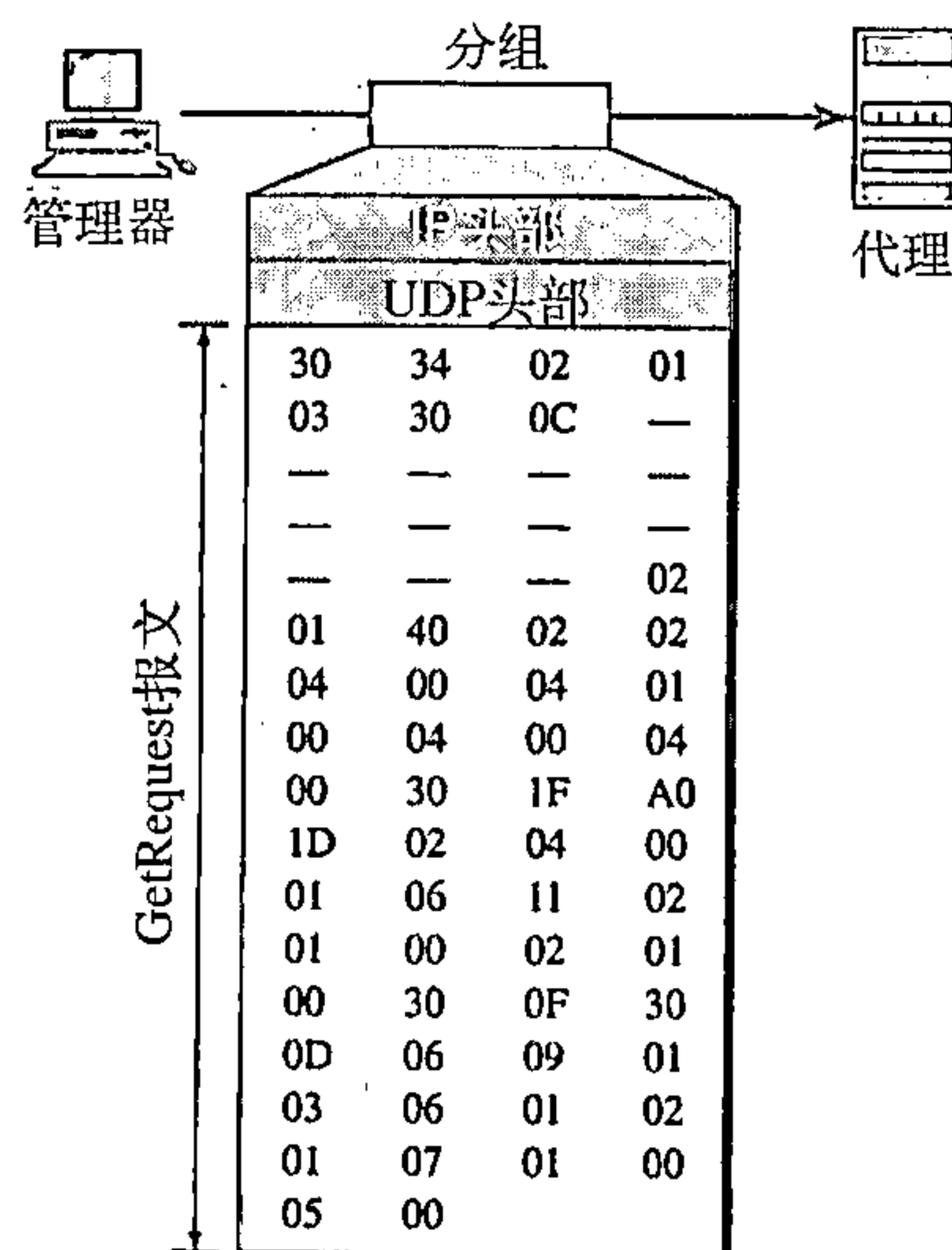


图28.24 GetRequest报文

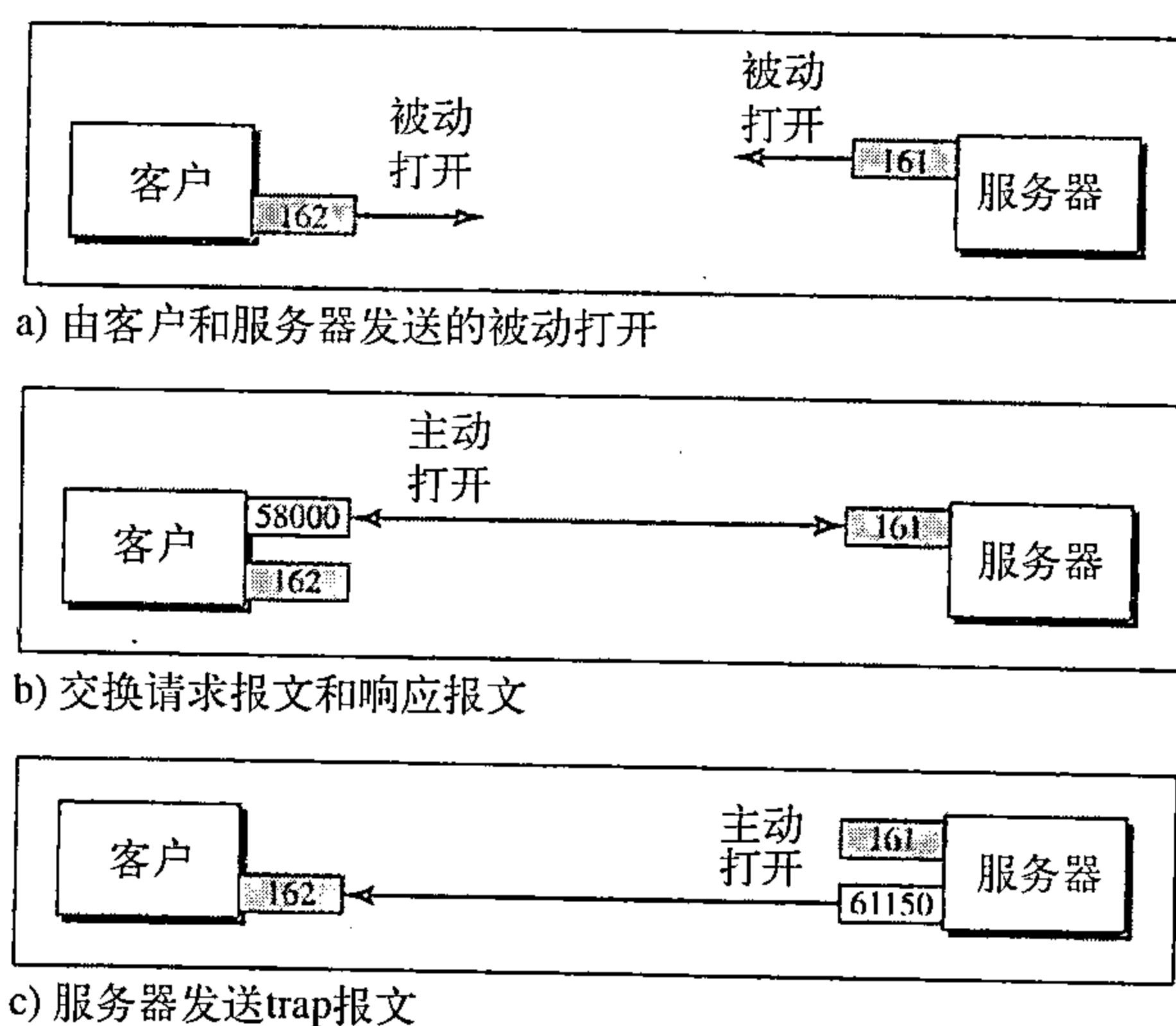


图28.25 SNMP的端口号

在SNMP中的客户/服务器机制与其他协议不同。此处的客户和服务端使用熟知端口。此外，客户和服务端都必须不停地运行。其原因是请求报文是由管理器（客户）发出的，但Trap报文是由代理（服务器）发出的。

28.2.9 安全性

SNMPv3与SNMPv2的主要差别是提高了安全性。SNMPv3提供两类安全：通用的和专用的。SNMPv3提供了报文鉴别、安全性与管理器授权。这三个方面的问题将在第31章中讨论。另外，SNMPv3允许管理器远程修改安全配置，这就是说在管理器站点的管理器可以没有物理设备。

推荐读物

为了深入讨论与本章有关的主题，推荐阅读下列读物和网站，在本书末列出方括号[...]中的参考资料。

读物

[MS01]、[Ste94]第25章、[Sta04]的22.3节和[Com04]第39章中有SNMP的讨论。网络管理在[Sub01]中有讨论。

网站

下面网站有与本章有关题目的讨论：

- www.ietf.org/rfc.html有请求评论的信息

请求评论

下列是有关SNMP、MIB和SMI有关的请求评论，

1065, 1067, 1098, 1155, 1157, 1212, 1213, 1229, 1231, 1243, 1284, 1351, 1352, 1354, 1389, 1398, 1414, 1441, 1442, 1443, 1444, 1445, 1446, 1447, 1448, 1449, 1450, 1451, 1452, 1461, 1472, 1474, 1537, 1623, 1643, 1650, 1657, 1665, 1666, 1696, 1697, 1724, 1742, 1743, 1748, 1749

本章小结

- 组成网络管理5个部分是配置管理、故障管理、性能管理、计费管理和安全管理。
- 配置管理涉及到网络实体的物理与逻辑的改变，它包括重新配置、软硬件文档和用户账号。
- 故障管理涉及到每个网络部件的正确操作，它可以是主动的或再生的。
- 性能管理涉及到监视和控制网络，确保网络尽可能有效地运行。用能力、通信量、吞吐量和响应时间等来度量。
- 安全管理涉及控制对网络的访问。
- 计费管理涉及通过付费控制用户对网络资源的访问。
- 简单网络管理协议（SNMP）是使用TCP/IP协议族对互联网中的设备进行管理。
- 管理器通常是主机，它控制和监视一组代理，而代理通常是路由器。
- 管理器是运行SNMP客户程序的主机。
- 代理是运行SNMP服务器程序的路由器或主机。
- SNMP使管理任务与被管理设备的物理特性和下层的联网技术都没关系。
- SNMP使用另外两个协议服务：管理信息结构（SMI）和管理信息库（MIB）。
- SMI给对象命名，定义可在对象中存储的数据类型，以及对数据进行编码。
- SMI的对象按照分层次的树结构命名。
- SMI的数据类型按照抽象语法记法1（ASN.1）来定义。
- SMI使用基本编码规则（BER）对数据进行编码。
- MIB是能够被SNMP管理的对象组的集合。
- MIB使用字典排序来管理其变量。
- SNMP的功能有3种：
 1. 管理器读取代理定义的对象的价值。
 2. 管理器将一个值存储在代理定义的对象中。
 3. 代理将告警报文发送给管理器。
- SNMP定义了5种报文：GetRequest、GetNextRequest、SetRequest、GetResponse和Trap。
- SNMP在两个熟知的端口161和162上使用UDP服务。
- SNMPv3是在前面版本上增加了安全机制。

习题

复习题

1. 定义网络管理。
2. 列出网络管理的5个功能。
3. 定义配置管理及其目的。
4. 列出配置管理的两个子功能。
5. 定义故障管理及其目的。
6. 列出故障管理的两个子功能。
7. 定义性能管理及其目的。
8. 列出性能管理的4个可测量的量。
9. 定义安全管理及其目的。
10. 定义计费管理及其目的。

练习题

11. 试给出INTEGER (整数) 1 456的编码。
12. 试给出OCTET STRING (串) "Hello World" 的编码。
13. 试给出长度为100的任一OCTET STRING (串) 的编码。
14. 试说明下列的记录 (sequence) 是如何编码的。
INTEGER OCTET STRING IP Address
2345 "COMPUTER" 185.32.1.5
15. 试说明下列的记录 (sequence) 是如何编码的。
Time Tick INTEGER Object Id
12000 14564 1.3.6.1.2.1.7
16. 试说明下列的数组 (sequence of) 是如何编码的。其中每个元素是一个整数。
2345
1236
122
1236
17. 试说明下列的记录数组 (sequence of sequence) 是如何编码的。
INTEGER OCTET STRING Counter
2345 "COMPUTER" 345
1123 "DISK" 1430
3456 "MONITOR" 2313
18. 试对下列的代码进行解码:
 - a. 02 04 01 02 14 32
 - b. 30 06 02 01 11 02 01 14
 - c. 30 09 04 03 41 43 42 02 02 14 14
 - d. 30 0A 40 04 23 51 62 71 02 02 14 12

第29章 多 媒 体

近年来，技术的进步已经改变了人们使用音频和视频的方式。过去，我们通过收音机收听广播，通过电视收看视频节目，通过电话网与另一方交互式通信。但是时代已经变了，人们使用因特网不仅限于文字和图像通信，而且用于音频和视频服务。本章将集中介绍因特网在音频和视频方面的应用。

我们可以把音频和视频服务分为三大类：流式存储音频/视频 (streaming stored audio/video)、流式实时音频/视频 (streaming live audio/video) 和交互式音频/视频 (interactive audio/video)，如图29.1所示。流式意味着用户在文件下载的同时就可以收听（或者观看）。

第一类，流式存储音频/视频是一些经过压缩并存储在服务器中的文件。客户机通过因特网下载这些文件，这就是我们有时所说的音频/视频点播 (on-demand audio/video)。存储音频文件的例子有歌曲、交响乐、磁带书籍和一些精品讲座。存储视频文件的例子有：电影、电视节目和音乐视频剪辑等。

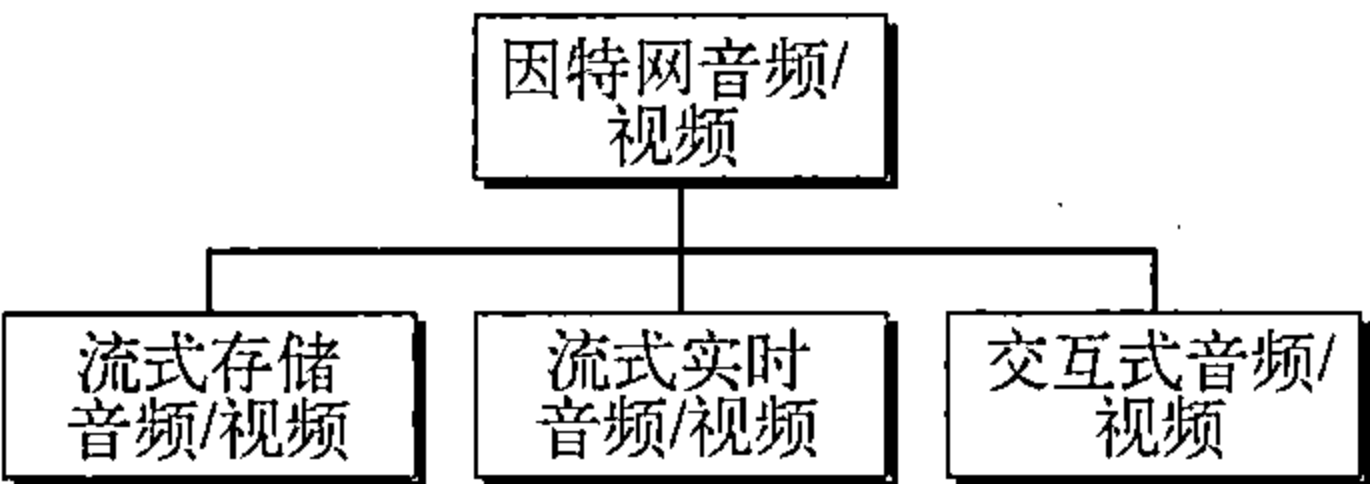


图29.1 因特网音频/视频

流式存储音频/视频是指对压缩的音频/视频文件的点播请求。

第二类，流式实时音频/视频是指用户通过因特网收听收看音频和视频节目。这方面比较好的一个应用实例是因特网广播。有一些广播电台仅仅在因特网上播出它们的节目，而多数电台用因特网和无线电波播放节目。因特网电视还没有普及，但是人们相信电视台将来会在因特网上播放它们的节目。

流式实时音频/视频是指电台和电视台通过因特网播放节目。

第三类，交互式音频/视频，人们使用因特网进行相互之间的交互式通信。这方面应用的一个较好实例是因特网电话和因特网远程会议。

交互式音频/视频是指使用因特网实现交互式音频/视频应用。

本章讨论这三种应用，首先要讨论与音频/视频有关的其他一些问题，如音频/视频数字化和音频/视频压缩。

29.1 音频与视频的数字化

在音频和视频信号通过因特网发送之前，需要进行数字化处理。下面将音频和视频分开讨论。

29.1.1 音频数字化

麦克风接收到声音时，会产生模拟的电子信号，表示为时间函数的声音振幅。这种信号称为模拟音频信号。模拟信号，如音频信号，可以数字化而产生数字信号。根据奈奎斯特定理，如果信号的最高频率是 f ，则需要每秒对信号采样 $2f$ 次。有许多用于将音频信号进行数字

化的方法，但原理是一样的。

声音每秒采样8 000次，每次采样值是8位，其结果是64Kbps的数字信号。音乐则每秒采样44 100次，每个采样值是16位，这样会产生单声道705.6Kbps和立体声1.411Mbps的数字信号。

29.1.2 视频数字化

视频由一系列的帧构成。如果帧在屏幕上显示足够快，我们就会有运动的感觉。原因是人的眼睛不能够分辨出快速闪过的单帧图像。每秒的帧数没有统一的标准。在北美，常见的是每秒25帧。但是为了避免闪烁的情况出现，需要不断重复刷新帧。电视需要每一帧刷新两次，也就是说需要发送50帧。换言之，如果在发送方有存储器的话，发送的25帧中每一帧都要从存储器中提取重画两次。

每一帧会被分为更小的栅格，称为图像元素或者像素 (pixel)。对于黑白电视，每个8位的像素可以表示为256个不同的灰度等级。对于彩色电视，每个像素是24位，每一种三原色（红、绿和蓝）占用8位。

可以计算出在特定分辨率下每秒所用的位数。在最低分辨率下，一个彩色帧由 $1\,024 \times 768$ 个像素构成。这表示我们需要：

$$2 \times 25 \times 1\,024 \times 768 \times 24 = 944 \text{ (Mbps)}$$

这种数据速率需要一种高数据速率技术，如同步光纤网络 (SONET) 技术。如果使用低速率技术发送视频，就需要对视频进行压缩。

在因特网上传送视频需要进行压缩。

29.2 音频和视频压缩

通过因特网传送音频和视频需要进行压缩。本节先讨论音频压缩，然后再讨论视频压缩。

29.2.1 音频压缩

音频压缩可以用于语音和音乐。对于语音，需要压缩为64-kHz的数字化信号；对于音乐，需要压缩为1.411-MHz的信号。用于音频压缩的两类技术是：预测编码和感知编码。

预测编码

在预测编码 (predictive encoding) 技术中，使用对采样间的差异进行编码取代对所有的采样值进行编码。这种类型的压缩技术常用于语音。已经定义了几种标准，如GSM (13kbps)、G.729 (8kbps) 和G.723.3 (6.4或5.3kbps)。对这些技术的详细讨论已经超出了本书的范围。

感知编码：MP3

用于创建CD质量音频的最常用的压缩技术是基于感知编码 (perceptual encoding) 的技术。如前所述，这种类型的音频至少需要1.411Mbps，如果不进行压缩是无法通过因特网进行发送的。MP3 (MPEG音频3层)，作为MPEG标准的一部分（在视频压缩的小节中讨论）就使用这种技术。

感知编码应用了声音心理学的科学知识。声音心理学是研究人们如何感知声音的。这种思想是基于人类听觉系统的某些缺陷。一些声音可以遮掩其他声音，遮掩现象在频率和时间上都可以出现。对于频率遮掩 (frequency masking)，某一频率范围内的较强声音能够部分或者全部遮掩另一个频率范围内较弱的声音。例如，如果在一个房间里，一个重金属乐队正在演奏，我们很难听到舞伴在说什么。对于暂时遮掩 (temporal masking)，很强的声音可以使

耳朵在它停止后的短时间内失去听觉。

MP3利用频率遮掩和暂时遮掩这两种现象来压缩音频信号。这种技术对频谱进行分析，并分为不同的组。完全被掩盖的频率分配零比特，部分遮掩的频率分配的比特数较小，无遮掩频率分配的比特数值较大。

MP3产生三种数据速率：96Kbps、128Kbps和160Kbps。速率是基于原始模拟声音频率范围内的。

29.2.2 视频压缩

前面已经讲过，视频是由多个帧构成的。每一帧是一幅图像。可以通过先压缩图像的方法来压缩视频。市场上目前有两种常用的标准。联合图像专家组（Joint Photographic Experts Group, JPEG）用于压缩图像，而运动图像专家组（Moving Picture Experts Group, MPEG）用于压缩视频。我们首先简单讨论JPEG，然后再讨论MPEG。

图像压缩：JPEG

如前所述，如果图片不是彩色的（灰度），那么每个像素可以用8位的整数（256级）来表示。如果图片是彩色的，那么每个像素可以用24位（ 3×8 位）表示，红、绿、蓝（RGB）分别用8位表示。为了简化讨论，我们只关注灰度图片。

在JPEG中，灰度图片被分割为 8×8 的像素块（如图29.2所示）。

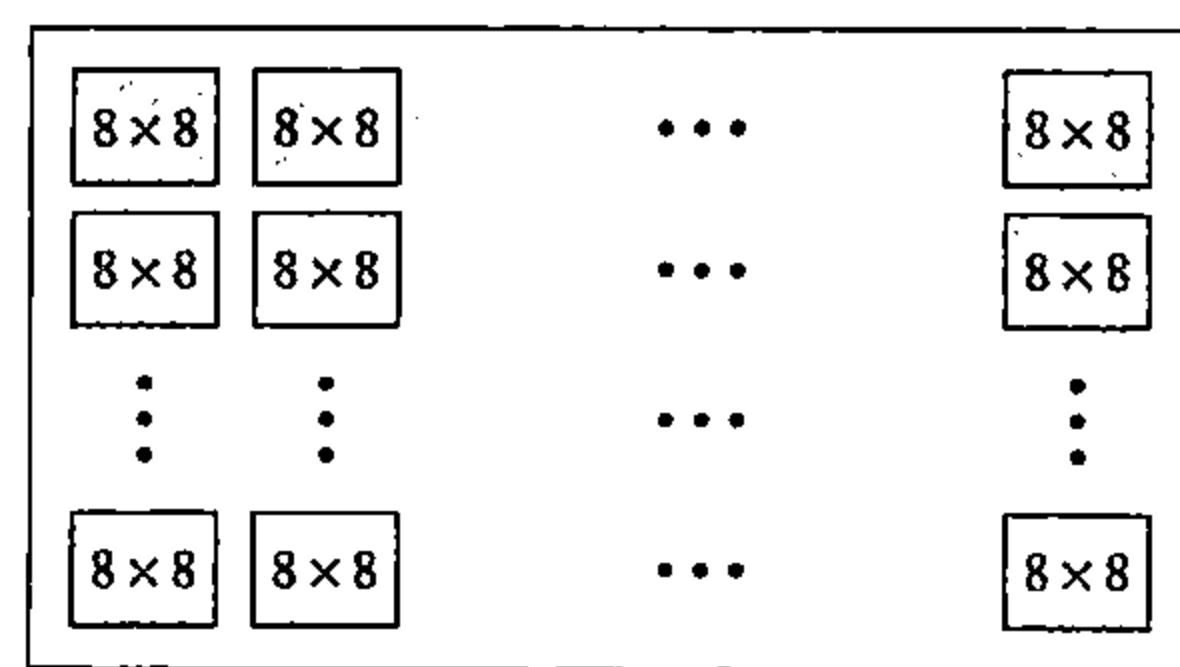


图29.2 JPEG灰度

将图片分割为块的目的是为了减少计算量。下面很快就会看到，每一张图片的数据计算次数是单元数目的平方。

JPEG的总体思想是将图片转换成揭示冗余的线性（矢量）数字集合。这样，冗余（不变化）部分就会通过使用文本压缩的方法来移除。图29.3是一个简化版本的处理过程。

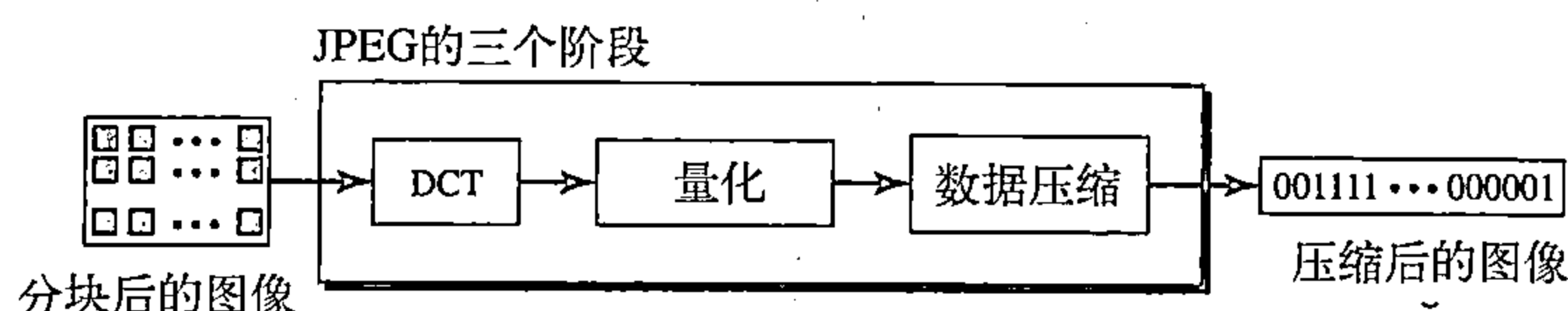


图29.3 JPEG处理

离散余弦变换 这一步将每一个64像素块进行一种变换，称为离散余弦变换（discrete cosine transform, DCT）。变换可以改变64个值，因而像素之间的相对关系得到保留，但却揭示了冗余部分。这里不给出公式，只给出三种情况下的变换结果。

第一种情况 在这种情况下，我们有一个单一灰度的块，每个像素的值是20。做变换时，我们得到第一个元素的值是非0值（左上角），其余像素的值是0。 $T(0, 0)$ 的值是 $P(x, y)$ 值的平均值（乘以一个常数），称为 dc 值（ dc 即直流，从电子工程中借用）。其余的值称为 ac 值， $T(m, n)$ 表示像素值的变化。由于没有变化，所以其余的值都是0（如图29.4所示）。

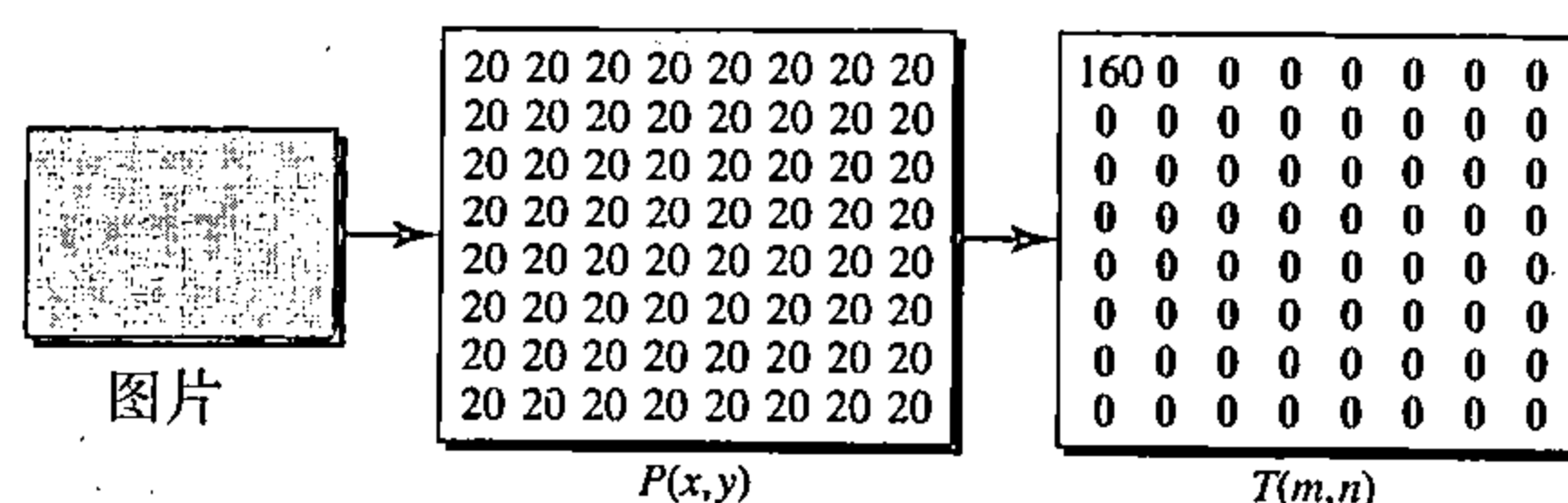


图29.4 第一种情况：单一灰度

第二种情况 在第二种情况下，我们有一个带有两种不同灰度的块。在像素值之间有一

个突变（从20跳到50）。做变换时，得到一个dc值与一系列非0的ac值。但是，只有少数非0值穿插在dc值的周围，多数值是0（如图29.5所示）。

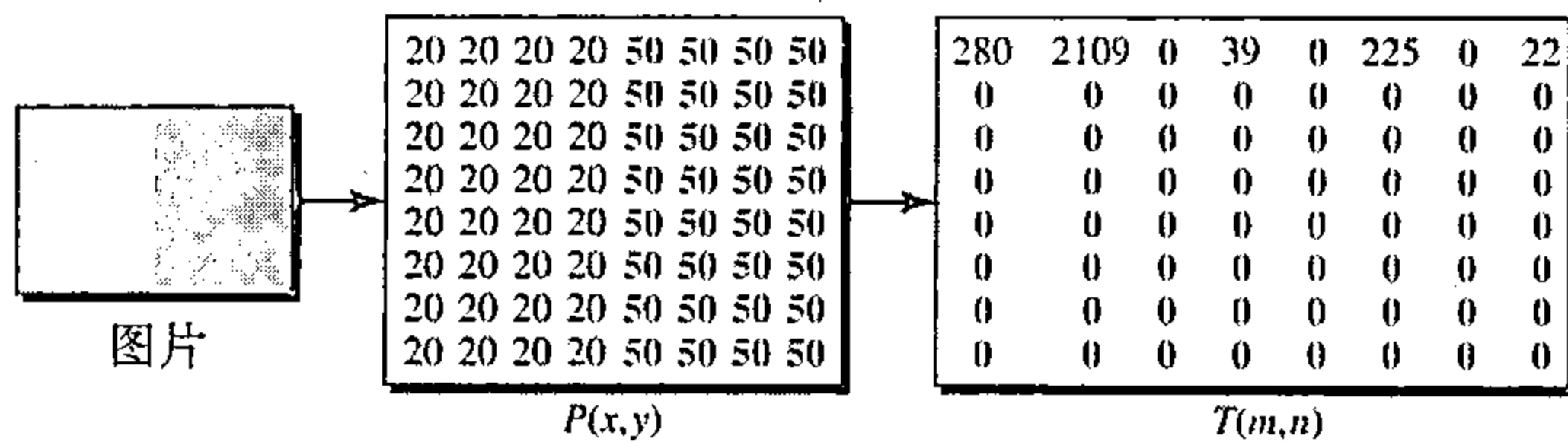


图29.5 第二种情况：两个部分

第三种情况 在第三种情况下，我们有一个块，它是逐步变化的。也就是说，相邻像素间的值没有突变。做变换时，得到一个dc值，也得到了很多非0的ac值（如图29.6所示）。

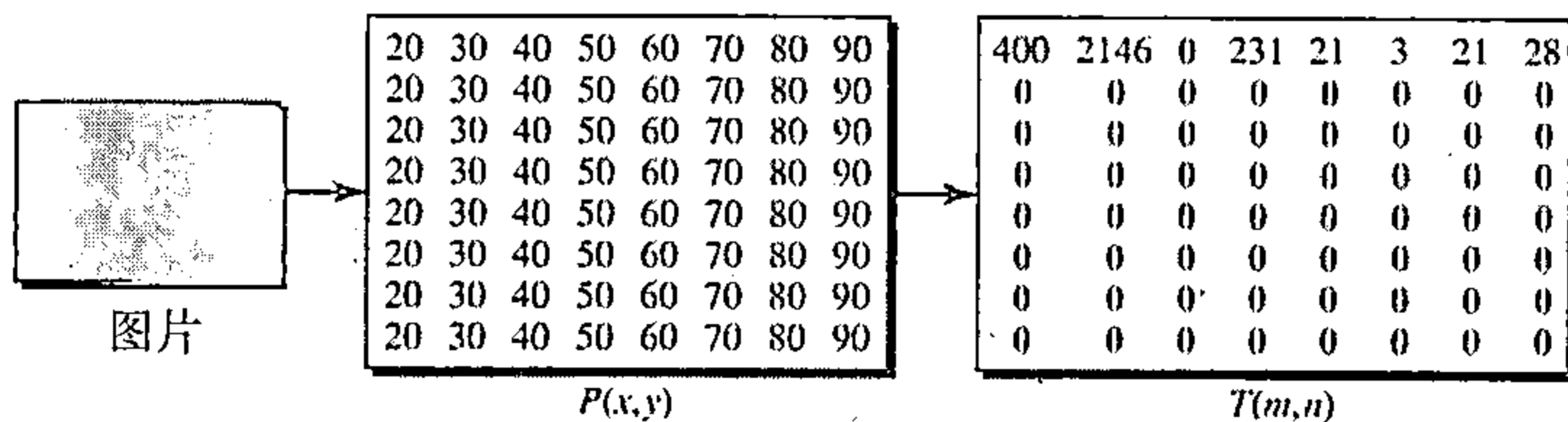


图29.6 渐变灰度等级

通过图29.4、图29.5和图29.6，我们可以得出下面的结论：

- 变换从表P中建立表T。
- dc值是所有像素的平均值（乘以一个常数）。
- ac值是变化的。
- 相邻像素之间没有变化时，生成0值。

量化 T表建立以后，将数值进行量化以减少编码所需要的位数。以前在量化（quantization）时，先要舍掉每一个值的小数部分，只保留整数部分。这里，我们用一个常数除每一个数，然后再舍掉小数部分，这样会更进一步减少所需要的位数。在大多数实现方法中，量化表（8乘8）定义了如何量化每一个数值。除数取决于值在T表中的位置。这样做是为了优化每一个特定应用中位的数量和0的数量。处理过程中唯一不可逆的阶段是量化阶段。在这里会丢掉某些信息，而且是不可恢复的。事实上，JPEG称为有损压缩的唯一原因是因为存在该量化阶段。

压缩 量化以后，从表中读取所有的值，并把冗余的0去掉。但是，为了把0组成串置于一一起，表是按对角呈“之”字形读取的，而不是按行或者按列读取。原因是如果图片没有细小的变化，则T表的右下角会是全0。图29.7说明了这一过程。

视频压缩：MPEG

运动图像专家组（MPEG）方法用于压缩视频。从原理上讲，运动图像是按顺序快速移动的一组帧，每一帧都是一幅图像。换句话说，帧是像素的空间组合，而视频是顺序发送的帧的时间组合。压缩视频，其含义是在空间上压缩每一帧，而在时间上压缩一组帧。

空间压缩 每一帧的空间压缩（spatial compression）通过JPEG完成（或者JPEG的修改版）。每一帧是一幅图像，可以单独进行压缩。

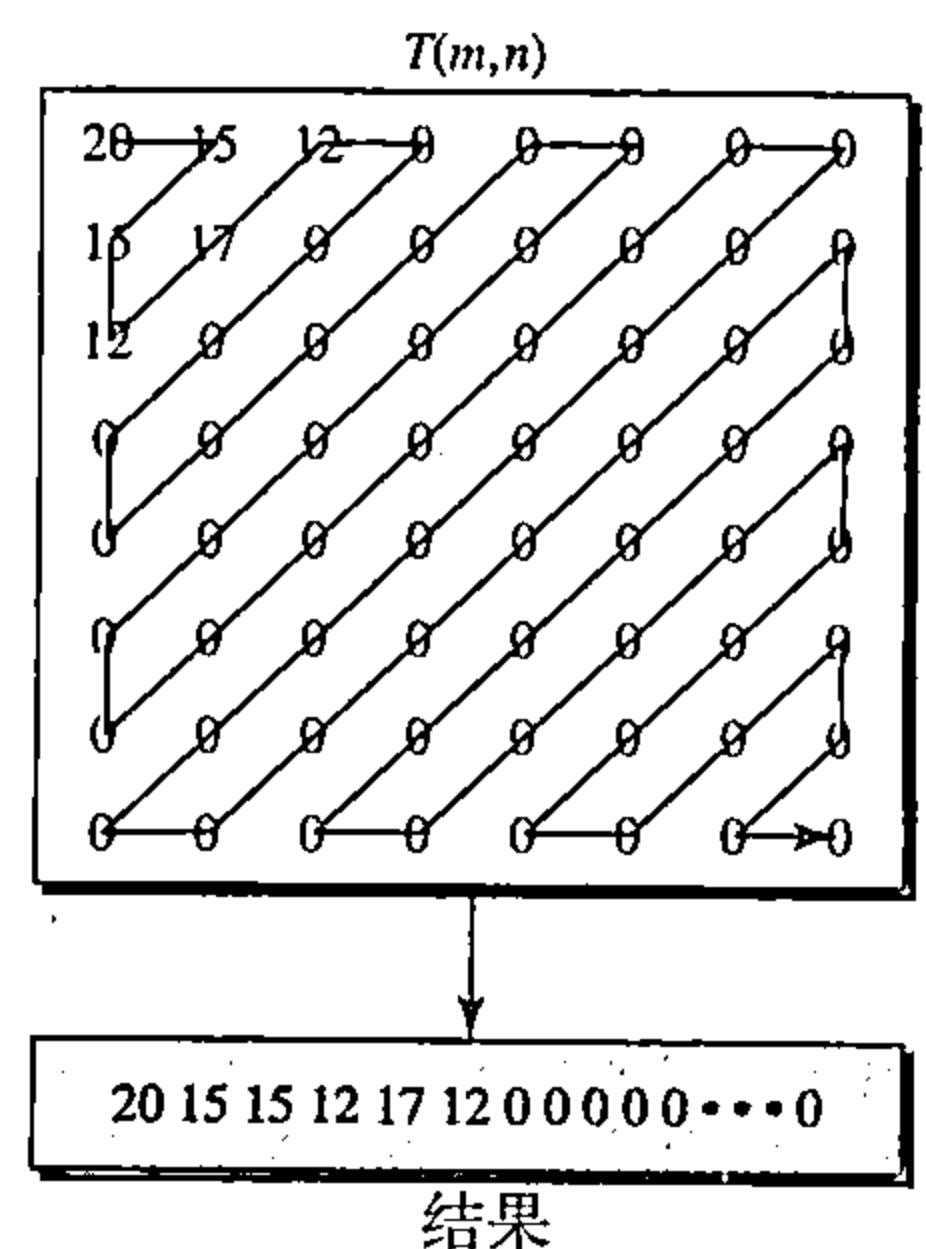


图29.7 表的读取方式

时间压缩 在时间压缩 (temporal compression) 中会去掉冗余帧。我们看电视时, 每秒接收50帧。但是, 大多数的连续帧几乎是相同的。例如, 假设某个人在说话, 除了帧中嘴唇部分外, 大多数帧与前一帧是相同的。而嘴唇这一部分在每一帧中都可能变化。

在时间域上压缩数据, MPEG方法首先将帧分为三类: I-帧、P-帧和B-帧, 如图29.8所示。

- **I-帧**。内部编码帧 (intracoded frame, I-帧) 是一个独立帧, 与其他任何帧没有关系 (与相邻前后发送的帧都没有关系)。它们在规定的时间内出现 (例如, 每一个第9帧是一个I-帧)。I-帧必须定期出现, 用于处理帧中的突然变化。由于这种突然变化, 使得前后帧无法显示。另外, 在视频播出时, 观看者可能会随时调整它的接收机。如果只在广播的开始时有一个I-帧, 那么观众随后微调其接收机时, 可能会接收不到完整的画面。I-帧独立于其他帧, 不能从其他帧中构造。
- **P-帧**。预测帧 (predicted frame, P-帧), 与前一个I-帧或P-帧有关。换言之, 每一个P-帧只包含前一帧中变化的部分。这些变化的部分当然不能覆盖大的段。例如, 一个快速移动的物体, 这些新变化不会记录在P-帧中。P-帧只能从前一个I-帧或者P-帧来构造。P-帧比其他类型的帧携带的信息要少得多, 压缩后占用的位数很少。

- **B-帧**。双向帧 (bidirectional frame, B-帧) 与前一个或者后一个I-帧或P-帧有关。换句话说, 每一个B-帧与过去和未来都有关联。注意, 一个B-帧与另一个B-帧没有任何关系。

图29.9显示了I-帧、P-帧和B-帧是如何从连续的七个帧中构造的。

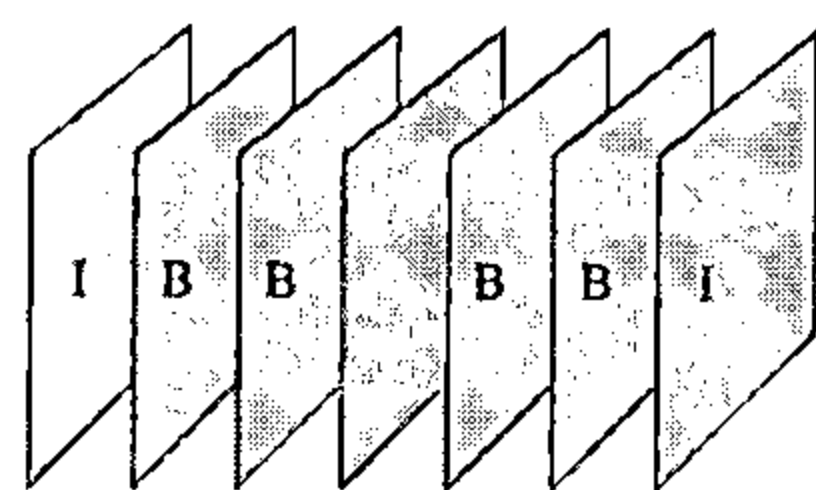


图29.8 MPEG帧

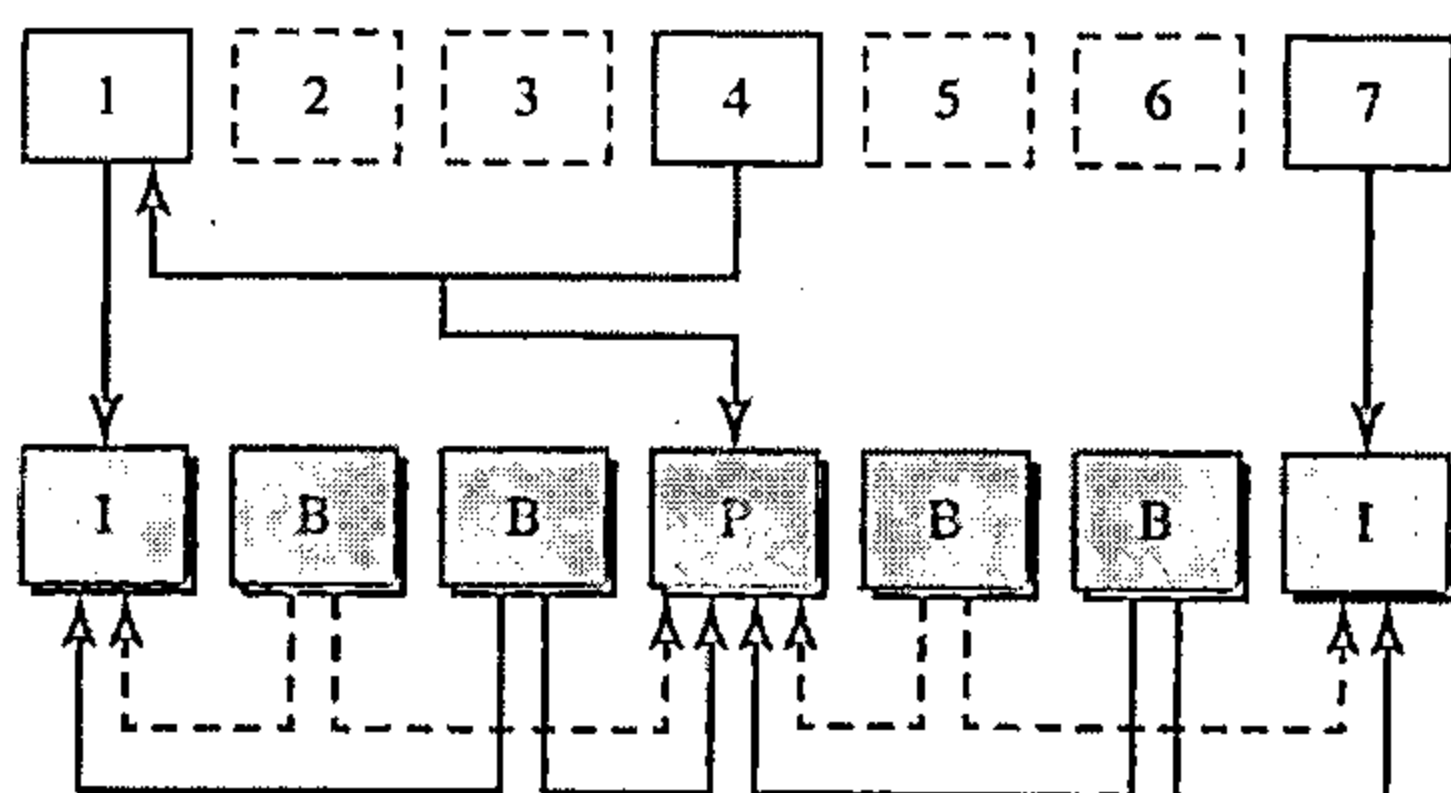


图29.9 MPEG帧的构造

MPEG已经发展了两个版本。MPEG1用于数据速率是1.5Mbps的CD-ROM。MPEG2用于数据速率是3Mbps~6Mbps的高质量DVD。

29.3 流式存储音频/视频

既然已经讨论过了音频/视频的数字化和压缩, 那么现在就可以把注意力转向一些特定的应用了。首先是流式存储的音频和视频。从Web服务器中下载这些类型的文件可能与下载其他类型的文件有所不同。要理解这一概念, 我们先使用三种方法, 每一种都有不同的复杂度。

29.3.1 第一种方法: 使用Web服务器

压缩的音频/视频文件可以像文本文件一样下载。客户端 (浏览器) 使用HTTP服务, 发送GET报文下载文件。Web服务器可以将压缩的文件发送到浏览器中。然后浏览器借助于一个通常称为媒体播放器 (media player) 的应用程序播放文件。图29.10说明了这一方法。

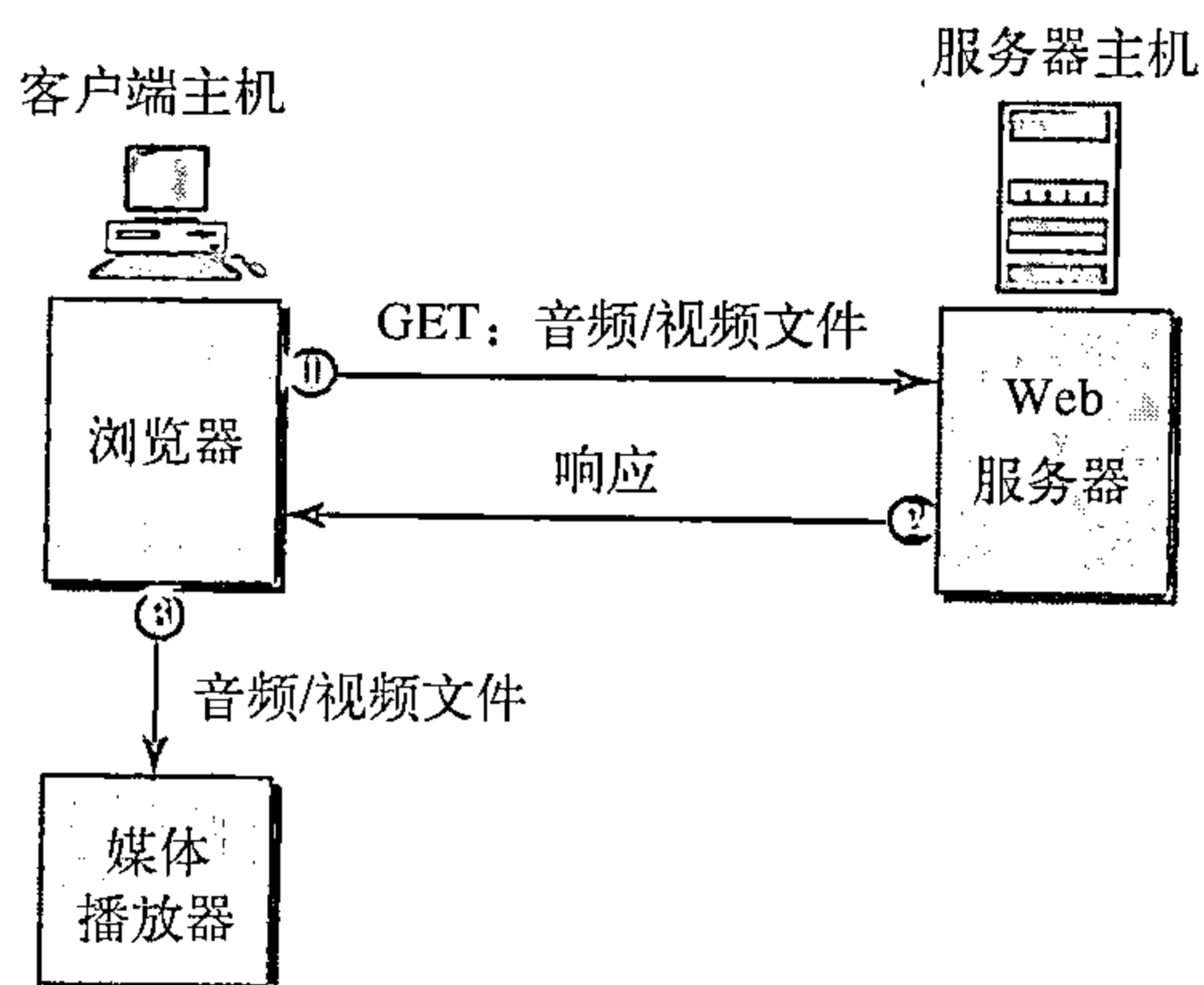


图29.10 使用Web服务器

这种方法很简单，还没有涉及到流。但是，它存在缺陷。一个音频/视频文件即使压缩以后通常仍然很大。音频文件可能包含几十兆字节，而视频文件可能包含几百兆字节。在这种实现方法中，文件在播放之前需要完整地下载下来。使用目前的数据速率，用户在文件播放之前需要几秒或者几十秒的时间来下载文件。

29.3.2 第二种方法：使用带有元文件的Web服务器

在这种方法中，媒体播放器直接连接到Web服务器并下载音频/视频文件。Web服务器存储两种文件，实际的音频/视频文件和一个元文件（metafile），元文件中包含了音频/视频文件的有关信息。图29.11说明了这种方法的步骤。

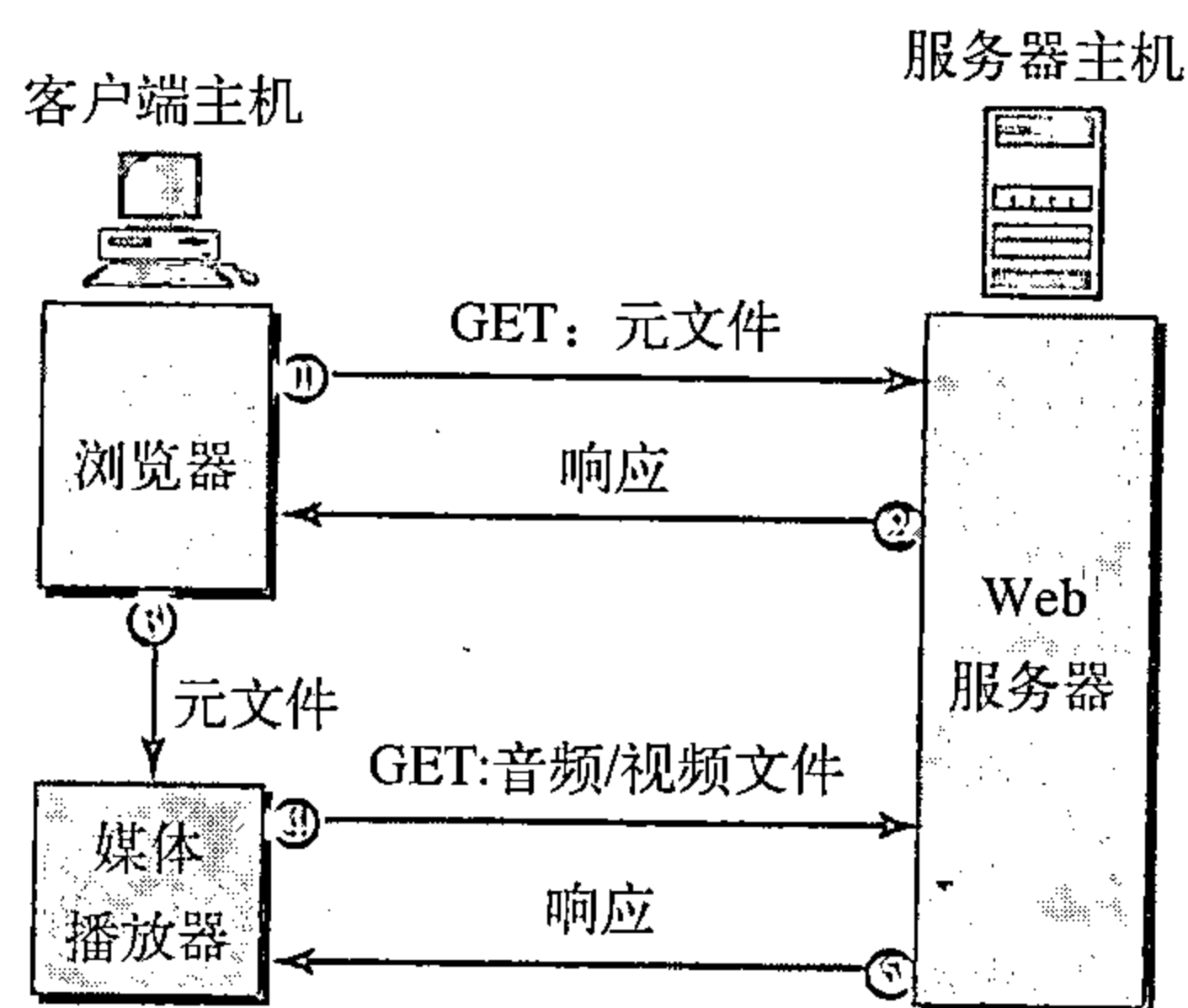


图29.11 使用带有元文件的Web服务器

1. HTTP客户端使用GET报文访问Web服务器。
2. 响应中包含元文件的有关信息。
3. 元文件传输到媒体播放器。
4. 媒体播放器使用元文件中指定的URL来访问音频/视频文件。
5. Web服务器做出响应。

29.3.3 第三种方法：使用媒体服务器

第二种方法中存在的问题是，浏览器和媒体播放器都使用HTTP服务。HTTP是运行在TCP之上的。它适合于检索元文件，但是不适合检索音频/视频文件。原因在于TCP会重发丢失或者损坏的段，这刚好与流的原理相反。所以我们不需要使用TCP和它的纠错控制，而是

应该使用UDP。但HTTP用于访问Web服务器，而Web服务器则是为TCP设计的，所以需要另外一种服务器，即媒体服务器（media server），图29.12说明了这一思想。

1. HTTP客户端使用GET报文访问Web服务器。
2. 响应中含有元文件的有关信息。
3. 元文件传送到媒体播放器。
4. 媒体播放器使用元文件中的URL访问媒体服务器以下载文件。通过任何使用UDP的协议都可以下载。
5. 媒体服务器做出响应。

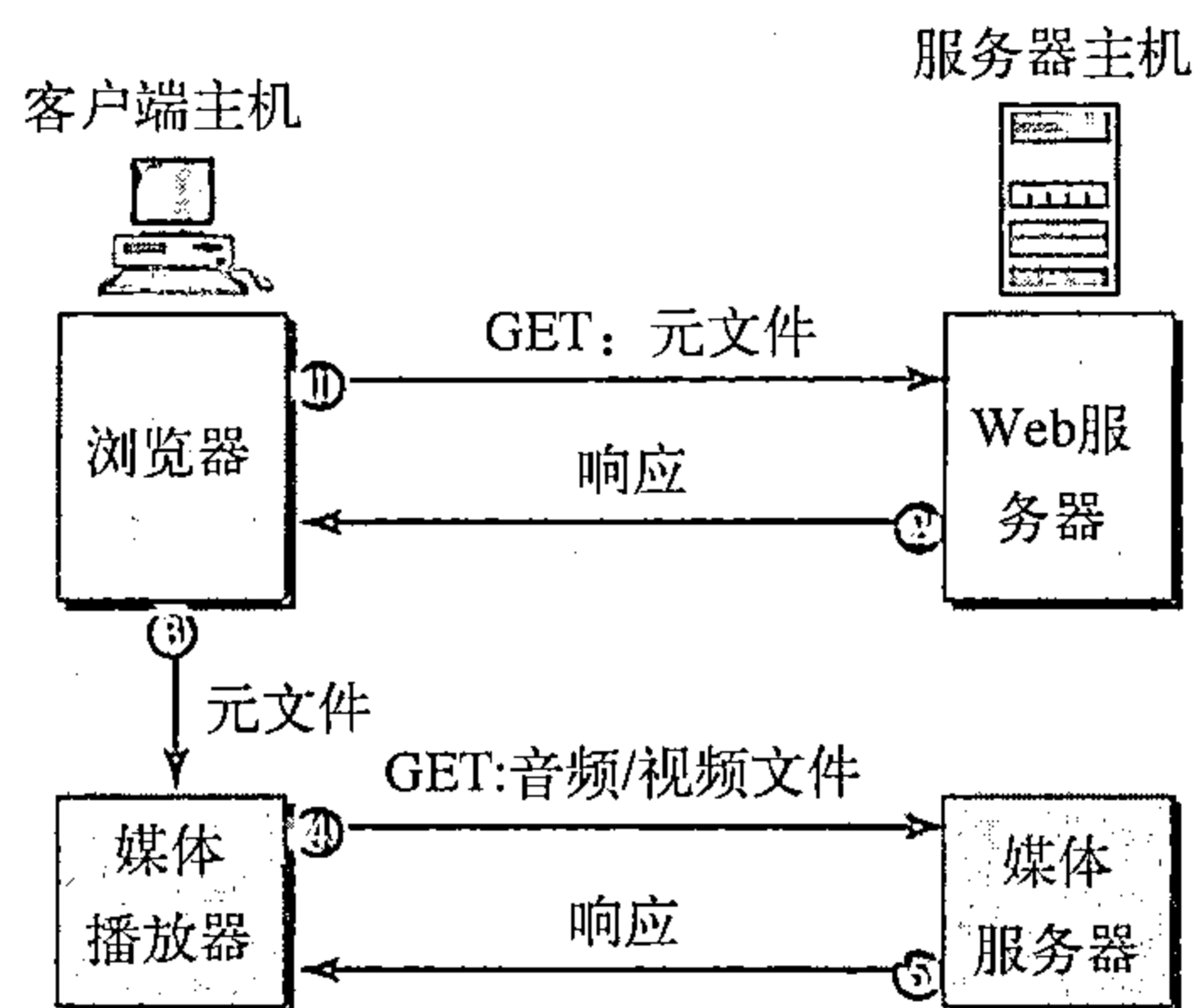


图29.12 使用媒体服务器

29.3.4 第四种方法：使用媒体服务器和RTSP

实时流协议（Real-Time Streaming Protocol, RTSP）是一种控制协议，被用来为流的处理增加更多的功能。使用RTSP，可以控制音频/视频的播放。RTSP是一种带外控制协议，与FTP中的第二种连接很相似。图29.13说明了媒体服务器和RTSP。

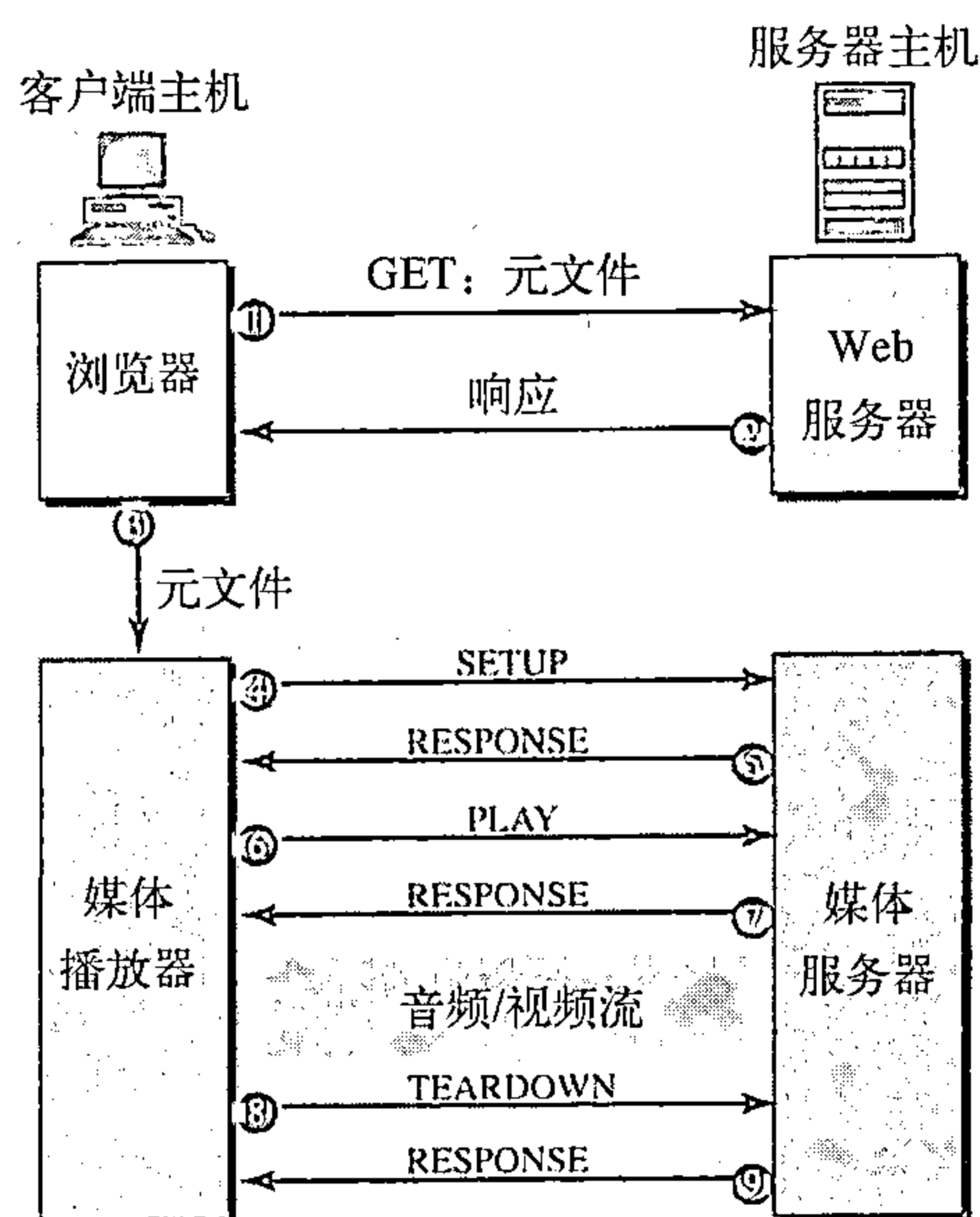


图29.13 使用媒体服务器和RSTP

1. HTTP客户使用GET报文访问Web服务器。

2. 响应中含有元文件的有关信息。
3. 元文件传送到媒体播放器。
4. 媒体播放器发送一条SETUP报文，建立与媒体服务器的连接。
5. 媒体服务器响应。
6. 媒体播放器发送一条PLAY报文以启动播放（下载）。
7. 使用一种运行在UDP之上的协议下载音频/视频文件。
8. 使用TEARDOWN报文断开连接。
9. 媒体服务器响应。

媒体播放器能够发送其他类型的报文。例如，一条PAUSE报文，能够暂时中止下载过程，通过PLAY报文，可以重新开始下载。

29.4 流式实时音频/视频

流式实时音频/视频类似于广播电台和电视台播放的音频和视频节目。区别在于电台通过因特网而不是无线电波播出。流式存储音频/视频与流式实时音频/视频之间有一些相似性，它们都对延迟很敏感，并且都不接受重传。但也有一些区别，在第一种应用情况下，通信是单播和点播的。在第二种情况下，通信是多播和实时的。实时流更适合于IP的多播服务和使用诸如UDP和RTP（后面将要讨论）这样的协议。但是目前，实时流仍然使用TCP和多个单播代替多播。在这一领域，仍然有许多需要改进的地方。

29.5 实时交互式音频/视频

在实时交互式音频/视频中，人们相互之间进行实时通信。因特网电话或者IP语音是这种应用的实例。视频会议是另一个例子，人们可以进行可视化通信和交谈。

特性

在讨论这类应用使用的协议之前，先讨论一下实时音频/视频通信的一些特性。

时间关系

分组交换网络中的实时数据，需要保留一次会话期内各分组之间的时间关系。例如，我们假定有一台实时视频服务器创建了实时视频图像，并在线发送。视频被数字化并打包分组。只有三个分组，每个分组保存10秒的视频信息。第一个分组从00:00:00开始，第二个分组从00:00:10开始，第三个分组从00:00:20开始。假定每个分组花费1秒的时间到达目的地（为了简化，夸张了一些）。接收方可以在00:00:01播放第一个分组，在00:00:11播放第二个分组，在00:00:21播放第三个分组。尽管在服务器发送和客户端在屏幕上所看到的之间有1秒的时间差，但动作基本是实时的，保留了分组之间的时间关系，而1秒的时间延迟并不重要。图29.14说明了这种思想。

但是，如果这些分组有不同的时间延迟，到达后会发生什么情况呢？例如，第一个分组在00:00:01到达（1秒的延迟），第二个分组在00:00:15时刻到达（5秒的延迟），第三个分组在00:00:27到达（7秒的延迟）。如果接收方在00:00:01时刻播放第一个分组，则会在00:00:11时刻结束播放。但是下一个分组还没有到达，要在4秒之后才会到达。远方站点观看这一视频时，在第一个分组和第二个分组之间会出现间断，同样第二个分组和第三个分组之间也有时间间断。这种现象称为抖动（jitter）。图29.15说明了这种情况。

由于分组之间的延迟，实时数据中会有抖动现象。

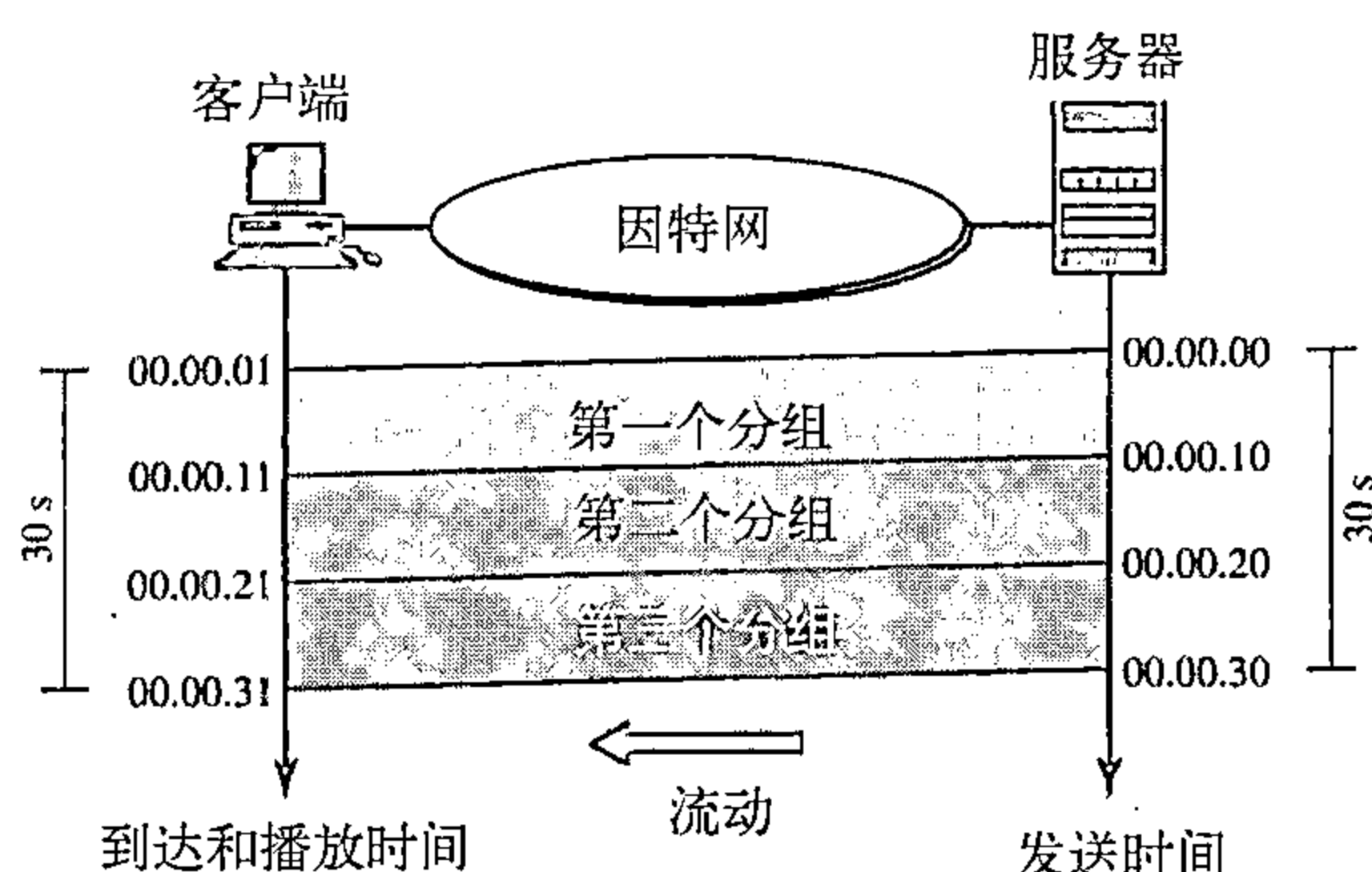


图29.14 时间关系

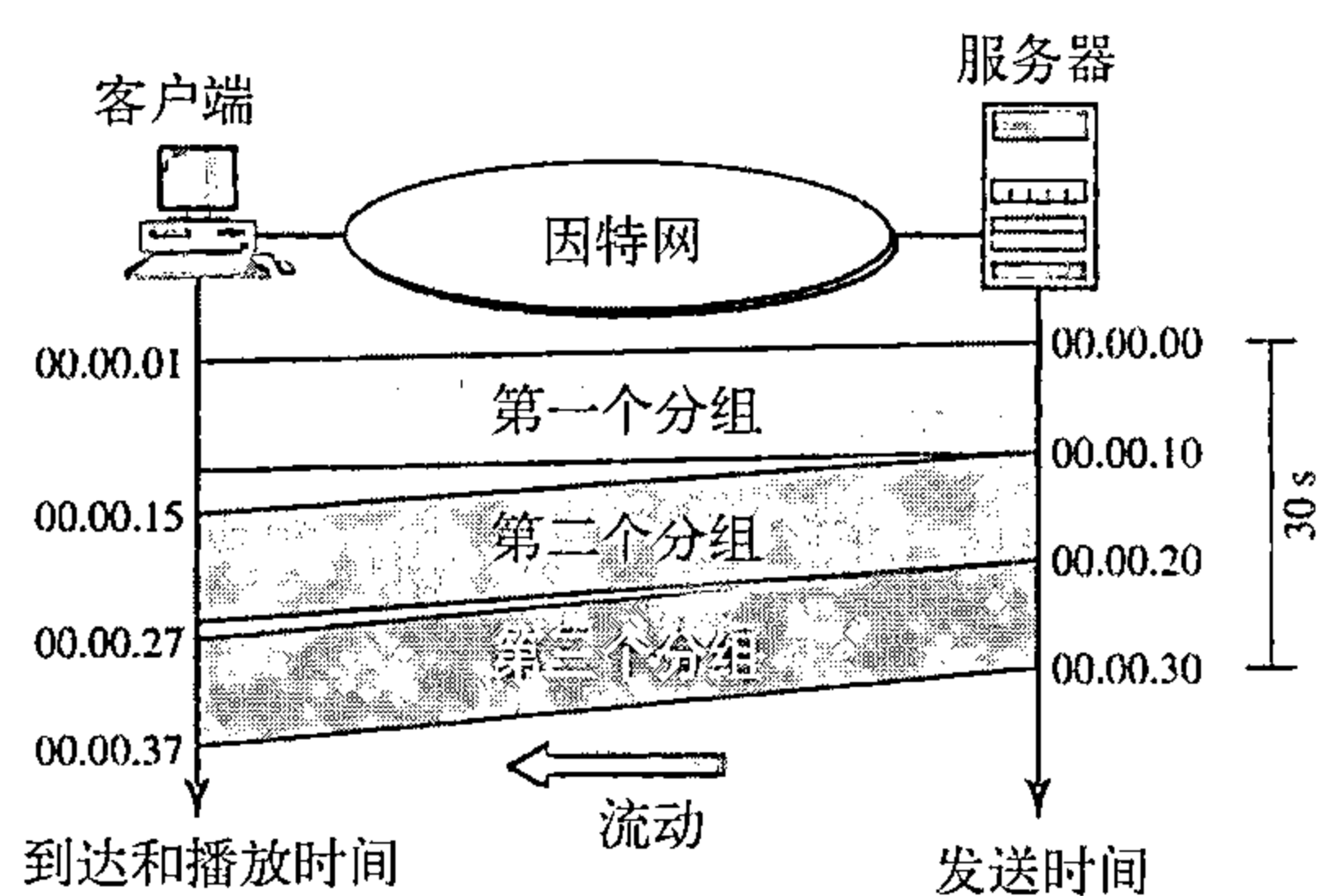


图29.15 抖动

时间戳

解决抖动的一种方案是使用时间戳 (timestamp)。如果每一个分组都有一个时间戳，标明它的生成时间与第一个（或者前一个）分组的时间间隔，那么接收方就能够将它的播放开始时间加上这一段时间。换句话说，接收方能够确定每一个分组的播放起始时间。在上面的例子中，假设第一个分组的时间戳是0，第二分组的时间戳是10，第三分组的时间戳是20。如果接收方在00:00:08时刻播放第一个分组，则第二个分组会在00:00:18时刻开始播放，第三个在00:00:28时刻开始播放。这样在分组之间就没有间断了。图29.16说明了这种情况。

为了防止抖动，可以为分组加上时间戳，将播放时间与到达时间进行分离。

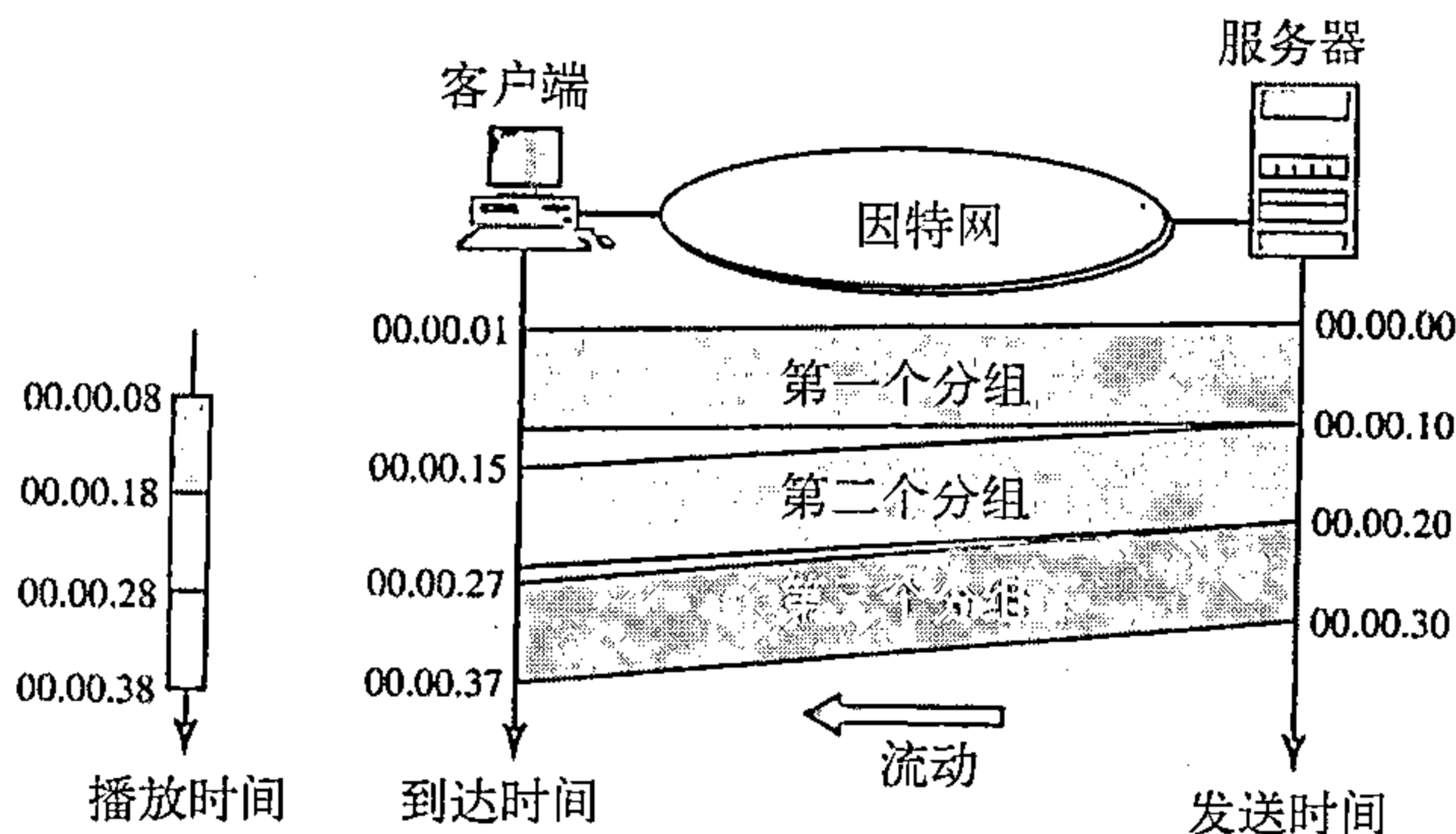


图29.16 时间戳

回放缓冲区

为了将到达时间与播放时间分离开，需要一个缓冲区存储数据直到回放开始。缓冲区是指回放缓冲区 (playback buffer)。当会话开始 (第一个分组的第一位到达) 时，接收方会延迟播放数据，直到到达一个阈值。在上一个例子中，第一个分组的第一位在00:00:01到达，阈值是7秒，回放时刻是00:00:08。阈值通过数据的时间单元计算。重新播放会一直等到数据的时间单元数等于阈值时才会开始。

数据可能会以变化的传输速率存储在缓冲区内，但是它们是以固定速率抽取和回放的。注意，缓冲区内数据的数量可以缩小或扩大，但是只要延迟小于播放阈值时间，就不会产生抖动。图29.17说明了该例在不同时刻的缓冲区的情况。

实时通信量需要有回放缓冲区。

排序

除了时间关系信息和用于实时通信量 (real-time traffic) 的时间戳以外，还需要更多的特性。每一个分组需要一个序列号。只靠时间戳是不能通知接收方是否有分组丢失的。例如，假定时间戳是0、10和20。如果第二个分组丢失了，那么接收方只收到两个分组，时间戳是0和20。接收方假定时间戳为20的分组是第二个分组，也就是第一个分组20秒钟以后产生的。接收方没有办法确定第二个分组已经丢失。要处理这种情况，需要一个序列号对分组进行排序。

实时通信量需要每一个分组都有一个序列号。

多播

多媒体播放在音频和视频会议中发挥着重要作用。通信量负载可能很重，数据通过使用多播 (multicasting) 的方式进行分发。会议在接收方和发送方之间需要双向通信。

实时通信量需要多播的支持。

转换

有时实时通信量需要转换 (translation)，转换器是一台计算机，它能够将高带宽视频信号的格式转换为低质量的窄带信号。这种转换是必要的，例如，源方以5Mbps的速率创建了一种高质量的视频信号，而是通过低于1Mbps的带宽发送给接收方。要接收到这种信号，就需要转换器将信号解码，然后再次编码成只需要低带宽的低质量视频信号。

转换的含义是改变有效载荷的编码，降低其质量以匹配接收方的网络带宽。

混合

如果有多个源方能够同时发送数据 (如在视频或者音频会议中)，通信量由多个数据流组成。要将通信量减少为一个数据流，就需要将来自多个源方的数据混合为一个数据流。混合器 (mixer) 通过数学算法将来自多个源方的信号相加，建立单一信号。

混合意味着将通信量中的多个数据流组合为一个数据流。

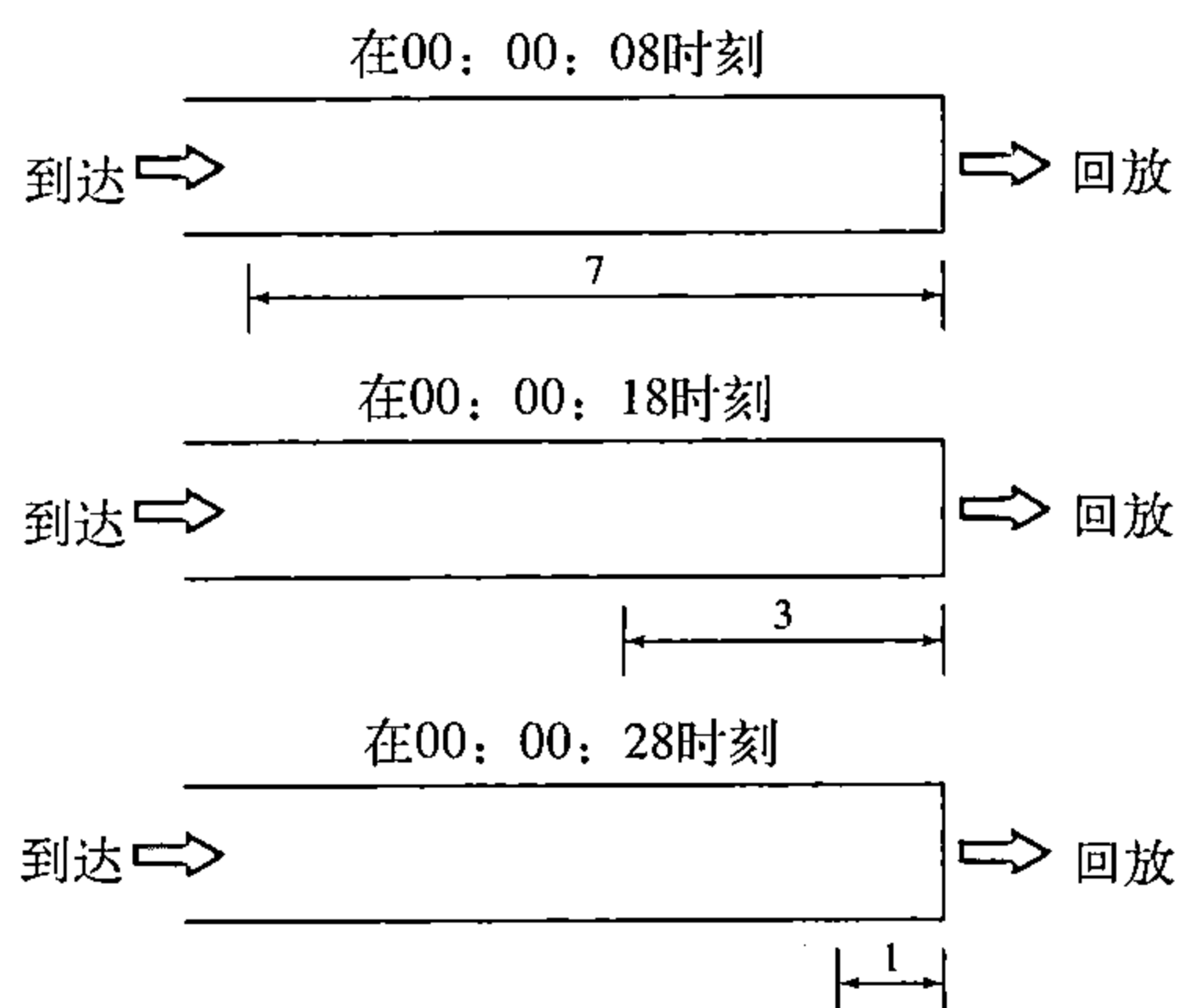


图29.17 回放缓冲区

传输层协议的支持

前面各小节提到的各个过程在应用层都能够实现。但由于它们在实时应用中有共同的特性，因而在传输层实现应该是更可取的。现在看一下哪一种现有的传输层协议适合于这种类型的通信量。

TCP不适合交互式通信量。它不提供时间戳，也不支持多播。但是它能够提供排序（序列号）。TCP的一个使它特别不适合于交互式通信量的特性就是纠错控制机制。在交互式通信量中，不允许重新发送一个丢失或者损坏的分组。如果交互式通信量中丢失或者被破坏可分组，就必须忽略它。重发会破坏时间戳和回放的整体思想。目前在音频和视频信号中有很多的冗余（即使进行了压缩），所以可以简单地忽略一个丢失的分组。远方的收听者和观看者甚至不会察觉到这种情况。

TCP，尽管有很多高级的特性，但它不适合于交互式多媒体通信量。原因是在交互式多媒体应用中不允许重发分组。

UDP更适合于交互式多媒体通信量。UDP支持多播并且没有重发策略。但是，UDP不支持时间戳、排序和混合。一个新的传输协议，实时传输协议（RTP）提供了这些混合的特性。

UDP比TCP更适合于交互式通信量。但是我们需要RTP服务（另一种传输层协议）来弥补UDP的不足。

29.6 实时传输协议

实时传输协议（Real-Time Transport Protocol, RTP）是用来处理因特网上实时通信的协议。RTP没有分发机制（多播、端口号等），它必须与UDP一起使用。RTP位于UDP和应用程序之间。RTP的主要作用是它的时间戳、排序和混合功能。图29.18说明了RTP在协议族中的位置。

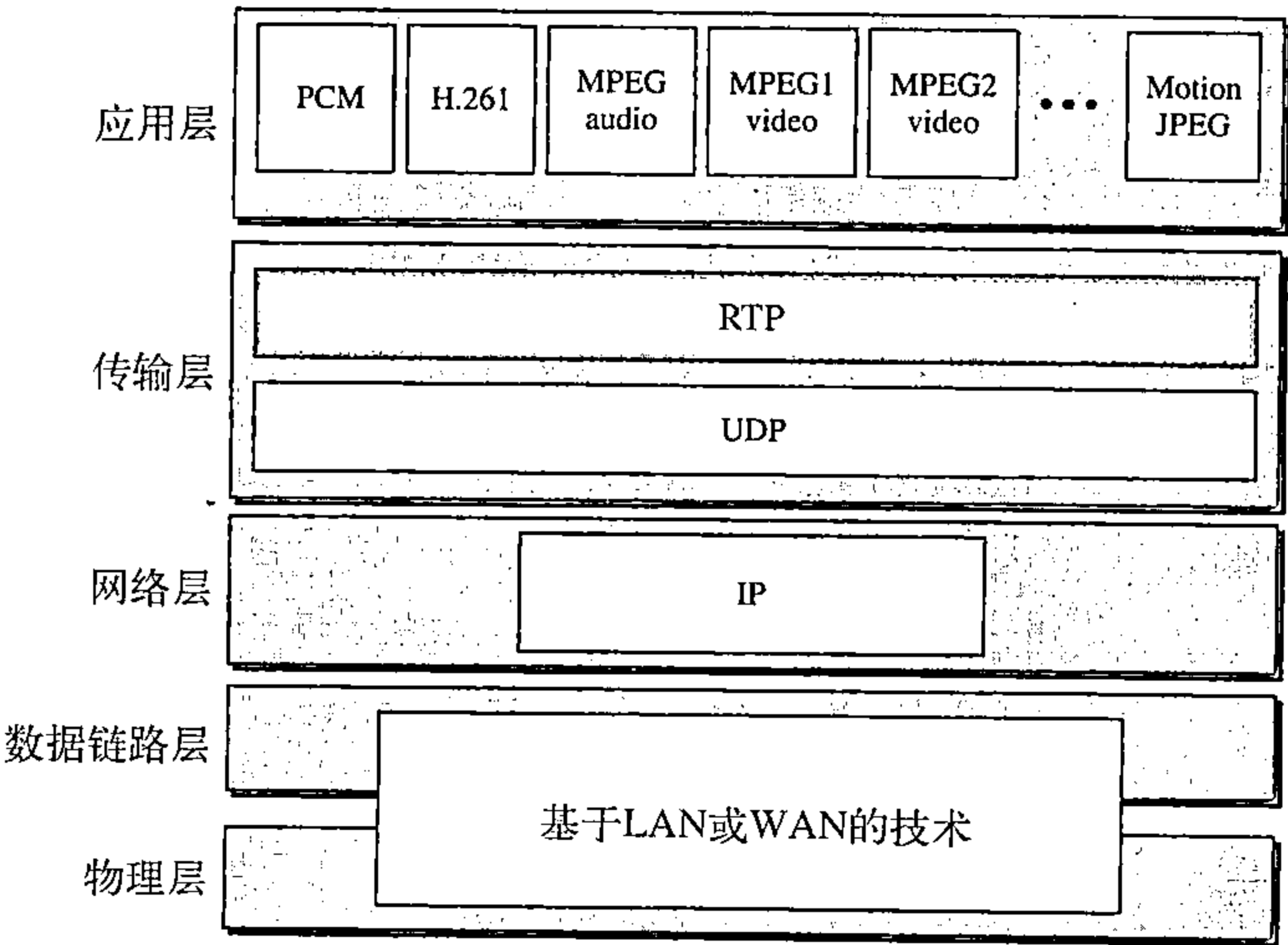


图29.18 RTP

29.6.1 RTP分组格式

图29.19说明了RTP分组头部的格式。这个格式非常简单而且通用，适合于所有的实时应用。如果应用需要更多的信息，则可以加入到头部开始的负载中。下面是每个字段的描述说明。

- 版本。这是一个2位的字段，定义了版本号。现在的版本是2。
- 填充位。这是一个1位的字段。如果值为1，分组末有填充，此时填充中最后一个字节的值为填充的长度。如果分组是加密的，则填充是标准的。如果值为0，则没有填充。
- 扩展标记位。这是一个1位的字段。如果值为1，则说明在基础头部与数据之间有额外的扩展头部；如果值为0，则没有额外的扩展头部。
- 贡献源计数（Contributor count）。这是一个4位的字段，说明了参与者的个数。注意：4位字段只能允许的最大数为15，所以最多参与者为15。
- 标志位。这1位的字段是由应用指示的标志，例如，数据的结束。
- 有效载荷类型。这是一个7位的字段，说明了有效载荷的类型。目前为止已经定义了许多有效载荷类型。在表29.1中列出了一些常用的应用。对这些类型的讨论超出了本书的范围。

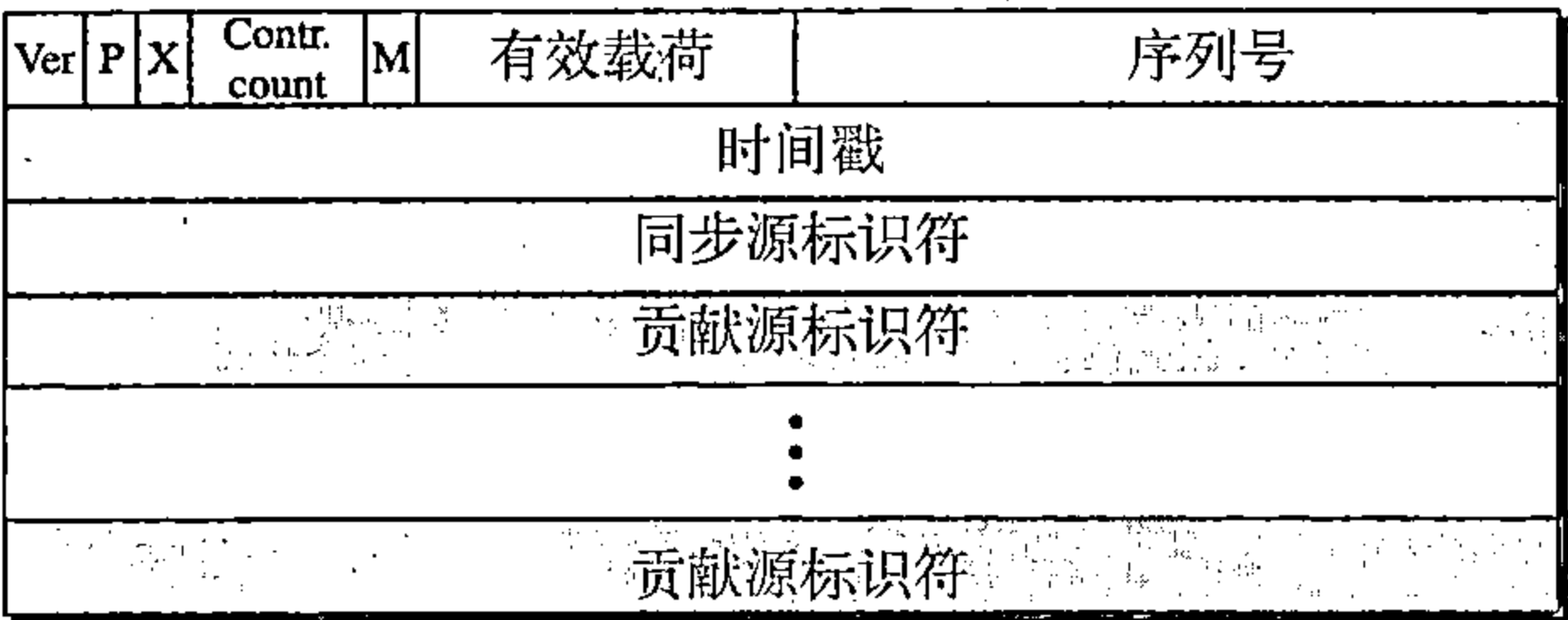


图29.19 RTP分组头部格式

表29.1 有效载荷类型

类 型	应 用	类 型	应 用	类 型	应 用
0	PCMμ音频	7	LPC音频	15	G728音频
1	1016	8	PCMA音频	26	Motion JPEG
2	G721音频	9	G722音频	31	H.261
3	GSM音频	10-11	L16音频	32	MPEG1视频
5-6	DV14音频	14	MPEG音频	33	MPEG2视频

- 序列号。这是16位长的字段，用来给RTP分组进行编号。第一个分组的序列号是随机选择的；对后续的每一个分组则加1。接收者用序列号检测丢失或者次序颠倒的分组。
- 时间戳。这是32位长的字段，用来说明各分组之间的时间关系。第一个分组的时间戳是一个随机数；对随后的每一个分组，它的时间戳的值为前一分组的值与第一个字节产生的时间之和（取样值）。时钟计时单元的值依赖于应用，例如，音频应用通常产生160字节的大块，那么这个应用的时钟计时单元就是160。这个应用的时间戳的值就是每一个RTP分组累加160。
- 同步源标识符。如果只有一个源，那么这个32位就定义了源。然而，如果存在多个源，那么混合器是同步源，其他源都是贡献源。源标识符的值是由源选取的一个随机数。协议提供冲突情况下的一个策略（两个源开始具有相同的序列号）。
- 贡献源标识符。每个32位的标识符（最大值是15）定义一个源，当会话中有多个源时，混合器是同步源而其余的源都是贡献源。

29.6.2 UDP端口

尽管RTP本身是一种传输层协议，但RTP分组并非直接封装为IP数据报。相反，RTP被当

作应用程序对待，被封装成UDP用户数据报。但是，与其他应用程序不同，没有为RTP分配熟知端口。端口可以按需要选择，这里只有一个限制：端口号必须是偶数。下一个端口号（奇数）由RTP的伴随协议——实时传输控制协议（RTCP）——使用。

RTP使用的临时端口号是偶数的UDP端口。

29.7 实时传输控制协议

RTP只支持一种类型的报文，该报文负责将数据从源方传送到目的方。在多数情况下，在会话中还需要其他报文。这些报文用于控制流量和数据质量，允许接收方向一个或者多个源方发送反馈。实时传输控制协议（Real-Time Transport Control Protocol, RTCP）就是为这种目的而设计的。RTCP有五种类型的报文，如图29.20所示。紧跟在每个方格后面的数字定义了报文的类型。

29.7.1 发送方报告

发送方报告由会议中的主动发送方定期发送，用来报告在一定的时间间隔内所有RTP分组的发送和接收的统计数据。发送方报告包括一个绝对时间戳，是从1970年1月1日午夜算起的秒数。绝对时间戳允许接收方同步不同的RTP报文。当音频和视频同时传输时（音频和视频传输使用单独的相对时间戳），这显得特别重要。

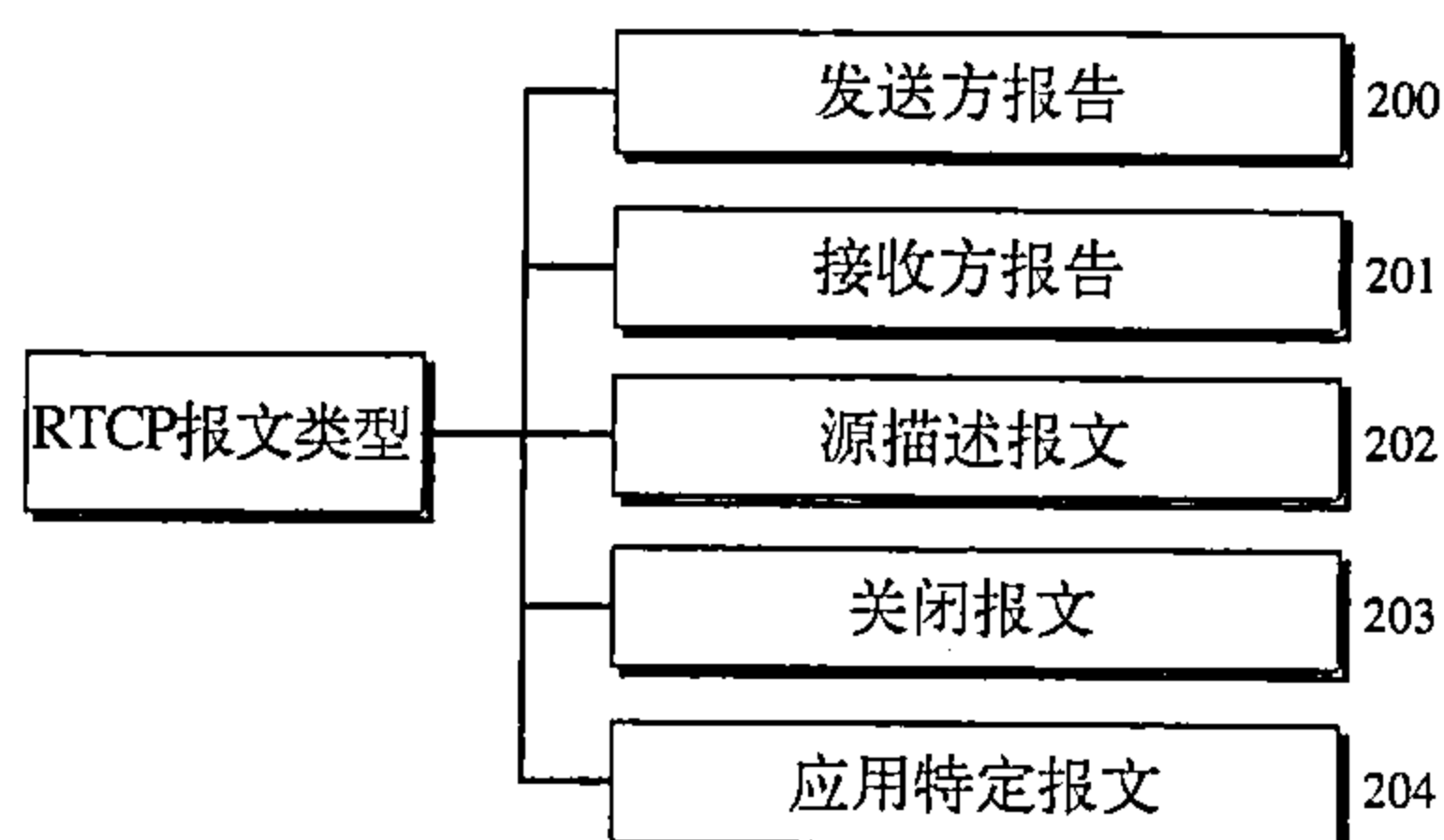


图29.20 RTCP报文类型

29.7.2 接收方报告

接收方报告是给被动参与者的，这些被动参与者不发送RTP分组。报告通知发送方和接收方有关服务质量的信息。

29.7.3 源描述报文

源方周期性发送源描述报文，提供关于自身的附加信息。这种信息可能是名字、电子邮件地址、电话号码、源控制者或者所有者的地址。

29.7.4 关闭报文

源方发送关闭报文来关闭流。它允许源方宣布要离开会议。尽管其他源方能够检测到一个源方的缺席，但是这条报文可以直接宣布。它对混合器也是非常有用的。

29.7.5 特定应用报文

特定应用报文是希望使用新的应用（没有在标准中定义）的一个应用分组。它允许定义新的报文类型。

29.7.6 UDP端口

与RTP一样，RTCP没有使用熟知的UDP端口。它使用临时端口。选择的UDP端口号必须是紧随UDP为RTP选定的端口号的那个数，它必须是一个编号为奇数的端口。

RTCP使用奇数的UDP端口号，该端口号必须是RTP选定的端口号的下一个数。

29.8 IP语音

下面主要讨论一种实时交互式音频/视频应用：IP语音（voice over IP），即因特网电话。这种思想是将因特网作为电话网络使用，并且提供一些附加功能。这种应用允许通过分组交换网络实现双方通信，从而取代了线路交换网络中的通信。SIP和H.323两种协议用来处理这种类型的通信。下面简要介绍一下这两种协议。

29.8.1 SIP

会话初始化协议（Session Initiation Protocol, SIP）由IETF设计。它是一种应用层协议，能够建立、管理和终止多媒体会话（呼叫）。它可以用来创建双方、多方或者多播会话。SIP设计上是独立于下面的传输层的，它能够运行在UDP、TCP或者SCTP之上。

报文

SIP是一种类似HTTP的基于文本的协议。与HTTP类似，SIP也使用报文。已定义的六种SIP报文，如图29.21所示。

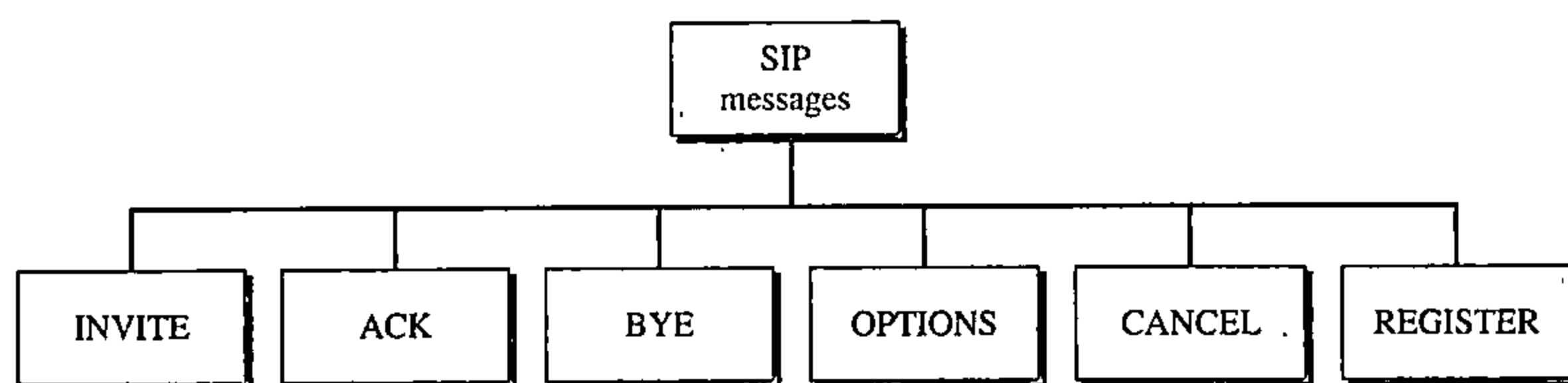


图29.21 SIP报文

每一种报文都有头部和主体。头部由多行构成，描述了报文的结构、呼叫方的功能、媒体类型等。下面给出每一种报文的简单描述，然后在简单会话中说明它们的应用。

呼叫方使用INVITE报文初始化会话。被呼叫方应答呼叫，呼叫方发送ACK报文用于确认。BYE报文可以终止会话。OPTIONS报文查询一台机器的功能。CANCEL报文取消已经启动的初始化进程。REGISTER报文会在被呼叫方不可用时建立连接。

地址

在常规的电话通信中，一个电话号码可用于识别发送方，而另一个电话号码用于识别接收方。SIP非常灵活。在SIP中，电子邮件地址、IP地址、电话号码和其他类型的地址都能用于识别发送方和接收方。但是，地址必须使用SIP格式（也就是呼叫方案）。图29.22列出了一些常用的格式。

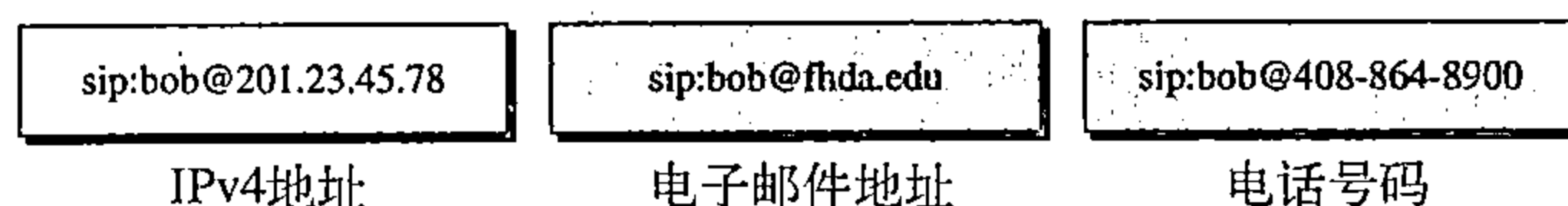


图29.22 SIP格式

简单会话

使用SIP的简单会话由三个模块构成：建立、通信和终止。图29.23说明一个使用SIP的简单会话。

建立会话 在SIP中建立一次会话需要三次握手。呼叫方使用UDP、TCP或者SCTP发送INVITE报文开始通信。如果被呼叫方愿意启动会话，她会发送一条响应报文。为了确认已经接收到的回复代码，呼叫方会发送一条ACK报文进行确认。

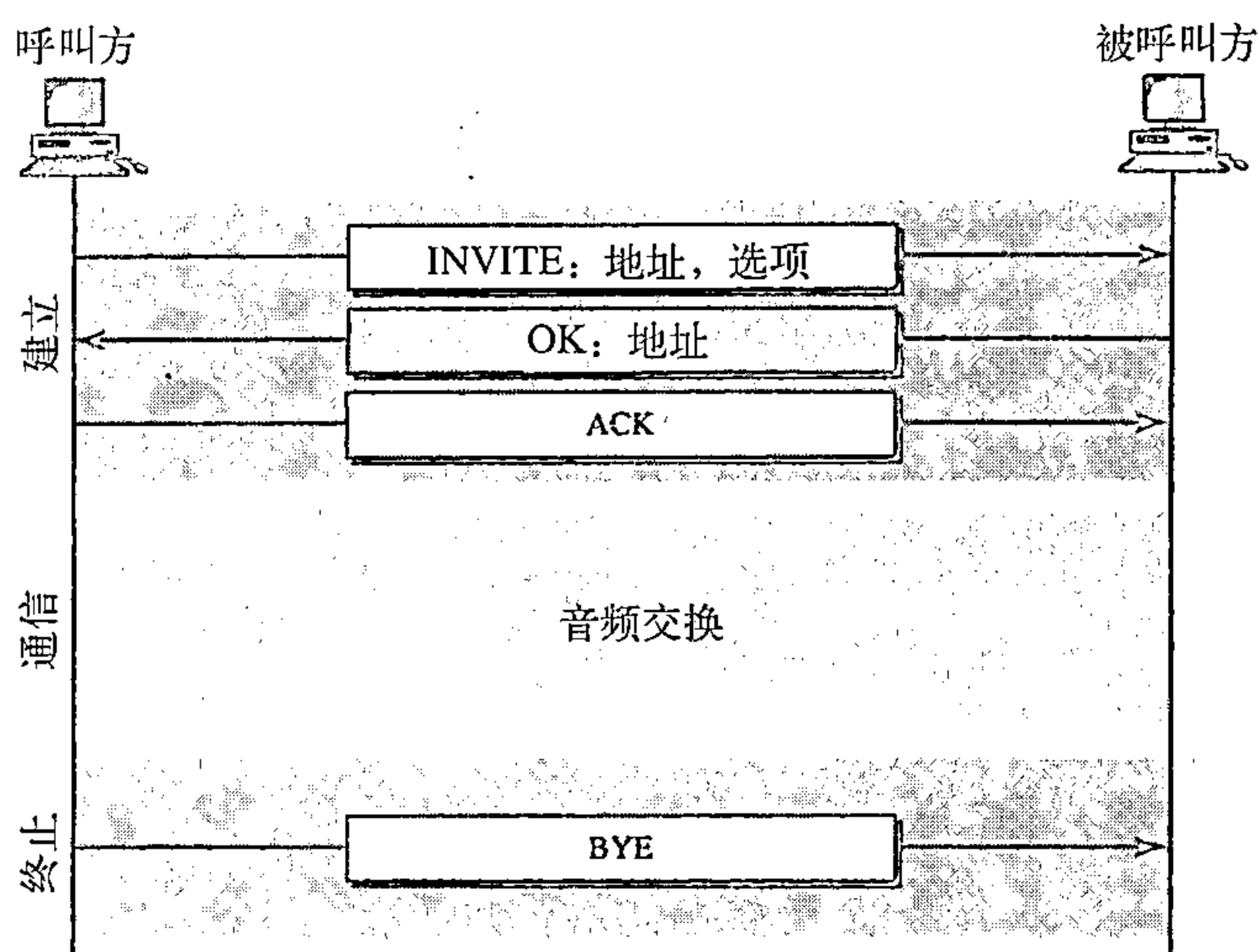


图29.23 SIP简单会话

通信 会话建立以后，呼叫方和被呼叫方能够通过两个临时端口进行通信。

终止会话 任何一方发送一条BYE报文，会话就会结束。

跟踪被呼叫方

如果被呼叫方没有坐在终端前面，会发生什么情况呢？她可能离开了自己的系统，或者在另一台终端前面。如果使用DHCP，他甚至不会有固定的IP地址。SIP使用一种机制（与DNS中的一种类似），能够查找到被呼叫方所处的终端的IP地址。要执行跟踪，SIP使用注册的概念。SIP规定了一些服务器作为注册服务器（register server）。任何时刻，用户至少注册到一台服务器上，这台服务器就能知道被呼叫方的IP地址。

当呼叫方需要与被呼叫方通信时，呼叫方可以用电子邮件地址代替在INVITE报文中的IP地址。报文会传送到代理服务器。代理服务器发送一条查询报文（不属于SIP的一部分）到某台被呼叫方注册的服务器。当代理服务器从注册服务器接收到回复报文时，代理服务器取出呼叫方的INVITE报文，插入新发现的被呼叫方的IP地址，然后将这条报文发送到被呼叫方。图29.24说明了这一过程。

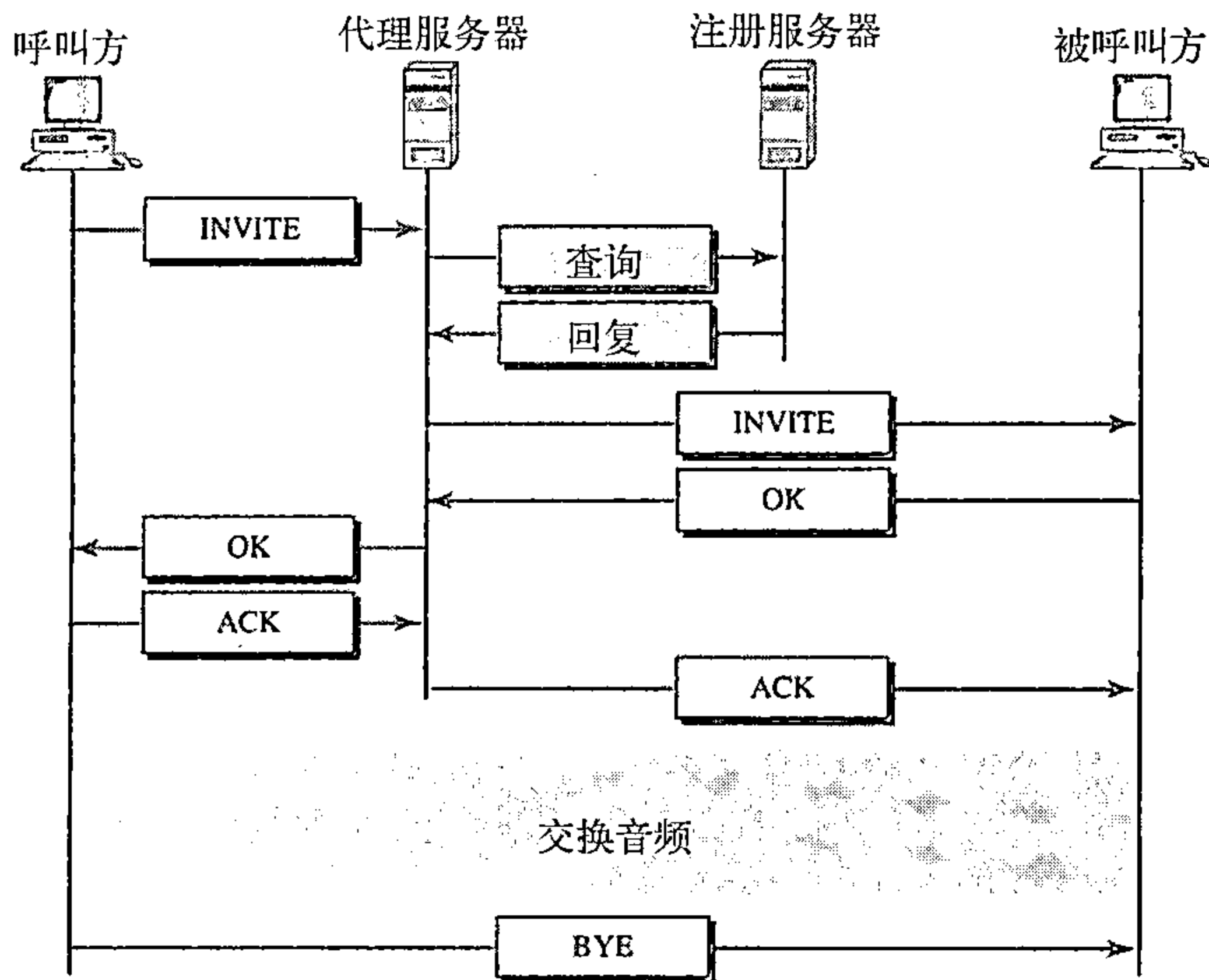


图29.24 跟踪被呼叫方

29.8.2 H.323

H.323是ITU制定的一个标准，它允许公共电话网络上的电话与连接到因特网上的计算机（在H.323中，称为终端）进行通话。图29.25说明了H.323的总体结构。

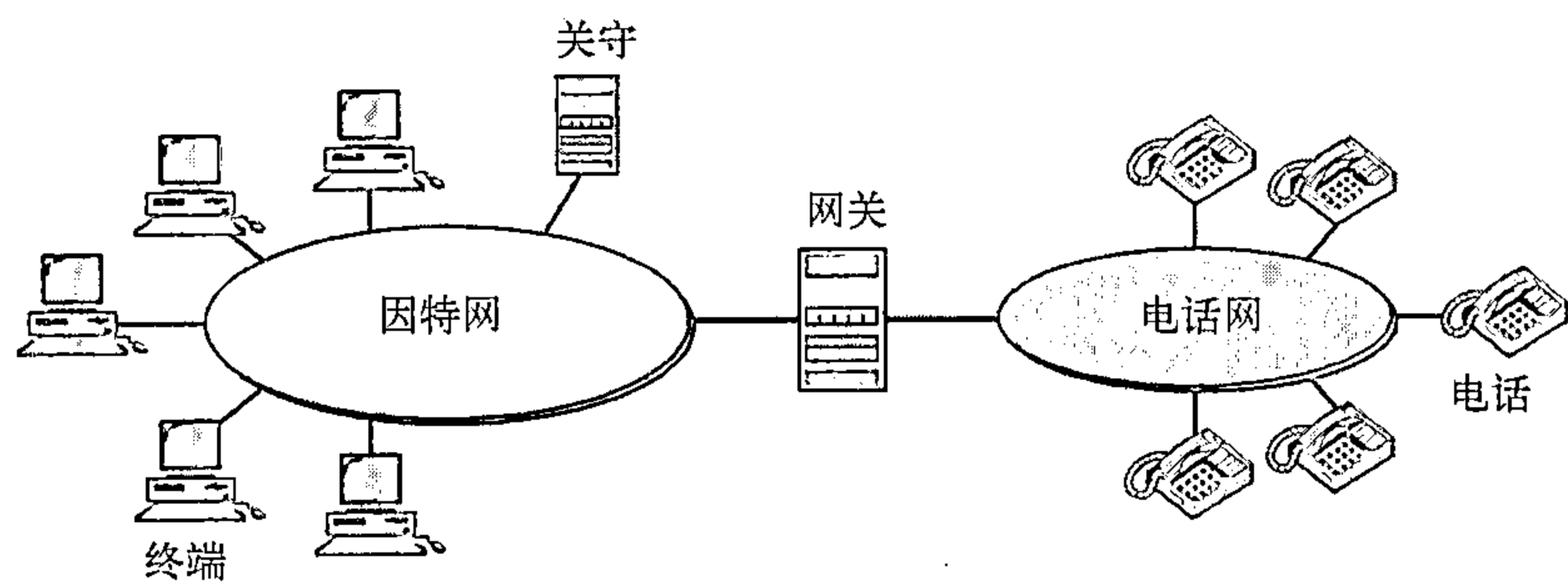


图29.25 H.323结构

网关（gateway）将因特网连接到电话网络中。通常，网关是一种五层设备，能够把报文从一个协议栈转换到另一个协议栈。此处的网关实际上做同样的事情，它将电话网络报文转换为因特网报文。本地局域网的关守（gatekeeper），承担着注册服务器的角色，正如在SIP协议中讨论的一样。

协议

H.323使用多种协议建立和维护音频（或者视频）通信。图29.26列出了这些协议。

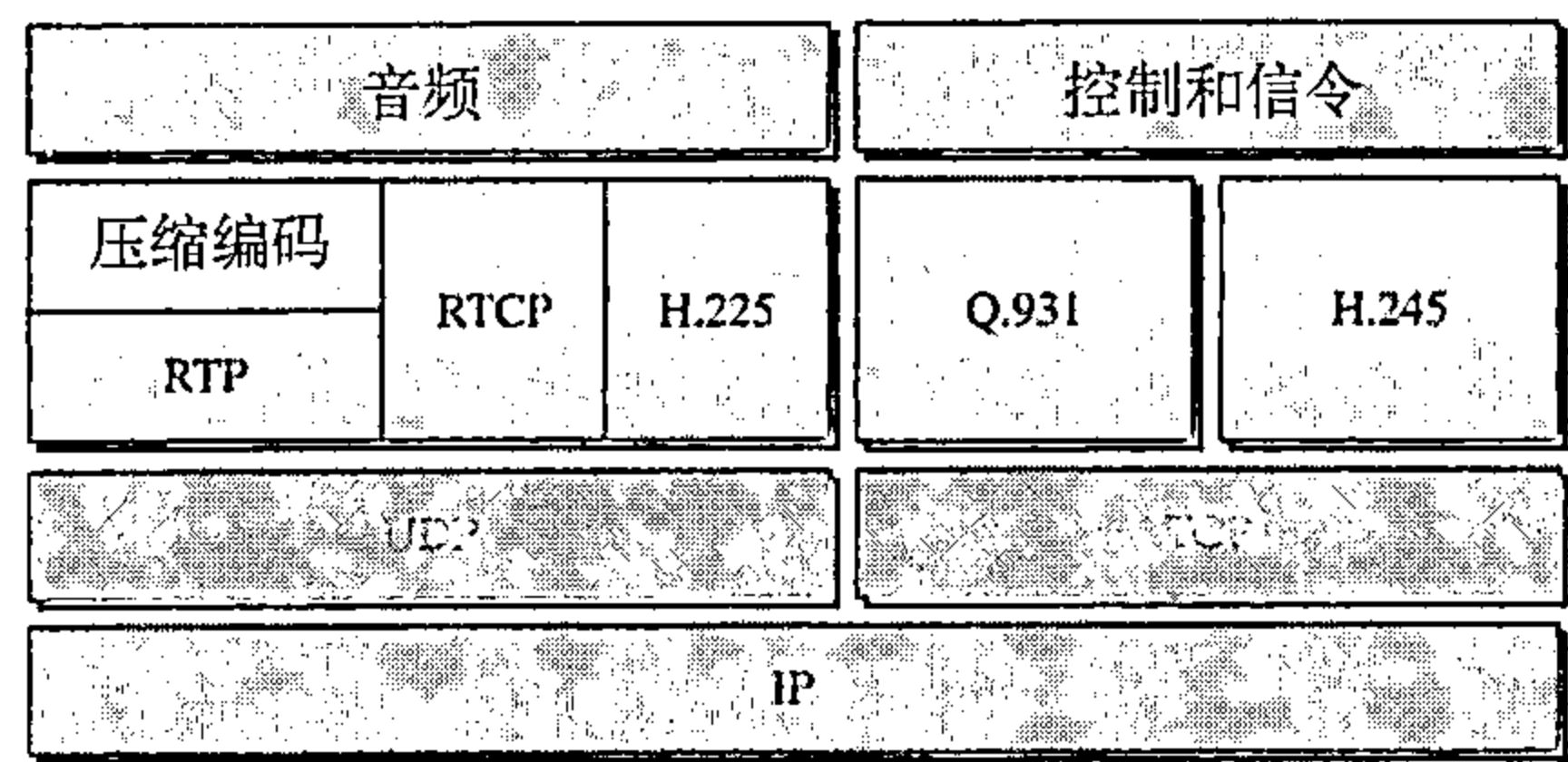


图29.26 H.263协议

H.323使用G.71或G.723.1进行压缩。使用一个名为H.245的协议实现通信方之间压缩方法的协商。Q.931协议用于建立和终止连接。H.225或RAS（Registration/Administration/Status）协议用于与关守的注册。

操作

下面用一个简单的例子说明使用H.323协议进行电话通信的操作步骤。图29.27说明了一台计算机终端与一部电话之间通信的步骤。

- 1. 终端向关守发送广播报文。关守用它自己的IP地址响应。
- 2. 终端与关守通信，使用H.225协商带宽。
- 3. 终端、关守、网关和电话使用Q.931通信，以建立连接。
- 4. 终端、关守、网关和电话通信，使用H.245协商压缩方法。
- 5. 终端、网关和电话使用RTP在RTCP的管理下交换音频。
- 6. 终端、网关和电话使用Q.931终止通信。

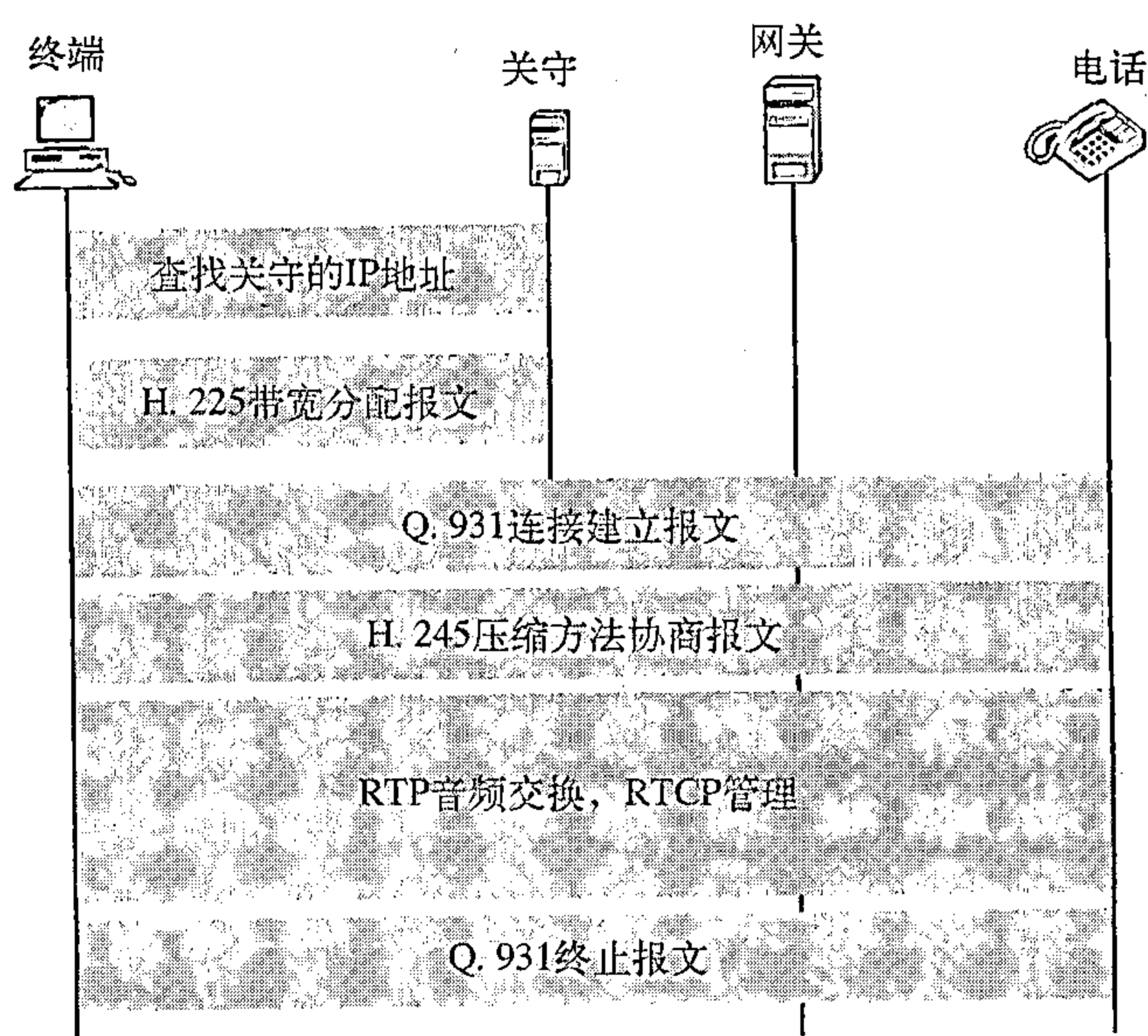


图29.27 H.323实例

推荐读物

为了深入讨论与本章有关的主题，推荐下面的读物和网站。在本书末列出方括号[……]中的参考资料。

读物

多媒体通信在[Hal01]和[Tan03]的7.4节中有讨论；图像和视频压缩在[Dro02]中讨论。

网站

www.ietf.org/rfc.html关于RFC的信息

本章小结

- 可以下载音频/视频文件以后使用（流式存储音频/视频），或者通过因特网向客户端播放（流式实时音频/视频）。因特网还可用于实时音频/视频交互。
- 音频和视频在通过因特网发送之前需要进行数字化处理。
- 音频文件通过预测编码或感知编码的方法进行压缩。
- JPEG是压缩图片和图像的一种方法。
- JPEG的处理过程包括：数据分块、离散余弦变换、量化和无损压缩。
- MPEG是一种压缩视频的方法。
- MPEG包括空间压缩和时间压缩。前者与JPEG类似，而后者去除冗余帧。
- 下载流式音频/视频文件可以使用Web服务器，或者带有元文件的Web服务器，或者媒体服务器，或者媒体服务器和RTSP。
- 分组交换网络中的实时数据需要保留一次会话期内的各个分组之间的时间关系。
- 在接收方，连续分组之间的间断会引起一种称为抖动的现象。
- 抖动可以通过使用时间戳和灵活选择回放时间来进行控制。
- 回放缓冲区会一直保存数据，直到回放开始。

- 接收方会延迟回放在回放缓冲区内保存的实时数据，直到达到设定的阈值。
- 实时数据分组中的序列号提供了一种纠错控制的方式。
- 实时数据是以多播的形式发送给接收方的。
- 实时通信量有时需要转换器，将高带宽信号转换为低质量窄带宽信号。
- 混合器可用于将来自不同源方的信号组合为一个信号。
- 实时多媒体通信量需要UDP和实时传输协议（RTP）。
- RTP负责处理时间戳、排序和混合。
- 实时传输控制协议（RTCP）提供了流控制、数据质量控制并为源方提供反馈。
- IP语音是一种实时交互式音频/视频应用。
- 会话初始化协议（SIP）是一种应用层协议，能够建立、管理和终止多媒体会话。
- H.323是一种ITU标准，允许连接在公众电话网的电话与连接在因特网上的计算机之间进行通话。

习题

复习题

1. 流式实时音频/视频与流式存储音频/视频有何不同？
2. 频率遮掩与时间遮掩有什么不同？
3. 在流式存储音频/视频中，元文件有何作用？
4. 在流式存储音频/视频中，RTSP有何用途？
5. 抖动是如何影响实时音频/视频的？
6. 讨论SIP是如何用于多媒体传输的。
7. 什么情况下使用JPEG？什么情况下使用MPEG？
8. 在JPEG处理过程中，数据分块的作用是什么？
9. 在JPEG处理过程中，为什么需要DCT？
10. 与时间压缩相比，空间压缩指的是什么？

练习题

11. 在图29.17中，在下列时刻，回放缓冲区中有多少数据？
 - a. 00:00:17
 - b. 00:00:20
 - c. 00:00:25
 - d. 00:00:30
12. 试比较TCP与RTP，它们的功能相同吗？
13. 可以说UDP加上RTP就是TCP吗？
14. 为什么RTP需要另一种协议，RTCP的服务，而TCP不需要？
15. 在图29.12中，Web服务器和媒体服务器可以运行在不同的机器上吗？
16. 本章中，我们只讨论了SIP用于音频的情况，它有什么缺点阻碍它用于视频吗？
17. 你认为H.323与SIP实际上是相同的吗？不同点是什么？试在两者之间做一下比较？
18. IP语音的完整实现存在哪些问题？你认为不久的将来我们会不再使用电话网吗？
19. H.323能够用于视频吗？

探究题

20. 找出RTCP发送方报告的格式，特别注意分组的长度以及对每一个源方重复的部分。试对每一个字段进行描述。
21. 找出RTCP接收方报告的格式，特别注意分组的长度以及对每一个源方重复的部分。试对每一个字段进行描述。
22. 找出RTCP源方描述报文的格式，特别注意分组的长度以及对每一个源方重复的部分。试对每一个字段进行描述。
23. 找出RTCP源方描述分组中的源方描述项的含义，特别是找出CNAME、NAME、EMAIL、PHONE、LOC、NOTE和PRIV。
24. 找出RTCP关闭报文的格式，特别注意分组的长度以及对每一个源方重复的部分。试对每一个字段进行描述。

第七部分 网络安全

目标

在数据通信与网络中，没有人会否认安全的重要性。互联网安全的基础是密码学，它是将报文变成安全的和免于攻击的科学与技术。密码学提供了有关通过网络交换报文的一些安全问题。这些问题是：保密性、完整性、鉴别性和不可否认性。

密码学提供保密性、完整性、鉴别性和不可否认性。

密码学也可用于报文的发送方与接收方彼此鉴别。例如，需要访问系统资源的用户首先应被授权，我们称这个问题为实体鉴别。

密码学也提供实体鉴别。

本书这部分，首先介绍密码学，但对它不深入研究数学基础。然后简要探讨网络应用安全性问题。最后，我们讨论因特网模型上三层的某些协议安全实现问题。

本书第七部分介绍不同的安全问题。

第30章 密码学

通过使用密码学（基于抽象代数的一门科学），网络安全的大部分能够得到实现。在本章中，我们简单讨论了适合本书范围的密码学，我们尽可能地减少有关抽象代数的讨论。我们的目标是给出密码学足够多的资料使得网络安全更加易懂。本章是第31章和第32章研究的基础知识。

30.1 引言

我们先介绍一些与密码学相关的问题。首先，需要定义一些术语，然后我们给出一些分类。

30.1.1 定义

这里，我们定义本章使用的一些术语。

密码学

密码学（Cryptography）这个词在希腊语中的意思是“秘密的文字”。然而，现在这个词指的是将报文转换成安全的、免受攻击的科学和艺术。图30.1显示的是密码学的组成。

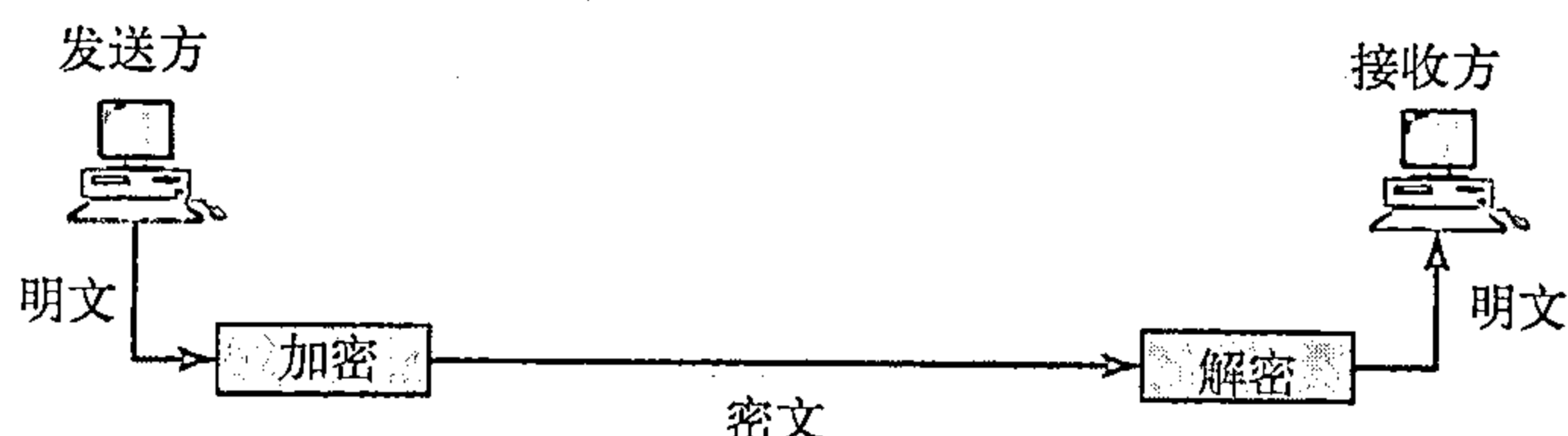


图30.1 密码学的组成

明文与密文

在进行转换之前，原始的报文称为明文（plaintext）。转换后的报文称为密文（ciphertext）。加密算法（encryption algorithm）将明文转换为密文；解密算法（decryption algorithm）将密文转换回明文。发送方使用加密算法，而接收方使用解密算法。

密码算法

我们把加密与解密算法合称为密码算法。术语密码算法（ciphers）有时也用来指密码学中不同类型的算法。这并不是说发送方和接收方在安全通信时都需要拥有自己唯一的加密算法，相反，一个密码算法可以为数百万对通信者服务。

密钥

密钥（key）是密码算法中参与运算的数值（或数值集）。对报文进行加密，我们需要一个加密算法、一个加密密钥以及明文，并由此产生密文。对报文进行解密，我们需要一个解密算法、一个解密密钥以及密文，并由此复原原始的明文。

Alice, Bob, and Eve

密码学中，在信息交换的情节里面，通常引入三个人。我们采用Alice、Bob和Eve。Alice是需要发送安全数据的人，Bob是数据接收方，Eve是个搅乱Alice和Bob通信的人，她截获报

文或者发送她自己伪造的报文。这三个名字代表的是实际发送或接收数据，或拦截或更改数据的计算机或者进程。

30.1.2 两类

我们可以将所有的加密算法（密码算法）分为两类：对称密钥（也称为秘密密钥）加密算法和非对称（公钥）加密算法。图30.2给出了分类。

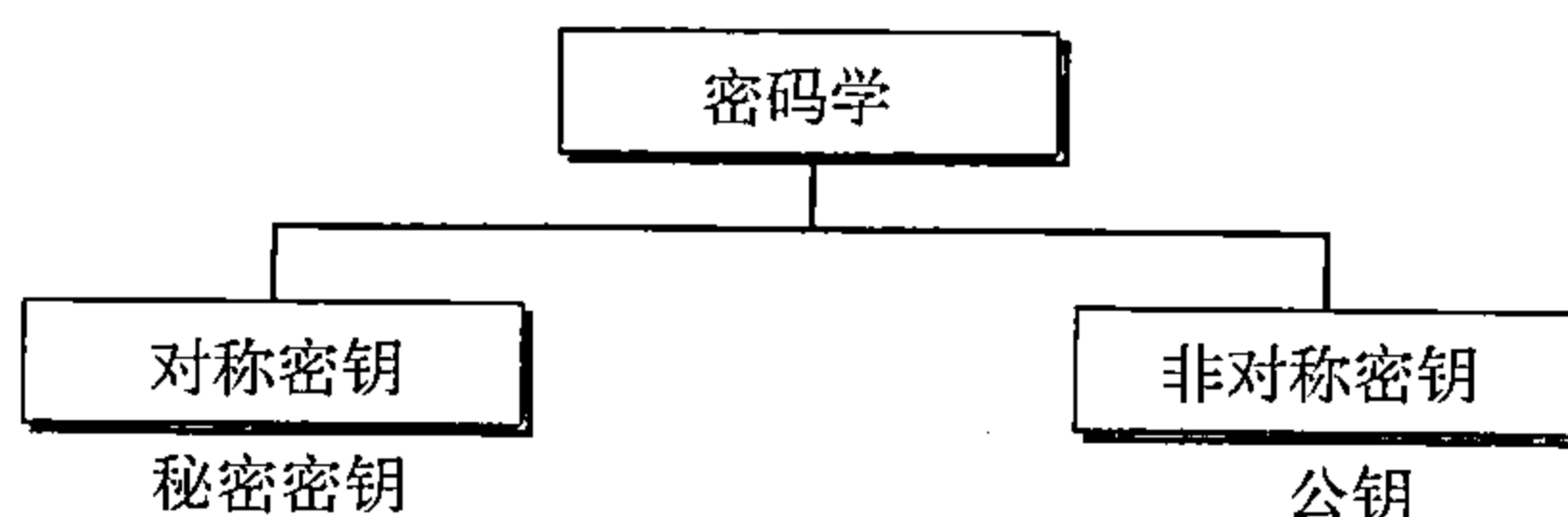


图30.2 密码学的分类

对称密钥密码学

在对称密钥密码学中，双方使用相同的密钥。发送方使用此密钥和加密算法对明文数据进行加密，接收方使用相同的密钥和相应的解密算法对密文进行解密（如图30.3所示）。

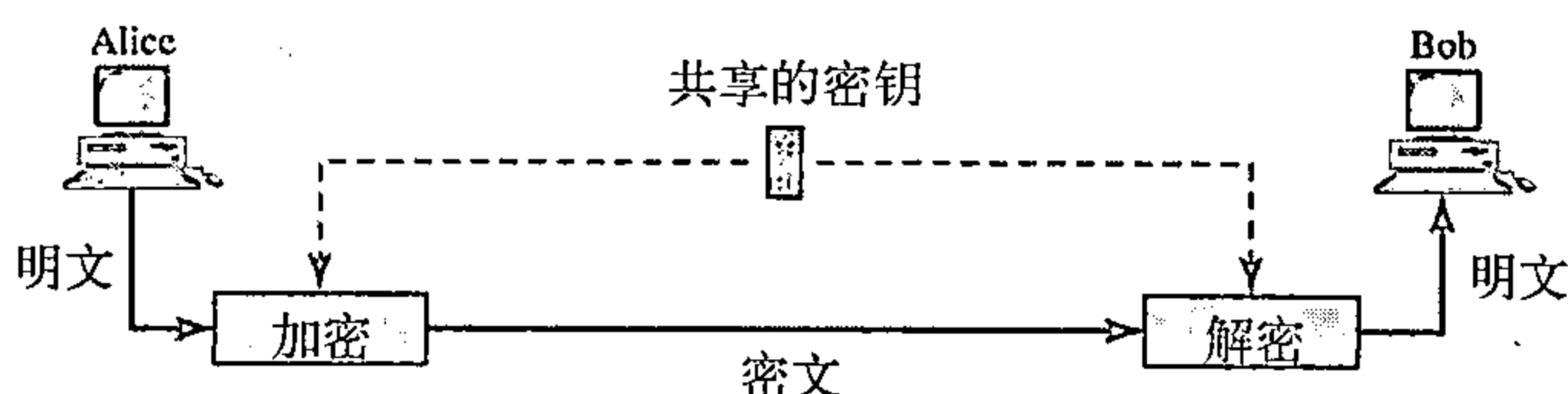


图30.3 对称密钥密码学

在对称密钥密码学中，发送方（加密）和接收方（解密）使用相同的密钥。密钥是共享的。

非对称密钥密码学

在非对称或公钥加密中有两个密钥：私钥和公钥。私钥（private key）由接收方保存，而公钥（public key）是公开发布的。如图30.4所示，假设Alice想给Bob发送报文。Alice使用公钥对报文加密。Bob接收到报文后，再用私钥进行解密。

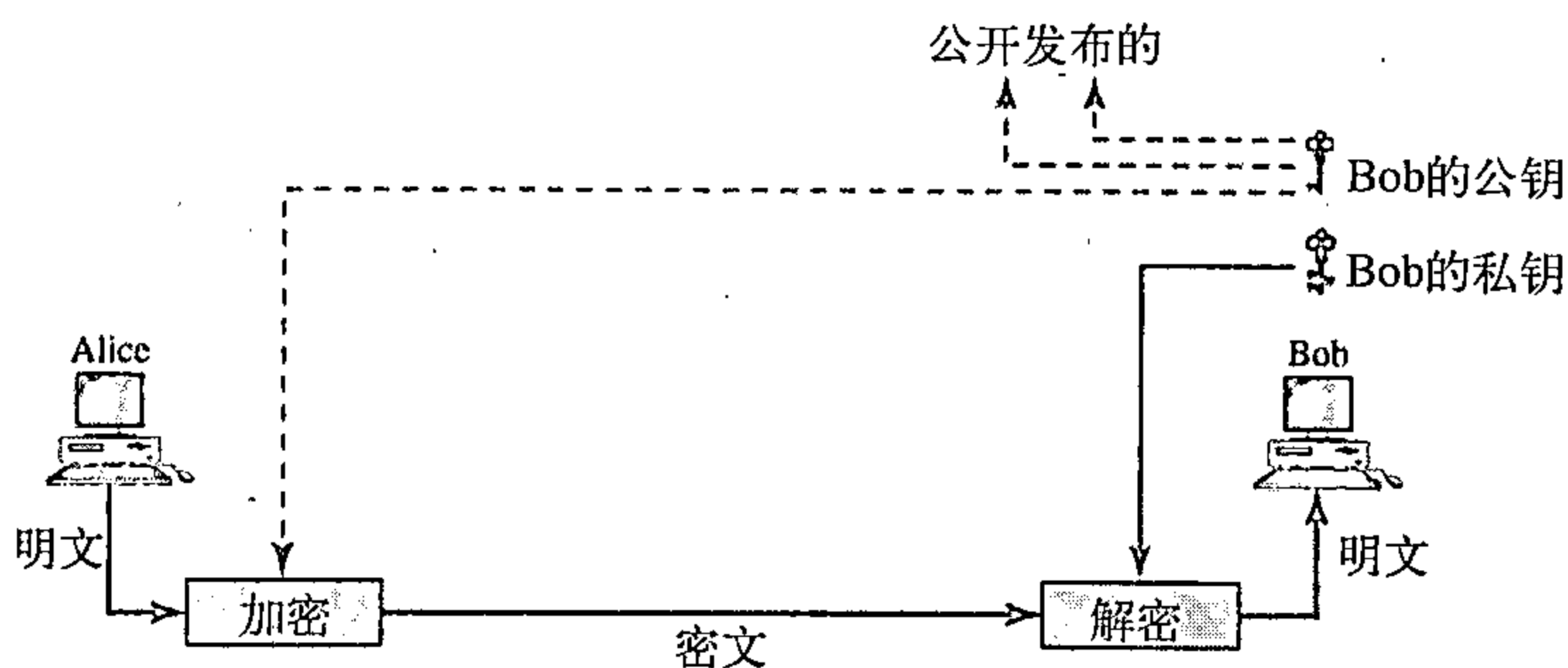


图30.4 非对称密码学

在公钥加密/解密中，用于加密的公钥与用于解密的私钥是不同的。公钥对于公众是可得到的，而私钥只有个人才能使用。

三种类型的密钥

读者应该已经注意到，在密码学中，我们要处理三种类型的密钥：秘密密钥、公钥及私钥。第一种秘密密钥是在对称密钥密码学中共享使用的；第二、三种分别是在非对称密钥密码学中使用的公钥和私钥。在本书中，我们使用三种不同的图标代表这三类密钥，将它们相互区分。如图30.5。

比较

我们比较一下对称与非对称密钥加密。加密可以想像为电子关锁；解密为电子开锁。发送方把报文放入盒子中，并使用密钥锁住盒子；接收方使用密钥打开盒子并取出报文。区别在于关锁与开锁的机制以及使用密钥的类型。

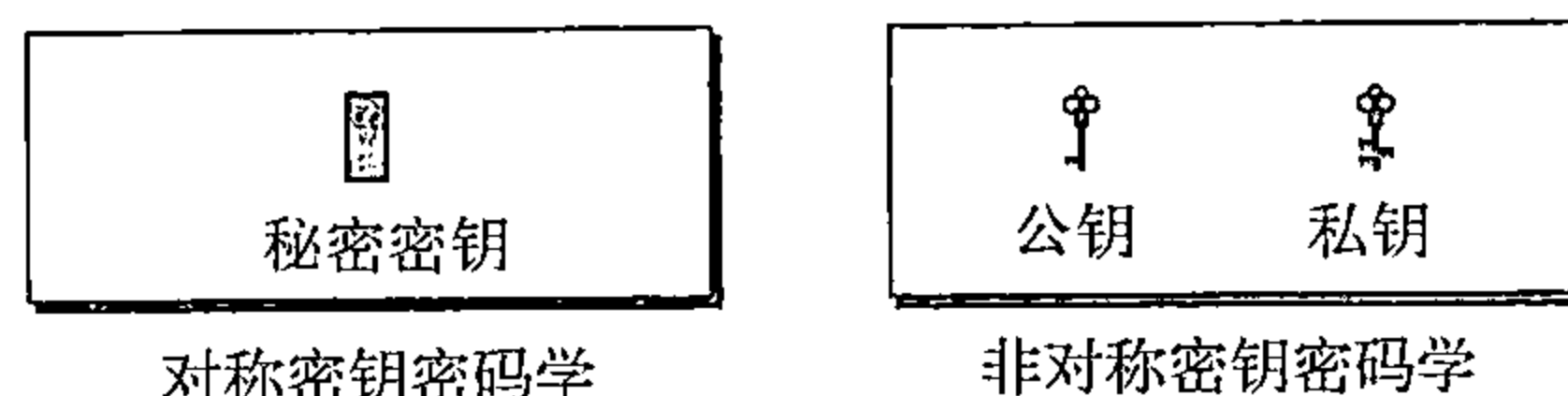


图30.5 在密码学中使用的密钥

在对称密钥密码学中，使用了相同的密钥对盒子关锁和开锁。在非对称密钥密码学中，用一个密钥锁住盒子，而用另外一个密钥打开盒子。如图30.6说明了它们之间的区别。

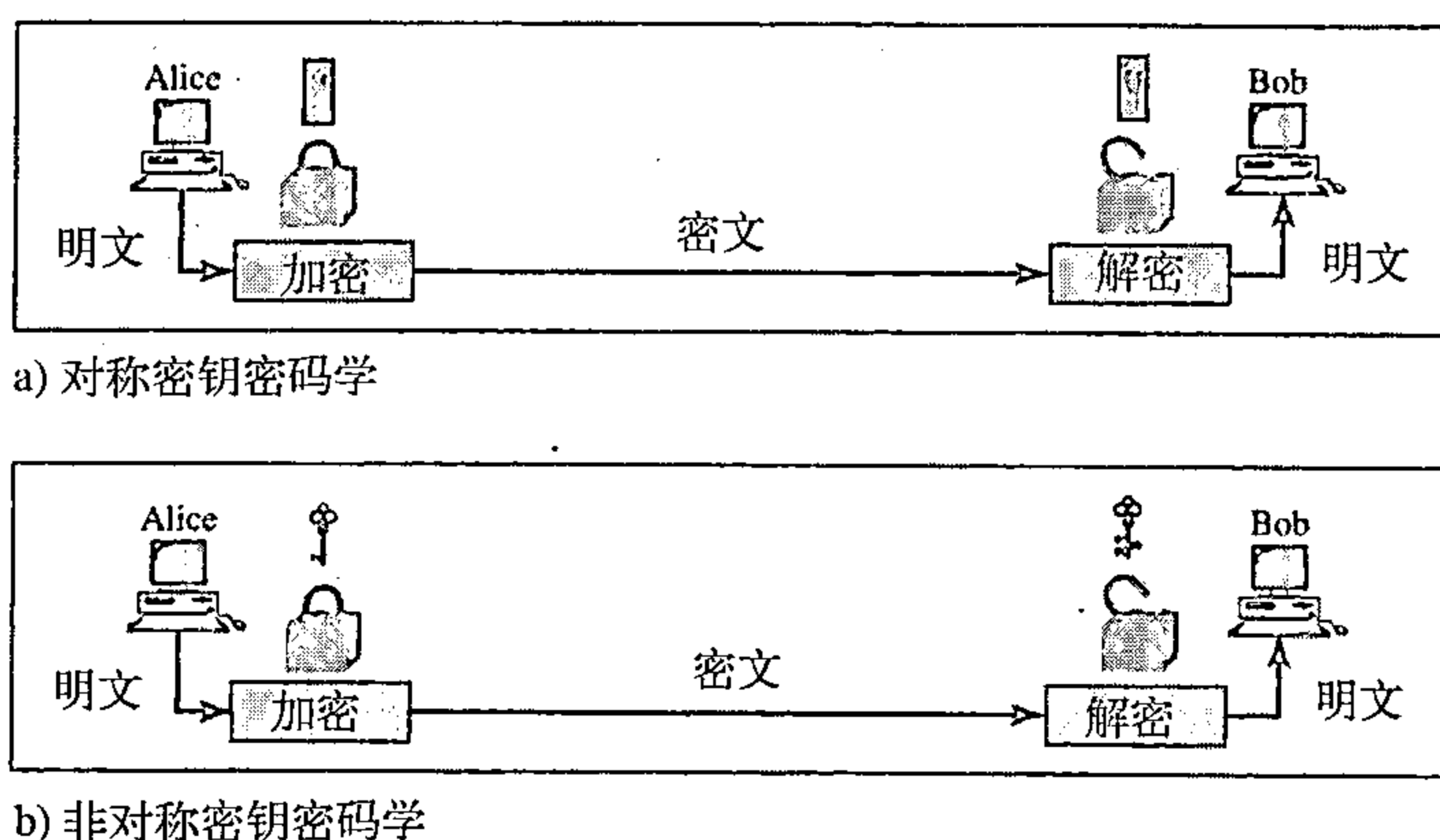


图30.6 两类密码学的比较

30.2 对称密钥密码学

几千年前，当人们需要交换秘密的时候（例如在战争中），对称密钥密码学就开始诞生了。在我们的网络安全中，我们仍主要使用对称密钥密码学；然而，现代的加密算法变得更为复杂。我们首先讨论传统的算法，它是面向字符的；然后我们讨论现代的算法，它是面向位的。

30.2.1 传统加密算法

我们简单地介绍一些传统的加密算法，这是面向字符的。尽管现在看来，这些传统加密算法已经过时，但我们主要的目的是为了说明现代的加密算法是怎样从传统加密算法中演变过来。我们可以把传统的对称密钥加密算法分成两大类：替换加密算法和换位加密算法，如图30.7所示。

替换加密算法

替换加密算法 (substitution cipher) 使用一个符号代替另一符号。如果明文中的符号是字母，就用一个字母代替另一个。例如，可以用字母D代替A，Z代替T。如果符号是数字（0~9），则可以用7代替3，6代替2。替换加密算法可以分为单字母替换和多字母替换加密算法。

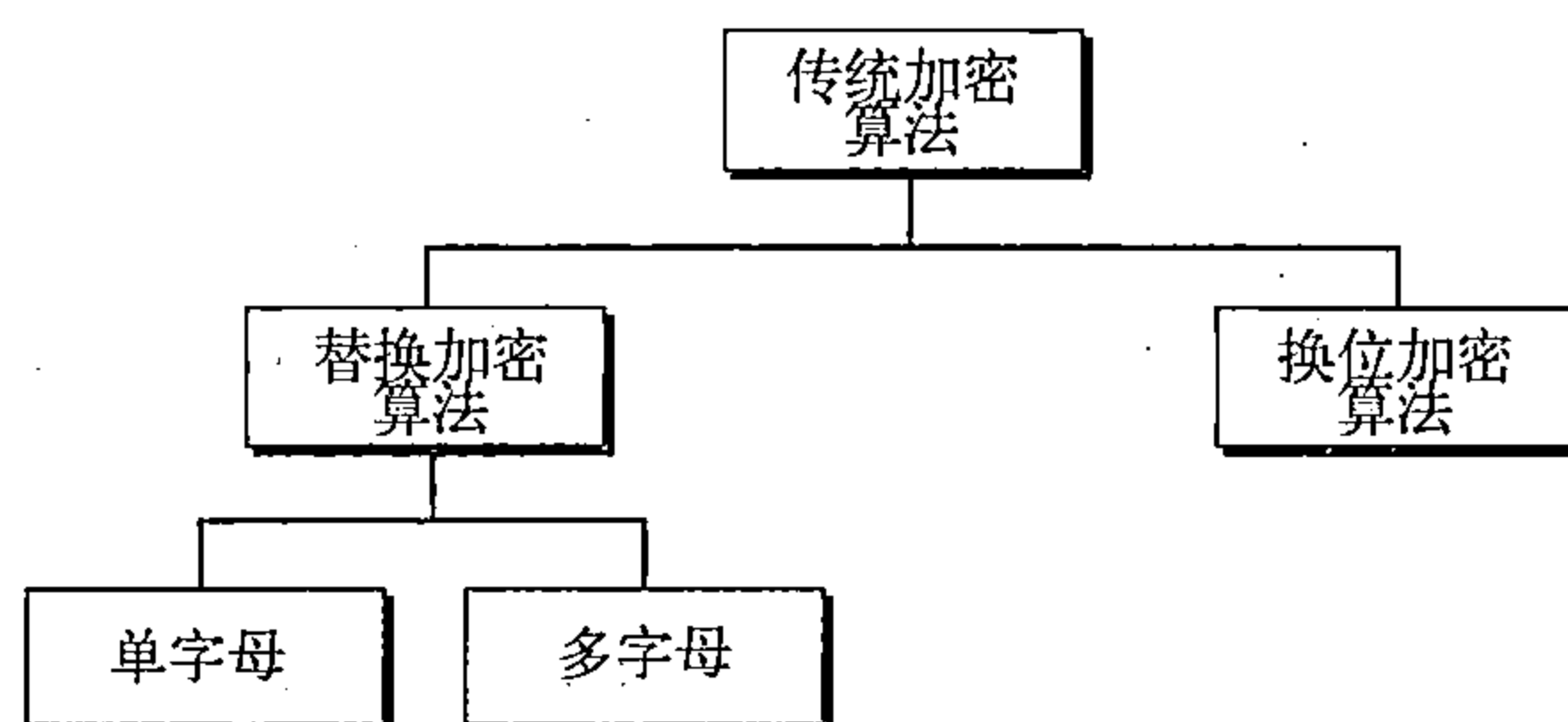


图30.7 传统加密算法

替换加密算法使用一个符号代替另一个符号。

在单字母替换加密算法 (monoalphabetic cipher) 中, 明文中的字符 (或符号) 不管其在文本中的位置, 在密文中总是替换为某个固定的字符 (或符号)。例如, 如果算法要求在明文中的字符A必须替换为D, 则每一个字符A都必须替换为字符D。换句话说, 明文和密文中的字符之间的关系是一对一的。

在多字母替换加密算法 (polyalphabetic cipher) 中, 明文中一个字符 (或符号) 的每次出现可以采用不同的字符 (或符号) 替换。明文中字符与密文中的字符之间的关系是一对多的关系。例如, 字符A在文本的开始可以改为字符D, 但在中间可以改为字符N。很明显, 如果明文字符和密文字符之间的关系是一对多的话, 那么密钥必须告诉我们诸多可能的字符中哪些可以用来加密。为了达到这个目的, 我们需要把文本分成多个字符组, 并使用一套密码集。例如, 我们可以把文本 “THISISANEASYTASK” 按每3个字符分为一组, 并使用3位密码集对其进行加密。然后我们对下3个字符重复这个加密过程。

例30.1

下面所示为明文和其相应的密文, 请问这是单字母替换加密算法吗?

Plaintext: HELLO

Ciphertext: KHOOR

解:

该加密算法可能是单字母替换加密。因为字符L出现的地方都加密为字符O。

例30.2

下面所示为明文和其相应的密文, 请问这是单字母替换加密算法吗?

Plaintext: HELLO

Ciphertext: ABNZF

解:

该加密算法不是单字母替换加密。因为每次字符L出现的地方都加密为不同的字符。第一个L加密为N; 第二个为Z。

移位加密 最简单的单字母替换加密算法可能就是移位加密 (shift cipher) 了。假设明文和密文中仅包含大写字母 (A到Z)。在这个加密过程中, 加密算法是“后移 key 位字符”, key 等于某一个数值; 解密算法就是“前移 key 位字符”。例如, 如果 key 为5, 加密算法就是“后移5位字符” (向字母结束的方向); 解密算法就是“前移5位字符” (向字母开始的方向)。当然, 如果到了字母的结束或开始的地方, 则进行环绕。

恺撒大帝使用移位加密与他的军官进行通信。正因为这个原因, 移位加密有时也称为恺撒密码 (Caesar cipher)。恺撒在他的通信中将 key 设为3。

移位加密有时也称为恺撒密码。

例30.3

使用移位加密算法, $key=15$, 加密报文 “HELLO”。

解:

我们一次加密一位字母, 每一位字母后移15位。字母H加密为W、字母E加密为T、第一个字母L加密为A、第二个字母L也加密为A、字母O加密为D。因此密文为 “WTAAD”。

例30.4

使用移位加密算法, $\text{key}=15$, 解密报文“WTAAD”。

解:

我们一次解密一位字母, 每一位字母前移15位。字母W解密为H、字母T解密为E、第一个字母A解密为L、第二个字母A也解密为A、最后字母D解密为O。因此明文为“HELLO”。

换位加密算法

在换位加密算法 (transposition cipher) 中, 字符不发生替换, 取而代之的是改变字符的位置。在明文中处于首位的字符可能出现在密文的第十位上; 而明文中第八位字符则可能出现在密文的首位。换句话说, 换位加密算法对符号分组中的符号进行重新排序。

换位加密算法对符号分组中的符号进行重新排序 (改变次序)。

密钥 在换位加密算法中, 密钥就是符号在明文和密文中位置的映射关系。例如, 下面说明了使用了4字符分组的密钥。

Plaintext: 2 4 1 3

Ciphertext: 1 2 3 4

经过加密, 我们把处于第二位的字符移位到第一位, 第四位的字符移位到第二位, 以此类推。解密时则相反。为了更有效率, 密钥需要更长, 这意味着可以对长数据分组进行加密和解密。图30.8说明了使用上面密钥的4字符分组的加密和解密。该图同时也说明了加密和解密使用了相同的密钥。加密采用了从上到下的移位, 而解密则采用从下往上的移位。

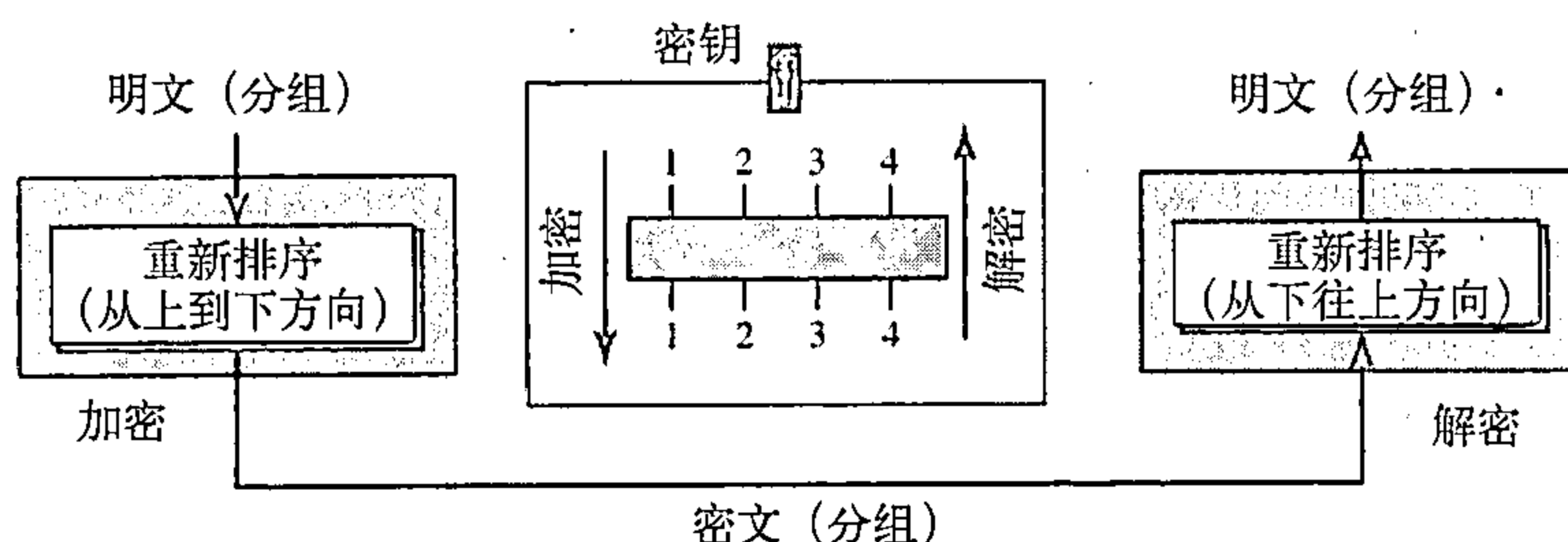


图30.8 移位加密算法

例30.5

使用上面的密钥对报文“HELLO MY DEAR”加密。

解:

我们首先把空格从报文中移除, 然后把文本按照4字符为一组进行分块; 在第三个分块的末尾处加上一个伪字符Z, 结果就是HELL OMYD EARZ。这三个分组的密文是ELHLMDOYAZER。

例30.6

利用例30.5, 解密报文“ELHLMDOYAZER”。

解:

结果是HELL OMYD EARZ, 去除伪字符并加入空字符后, 我们得到原始的报文“HELLO MY DEAR”。

30.2.2 简单的现代加密算法

到目前为止, 我们所学的传统加密算法都是面向字符的; 随着电脑的出现, 加密算法必须要面向位。这是因为需要加密的信息不仅仅是文本, 而且可能包含数字、图片、音频以及

视频数据。把这些类型的数据转换成位流是很方便的，对流进行加密，然后发送加密之后的位流。除此之外，当文本在位级进行处理的时候，每一位字符就被8位（或16位）替代，这意味着符号的数量变成了8个（或16个）。杂乱的位比杂乱的字符能够提供更多的安全。现代加密算法与传统算法相比，使用了不同的策略。现代的对称加密算法是简单加密算法的混合。换句话说，现代的加密算法使用了许多简单的加密算法来达到加密的目的。我们首先讨论这些简单的加密算法。

XOR加密算法

如今，现代的加密算法通常是由一系列的简单加密算法（simple ciphers）组成，这些都是在数学或计算机科学中预定义的简单函数。在这里第一个讨论的是异或加密算法（XOR cipher），因为它使用了计算机科学中定义的“异或”操作。图30.9说明了异或加密算法。

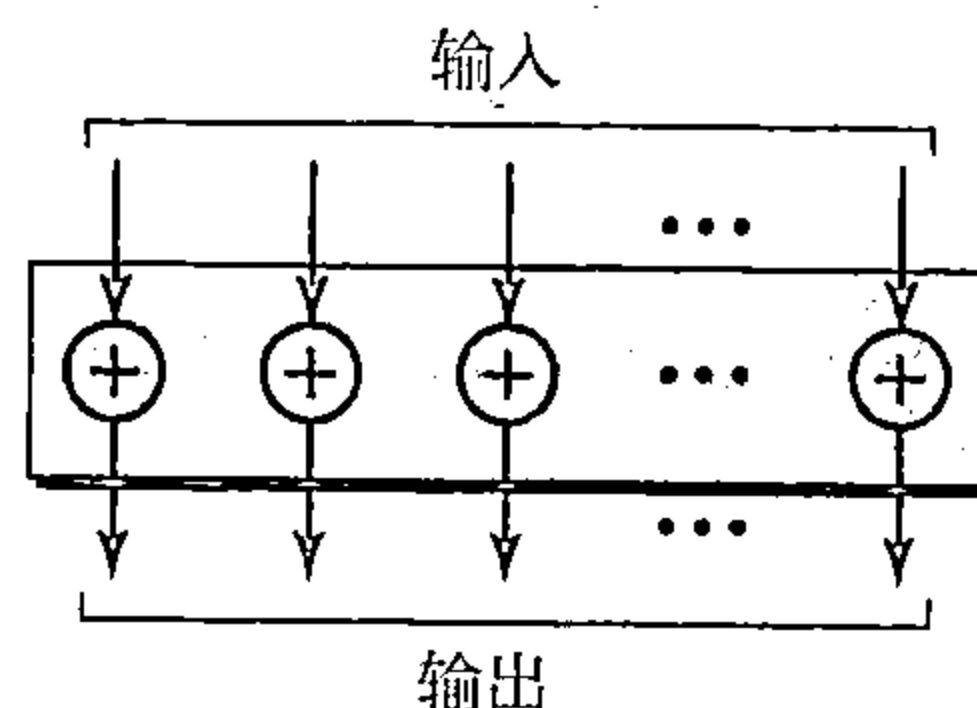


图30.9 异或加密算法

XOR操作需要两个数据输入，明文作为第一个数据，密钥作为第二个数据。换句话说，其中一个输入是需要加密的数据块，另一个输入就是密钥；运算后的结果就是加密后的数据块。注意：在XOR加密算法中，你可以发现密钥、明文及密文的长度是相同的；而且它还有一个非常有趣的属性：加密和解密是相同的。

旋转加密算法

另一个常用的加密算法是旋转加密算法（rotation cipher），在这算法中，输入的位被向左或向右进行旋转移动。旋转加密算法可以有密钥、也可以无密钥。有密钥，则密钥的值定义了旋转移动的次数；没有密钥，则旋转移动的次数是固定不变的。图30.10举例说明了一个旋转加密算法。注意：旋转加密算法可以认为是用位替代字符的替换加密算法的特殊案例。

旋转加密算法有一个有趣的属性。如果原始流的长度为 N ，经过 N 次旋转移动后，我们得到了原始的输入流。这意味着多于 $N-1$ 次的旋转移动是没有意义的，换句话说，旋转移动的次数必须介于1和 $N-1$ 之间。

旋转加密的解密算法使用了相同的密钥及相反的旋转方向。如果我们在加密中向右旋转，解密时则向左旋转，反之亦然。

替换加密算法：S-盒

S-盒（替换盒）与传统的面向字符的替换加密算法是类似的。S-盒的输入是长度为 N 的位流；输出的结果是另一串长度为 M 的位流。 N 与 M 并不一定需要相等。图30.11所示为S-盒。

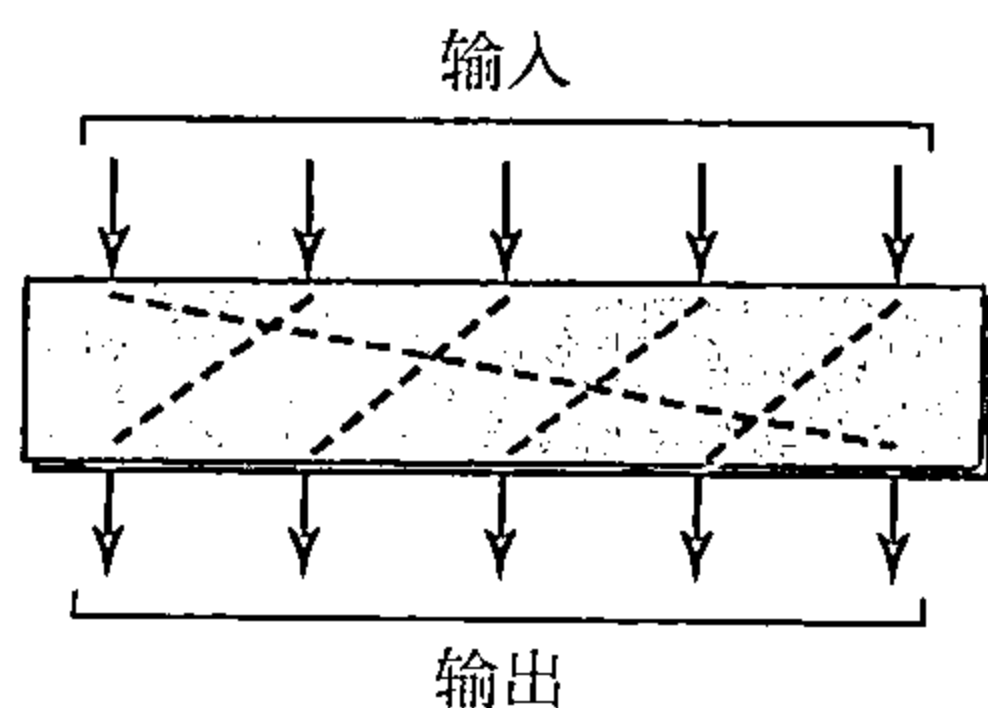


图30.10 旋转加密算法

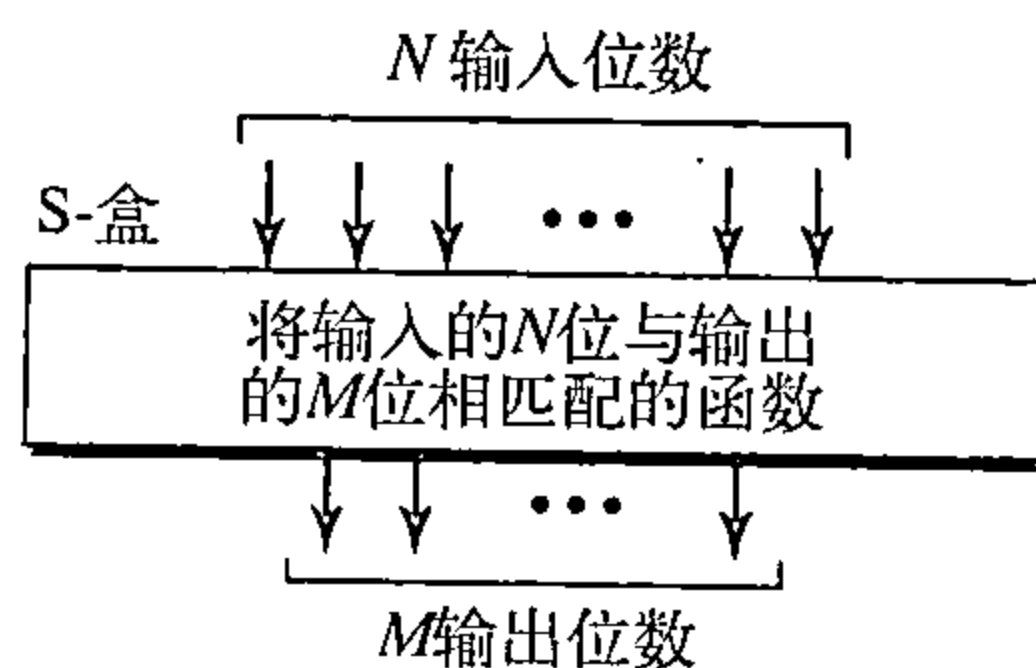


图30.11 S-盒

一般情况下，S-盒是没有密钥的，它作为加密或解密的一个中间过程。将输入与输出相匹配的函数可能是预定义的一个数学公式或者是一张表。

换位加密算法：P-盒

P-盒（置换盒）是位级的，与传统的面向字符的换位加密算法类似。它执行位级的换